

# **Análisis isogeométrico**



**Carlos Palomera Oliva**  
Trabajo de fin de grado de Matemáticas  
Universidad de Zaragoza

Noviembre de 2025



# Resumen

Se creará al finalizar el trabajo.



# Índice general

|                                                                       |            |
|-----------------------------------------------------------------------|------------|
| <b>Resumen</b>                                                        | <b>III</b> |
| <b>1. Introducción</b>                                                | <b>1</b>   |
| 1.1. Contexto y relación con el método de elementos finitos. . . . .  | 1          |
| 1.2. Evolución del análisis isogeométrico. . . . .                    | 2          |
| 1.3. Objetivos. . . . .                                               | 2          |
| 1.4. El Problema Modelo: La Ecuación de Poisson. . . . .              | 3          |
| 1.4.1. Formulación fuerte (o clásica). . . . .                        | 3          |
| 1.4.2. Formulación variacional (o débil). . . . .                     | 3          |
| 1.5. Estructura del documento. . . . .                                | 3          |
| <b>2. Construcción de estructuras mediante CAD</b>                    | <b>5</b>   |
| 2.1. Bézier . . . . .                                                 | 5          |
| 2.1.1. Funciones base . . . . .                                       | 5          |
| 2.1.2. Curvas . . . . .                                               | 6          |
| 2.2. B-Splines . . . . .                                              | 7          |
| 2.2.1. Funciones base . . . . .                                       | 7          |
| 2.2.2. Curvas . . . . .                                               | 8          |
| 2.3. NURBS . . . . .                                                  | 8          |
| 2.3.1. Funciones base . . . . .                                       | 8          |
| 2.3.2. Curvas . . . . .                                               | 8          |
| 2.4. Creación de superficies mediante el producto tensorial. . . . .  | 9          |
| 2.5. Refinamiento de nudos (h-refinement) . . . . .                   | 10         |
| 2.5.1. Implementación de la Inserción de Nudos . . . . .              | 11         |
| <b>3. Análisis isogeométrico.</b>                                     | <b>13</b>  |
| 3.1. Ensamblaje de matrices y vector de carga. . . . .                | 13         |
| 3.1.1. Caso 1D. . . . .                                               | 13         |
| 3.1.2. Caso 2D. . . . .                                               | 13         |
| 3.2. Imposición de condiciones de contorno. . . . .                   | 13         |
| 3.2.1. Tipo Dirichlet. . . . .                                        | 13         |
| 3.2.2. Tipo Neumann. . . . .                                          | 13         |
| 3.2.3. Tipo Robin o mixtas. . . . .                                   | 13         |
| 3.3. Convergencia en L2 y H1 y tiempo de computación teórico. . . . . | 13         |
| <b>4. Implementación computacional.</b>                               | <b>15</b>  |
| 4.1. Lenguaje y librerías utilizadas. . . . .                         | 15         |
| 4.2. Metodos numéricos empleados. . . . .                             | 15         |
| 4.3. Estructura del programa. . . . .                                 | 15         |

|                                                                                          |           |
|------------------------------------------------------------------------------------------|-----------|
| <b>5. Resultados.</b>                                                                    | <b>17</b> |
| 5.1. Resultados en 1D . . . . .                                                          | 17        |
| 5.2. Resultados en 2D . . . . .                                                          | 17        |
| <b>6. Conclusiones.</b>                                                                  | <b>19</b> |
| <b>Bibliografía</b>                                                                      | <b>21</b> |
| <b>Anexos</b>                                                                            | <b>23</b> |
| 6.1. Método de los elementos finitos. . . . .                                            | 23        |
| 6.2. Demmostraciones omitidas . . . . .                                                  | 26        |
| 6.2.1. Paso de solución fuerte a solución variacional . . . . .                          | 26        |
| 6.2.2. Existencia y unicidad de la solución variacional (Lema de Lax-Millgram) . . . . . | 26        |
| 6.2.3. Equivalencia de la solución de la formulación variacional y fuerte . . . . .      | 28        |
| 6.2.4. Equivalencia de normas . . . . .                                                  | 29        |
| 6.2.5. Condiciones del Lema de Céa . . . . .                                             | 30        |
| 6.2.6. Continuidad de la forma lineal dual . . . . .                                     | 30        |
| 6.2.7. Propiedades de los B-Splines . . . . .                                            | 31        |
| 6.2.8. Igualdad de las nuevas funciones base bajo refinamiento . . . . .                 | 33        |
| 6.2.9. Equivalencia de curvas bajo refinamiento . . . . .                                | 34        |
| 6.3. Código omitido . . . . .                                                            | 35        |
| 6.3.1. FEM . . . . .                                                                     | 35        |
| 6.3.2. Figuras de Bézier . . . . .                                                       | 37        |
| 6.3.3. Código fundamental para el funcionamiento en C++ . . . . .                        | 39        |
| 6.3.4. Curvas de NURBS . . . . .                                                         | 75        |
| 6.3.5. Superficies de NURBS . . . . .                                                    | 77        |

# Capítulo 1

## Introducción

### 1.1. Contexto y relación con el método de elementos finitos.

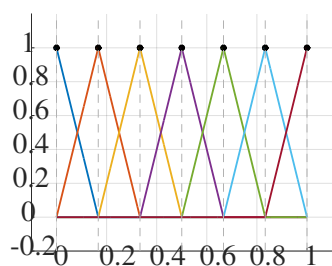
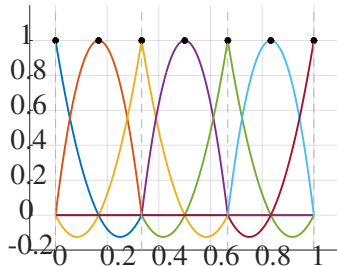
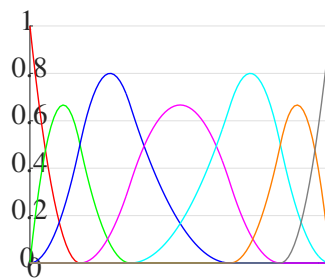
El análisis isogeométrico (IGA) es un método numérico, cuya finalidad se basa en la resolución de problemas de ecuaciones en derivadas parciales.

Se asemeja al método de elementos finitos (FEM), ambos buscan la solución mediante una combinación lineal de funciones base a partir de la formulación variacional, su principal diferencia con él radica en la preservación de la geometría original del espacio en el que se trabaja.

Cuadro 1.1: Comparación técnica entre FEM e IGA.

| Criterio          | FEM Clásico                                                 | Análisis Isogeométrico (IGA)                          |
|-------------------|-------------------------------------------------------------|-------------------------------------------------------|
| Geometría         | Aproximada. Malla poligonal que introduce error geométrico. | Exacta. Geometría NURBS idéntica al CAD.              |
| Funciones de Base | Interpolatorias (Nodos).                                    | NURBS (Puntos de Control).                            |
| Continuidad       | Baja ( $C^0$ ).                                             | Alta ( $C^{p-1}$ ).                                   |
| Refinamiento      | $h$ y $p$ .                                                 | $h$ , $p$ y $k$ -refinamiento.                        |
| Coste Ensamblaje  | Económico. Cuadratura sencilla.                             | Elevado. Requiere reglas de cuadratura mas complejas. |
| Mapeo Inverso     | Directo. Inversión simple o analítica.                      | Complejo. aumento en el coste computacional.          |

*Nota:* Si se desea conocer mas en detalle los distintos elementos del FEM para comparar más meticulosamente ambos métodos refierase al anexo 6.1.

(a) Funciones base FEM,  $p=1$ .(b) Funciones base FEM,  $p=2$ .(c) Funciones base NURBS,  $p=2$ .

*Nota:* El código fuente de estas figuras se encuentra disponible en los anexos 6.3.1 y 6.3.4.

## 1.2. Evolución del análisis isogeométrico.

Para comprender el IGA, es necesario comprender la evolución del diseño asistido por ordenador (CAD), en **1967** Ahlberg, Nilson y Walsh publican su obra *Theory of Splines and Their Applications*, pionera en formalizar la teoría de los **splines**.

Con la llegada de la industria de automoción y aeronáutica en los 70 y 80, comienza la necesidad de representar formas complejas de manera eficiente impulsando así el CAD.

Durante esta época se estandarizaron las técnicas y los algoritmos esenciales que posteriormente serían las bases del IGA.

Paralelamente a este proceso, en los años 50 y 60, impulsado por la industria aeroespacial, con el fin de resolver ecuaciones en derivadas parciales en dominios complejos, surgió el FEM, ignorando el CAD y creando una brecha entre ellos que duró décadas.

En una búsqueda de unificar el diseño asistido por ordenador (CAD) y el análisis asistido por ordenador (CAE), tratando de eliminar la necesidad de crear una conversión geométrica que afecta a la precisión, nace el análisis isogeométrico.

Actualmente IGA se usa en diferentes industrias como: **automoción**, simulación de choques, **aeroespacial**, diseño de álabes de turbinas, **biomedicina**, simulación de flujo sanguíneo, **eólica**, Interacción fluido-estructura en las palas.

Finalmente mencionamos los campos de estudio en los que se avanza actualmente en referencia a IGA como son los **T-Splines**, para un mayor refinamiento local, y los métodos **espacio-temporales**, campo actualmente en investigación.

## 1.3. Objetivos.

El objetivo principal del trabajo es entender el marco teórico y las ventajas del IGA frente al FEM, tanto en costo computacional como en orden de convergencia, en problemas de contorno de ecuaciones en derivadas parciales.

Con el fin de cumplir este objetivo, este se segmentará en los siguientes puntos que se seguirán en el orden siguiente:

1. **Marco teórico:** Establecer el marco teórico del IGA, contrastando la formulación variacional basada en las funciones NURBS con la aproximación polinómica del FEM.
2. **Implementación computacional:** Desarrollo e implementación de código propio que pueda resolver problemas de contorno de edps usando el IGA.
3. **Validación y comparación:** Validar los resultados numéricos comparándolos con las soluciones analíticas de los problemas y comprobar las diferencias en velocidad de convergencia con el FEM.



## 1.4. El Problema Modelo: La Ecuación de Poisson.

Durante la realización del trabajo nos centraremos en el problema de Poisson como benchmark para mostrar en este las propiedades del IGA.

### 1.4.1. Formulación fuerte (o clásica).

**Definición.** Sea  $\Omega \subset \mathbb{R}^n$  un conjunto abierto y acotado, donde  $n \in \mathbb{N}$ , con frontera  $\partial\Omega = \Gamma = \overline{\Gamma_D \cup \Gamma_R \cup \Gamma_N}$ , que sea suficientemente regular, tal que  $\Gamma_D \cap \Gamma_R = \emptyset$ ,  $\Gamma_D \cap \Gamma_N = \emptyset$ ,  $\Gamma_R \cap \Gamma_N = \emptyset$ .

Buscamos una función solución  $u : \Omega \rightarrow \mathbb{R}$ , que cumpla  $u \in C^2(\Omega)$  y tal que:

$$\begin{cases} -\Delta u = f & \text{en } \Omega, \\ u = g & \text{en } \Gamma_D, \quad (\text{Condición de Dirichlet}), \\ \nabla u \cdot \mathbf{n} = h & \text{en } \Gamma_N, \quad (\text{Condición de Neumann}), \\ \alpha u + \beta \nabla u \cdot \mathbf{n} = r & \text{en } \Gamma_R, \quad (\text{Condición de Robin}). \end{cases}$$

Donde las formas bilineal  $a(\cdot, \cdot)$  y lineal  $L(\cdot)$  se definen como:

- $f : \Omega \rightarrow \mathbb{R}$ ,  $f \in L^2(\Omega)$ .
- $g : \Gamma_D \rightarrow \mathbb{R}$ ,  $g \in H^{1/2}(\Gamma_D)$ .
- $h : \Gamma_N \rightarrow \mathbb{R}$ ,  $h \in L^2(\Gamma_N)$ .
- $r : \Gamma_R \rightarrow \mathbb{R}$ ,  $r \in L^2(\Gamma_R)$ .
- $\mathbf{n} : \Gamma \rightarrow \mathbb{R}^n$ , vector normal exterior unitario a la frontera  $\Gamma$ .
- $\alpha, \beta \in \mathbb{R}$ , tal que  $\frac{\alpha}{\beta} \geq 0$ .

### 1.4.2. Formulación variacional (o débil).

**Definición.** Buscamos  $u \in V(\Omega) = \{v \in H^1(\Omega) \mid v = g \text{ en } \Gamma_D\}$ , tal que para  $v \in V_0(\Omega) = \{v \in H^1(\Omega) \mid v = 0 \text{ en } \Gamma_D\}$ , se cumple que:

$$a(u, v) = L(v). \quad (1.1)$$

Donde:

- $a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma.$
- $L(v) = \int_{\Omega} f v d\Omega + \int_{\Gamma_N} h v d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma.$

*Nota:* El desarrollo detallado para obtener (1.1) a partir de la formulación fuerte, la demostración de existencia y unicidad de la solución variacional, así como la prueba de que esta (bajo suficiente regularidad) es también solución del problema fuerte, se detallan en los Anexos 6.2.1, 6.2.2 y 6.2.3.

## 1.5. Estructura del documento.

resumen rapido de los posteriores capitulos asi como explicación de donde apareceran el codigo o diferentes elementos en el anexo.

Se creará al finalizar el trabajo.



## Capítulo 2

# Construcción de estructuras mediante CAD

Las curvas y superficies creadas mediante **NURBS** son la base y el estándar del diseño CAD, durante esta sección introduciremos los conceptos necesarios para entender su funcionamiento e implementación siguiendo la progresión natural e histórica de las siguientes curvas:

$$\text{Bézier} \longrightarrow \text{B-Splines} \longrightarrow \text{NURBS}.$$

### 2.1. Bézier

En los años 60, impulsados por la industria de la automoción francesa y su necesidad de pasar los diseños en arcilla y papel a ordenador. Pierre Bézier y Paul Casteljau desarrollan la manera de crear curvas parametrizadas mediante el uso de **polinomios de Bernstein** como funciones base y **puntos de control**.

#### 2.1.1. Funciones base

**Definición.** Los polinomios de Bernstein se definen como:

$$B_{i,p}(u) = \binom{p}{i} u^i (1-u)^{p-i}.$$

**Proposición 2.1.** Algunas de las propiedades fundamentales de los **polinomios de Bernstein** en este contexto son:

- **No Negatividad:**  $B_{i,p}(u) \geq 0, \forall u \in [0, 1]$ .
- **Partición de la Unidad:**  $\sum_{i=0}^p B_{i,p}(u) = 1, \quad \forall u \in [0, 1]$ .
- **Interpolación de los extremos:**  $B_{0,p}(0) = 1, \quad B_{p,p}(1) = 1$ .
- **Soporte Global:**  $B_{i,p}(u) > 0, \quad \forall u \in (0, 1)$ .

*Demostración.* Demostraciones omitidas temporalmente. □

### 2.1.2. Curvas

**Definición.** Las curvas de Bézier de grado  $p$  son curvas parametrizadas en el espacio  $\mathbb{R}^n$  con  $n, p \in \mathbb{N}$ , de la forma:

$$\mathbf{C}(u) = \sum_{i=0}^p B_{i,p}(u) \mathbf{P}_i, \quad u \in [0, 1].$$

Donde:

- $\{\mathbf{P}_0, \mathbf{P}_1, \dots, \mathbf{P}_p\}^1 \subset \mathbb{R}^n$ , es el conjunto formado por los **puntos de control**.

**Proposición 2.2.** Sea  $\mathbf{C}(u) = \sum_{i=0}^p B_{i,p}(u) \mathbf{P}_i$  una curva de Bézier de grado  $p$ , con  $u \in [0, 1]$ . Se satisfacen las siguientes propiedades:

1. **Interpolación en los extremos:**  $\mathbf{C}(0) = \mathbf{P}_0$  y  $\mathbf{C}(1) = \mathbf{P}_p$ .
2. **Tangencia en los extremos:**

$$\mathbf{C}'(0) = p(\mathbf{P}_1 - \mathbf{P}_0), \quad \mathbf{C}'(1) = p(\mathbf{P}_p - \mathbf{P}_{p-1}).$$

*Implicación:* La continuidad  $C^1$  entre dos curvas de Bézier unidas requiere la colinealidad de  $\mathbf{P}_{p-1}, \mathbf{P}_p$  (primera curva) y  $\mathbf{P}_1$  (segunda curva).

3. **Invarianza afín:** Sea  $\Phi: \mathbb{R}^n \rightarrow \mathbb{R}^n$  una transformación afín ( $\Phi(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$ ). Entonces:

$$\Phi\left(\sum_{i=0}^p B_{i,p}(u) \mathbf{P}_i\right) = \sum_{i=0}^p B_{i,p}(u) \Phi(\mathbf{P}_i).$$

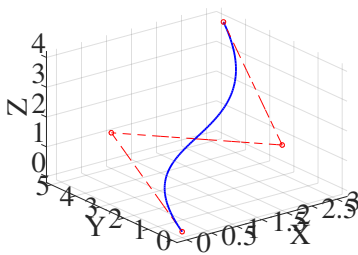
*Demostración.* Demostraciones omitidas temporalmente. □

Uno de los principales inconvenientes de las curvas de Bézier es su incapacidad de representar cónicas de forma exacta, para ello se introduce el concepto de racionalizarlas.

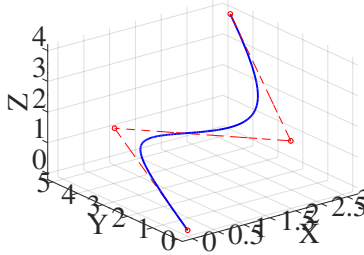
**Proposición 2.3.** Racionalización de las curvas de Bézier.

Se le asigna un peso  $w_i \in \mathbb{R}^+$  a cada punto de control  $\mathbf{P}_i$ , de manera que elevamos cada uno a coordenadas homogéneas en el **espacio proyectivo**  $\mathbb{P}^n$ <sup>2</sup> y proyectando de vuelta al espacio euclídeo  $\mathbb{R}^n$ .

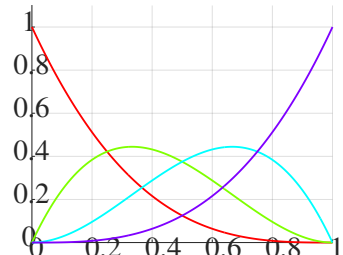
$$\mathbf{C}^R(u) = \frac{\sum_{i=0}^p B_{i,p}(u) w_i \mathbf{P}_i}{\sum_{i=0}^p B_{i,p}(u) w_i}$$



(a) Curva Bézier no racional.



(b) Curva racional,  $W = (1, 10, 7, 1)^T$ .



(c) Funciones base de Bernstein.

*Nota:* El código fuente para la generación de estas figuras se encuentra disponible en el anexo 6.3.2.

<sup>1</sup>El conjunto de puntos de control puede contener repeticiones, es decir, **NO** satisfacen necesariamente  $\mathbf{P}_i \neq \mathbf{P}_j$ , si  $i \neq j$ .

<sup>2</sup>Sea  $\sim$  una relación de equivalencia, donde  $\mathbf{X} \sim \mathbf{Y}$  si  $\mathbf{Y} = \lambda \mathbf{X}$ , con  $\lambda \in \mathbb{R} \setminus \{0\}$ . El **espacio proyectivo** se define como  $\mathbb{P}^n = (\mathbb{R}^{n+1} \setminus \{\mathbf{0}\}) / \sim$ .

## 2.2. B-Splines

A pesar de que la racionalización de las curvas de Bézier permitió la representación de cónicas exactas, hubo otros problemas intrínsecos a estas que motivaron la transición en los años 70 debido a la necesidad de representar estructuras mas complejas en la industria aeroespacial y de automoción, se transiciona entonces a la implementacion de funciones base **B-spline** que permitian, debido a tener un soporte local y un número de elemtos no ligado obligatoriamente al grado polinomial de estas, superar las limitaciones de las funciones base de Bernstein.

### 2.2.1. Funciones base

**Definición.** El **Vector de nudos** se define como

$$U = \{u_0, u_1, \dots, u_m\},$$

cumplen  $u_i \leq u_{i+1}$ ,  $i = 0, \dots, m-1$ , el número de nudos  $m+1$  está relacionado con  $p$  y  $n$  tal que cumple:  $m = n + p + 1$ .

A lo largo del trabajo nos enfocaremos en los vectores de nudos **abiertos**, es decir, aquellos cuyos extremos se repiten  $p+1$  veces,  $u_0 = u_1 = \dots = u_p$ ,  $u_m = u_{m-1} = \dots = u_{m-p+1}$ .

**Definición.** Las **Funciones base B-Spline**,  $N_{i,p}(u)$ : se definen recursivamente mediante el **algoritmo de Cox-de Boor**:

- Caso base ( $p = 0$ ):

$$N_{i,0}(u) = \begin{cases} 1 & \text{si } u_i \leq u < u_{i+1}, \\ 0 & \text{en caso contrario.} \end{cases}$$

- Caso genérico ( $p \geq 1$ ):

$$N_{i,p}(u) = \frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u)^3.$$

**Proposición 2.4.** Algunas de las propiedades importantes de las funciones base B-Splines son:

1. **Soporte Local:** Cada función base  $N_{i,p}(u)$  es no nula solo en un número limitado de intervalos de nudo, concretamente en  $[u_i, u_{i+p+1})$ . En consecuencia la curva adquiere la propiedad de **control local**: mover un punto de control  $\mathbf{P}_i$  solo altera la forma de la curva en una región limitada.
2. **No Negatividad:** El valor de las funciones base B-Spline es siempre no negativo en todo el dominio paramétrico, es decir,  $\forall u \in [u_0, u_m]$ ,  $N_{i,p}(u) \geq 0$ .
3. **Número de Bases Activas:** Para cualquier intervalo de nudos  $[u_i, u_{i+1})$ , las únicas funciones base  $N_{j,p}(u)$  cuyo soporte contiene dicho intervalo son aquellas con el índice:

$$j \in \{i-p, i-p+1, \dots, i\}$$

Por tanto el número de funciones base no nulas en el intervalo es a lo sumo  $p+1$ .

El número exacto de funciones base no nulas decrece con la multiplicidad  $k$  de  $u_i$ , reduciéndose a  $p+1-k$ .

4. **Partición de la Unidad:**  $\forall u \in [u_0, u_m]$ , se cumple,  $\sum_{i=0}^n N_{i,p}(u) = 1$ .

**Demostración.** La demostración puede encontrarse en el anexo 6.2.7. □

<sup>3</sup>Para el caso en el que los coeficientes toman valor  $\frac{0}{0}$  se les asignará como valor 1.

### 2.2.2. Curvas

**Definición.** Con estas definiciones podemos ahora definir la curva generada por los B-Splines:

$$C(u) = \sum_{i=0}^n N_{i,p}(u) \cdot \mathbf{P}_i.$$

**Proposición 2.5.** *Algunas de las propiedades importantes de las curvas B-Splines son:*

1. **Invarianza bajo Transformaciones Afines:** *La aplicación de una transformación afín al polígono de control resulta en la misma transformación aplicada directamente a la curva generada.*
2. **Continuidad Controlable:** *La continuidad de la curva en cualquier unión (nudo) se controla localmente mediante la multiplicidad  $k^4$  del nudo, sin depender de la manipulación compleja de los puntos de control. La continuidad es  $C^r$ , donde  $r = p - k$ .*

*Demostración.* La demostración puede encontrarse en el anexo 6.2.7. □

## 2.3. NURBS

La transición de **B-splines** a **NURBS** no fue instantánea, se tuvo que esperar hasta **1979**, donde Boeing sufría las consecuencias de no poder representar cónicas exactas mediante B-splines, teniendo así su software segmentado, usando un programa basado en B-splines para el fuselaje y otro basado en las **cónicas de Rho** para las partes cónicas, siendo incapaces de conectarlos sin perder precisión.

En este caos Boeing inicia el proyecto **TIGER**, reescribiendo así todo su sistema CAD interno basándose en la tesis de **Verspille**, proponiendo así en **1981** a la IGES que las **NURBS** sean el estandar universal, transicionando así la industria de manera natural entre **1983** y **1990**.

### 2.3.1. Funciones base

**Definición.** Las **Funciones base NURBS** se definen como:

$$R_{i,p}(u) = \frac{N_{i,p}(u)w_i}{\sum_{j=0}^n N_{j,p}(u)w_j}.$$

**Proposición 2.6.** *Con el fin de preservar todas las propiedades de las **funciones base B-spline** dadas en 2.4, nos restringiremos al caso  $w_i > 0, \forall i$ .*

### 2.3.2. Curvas

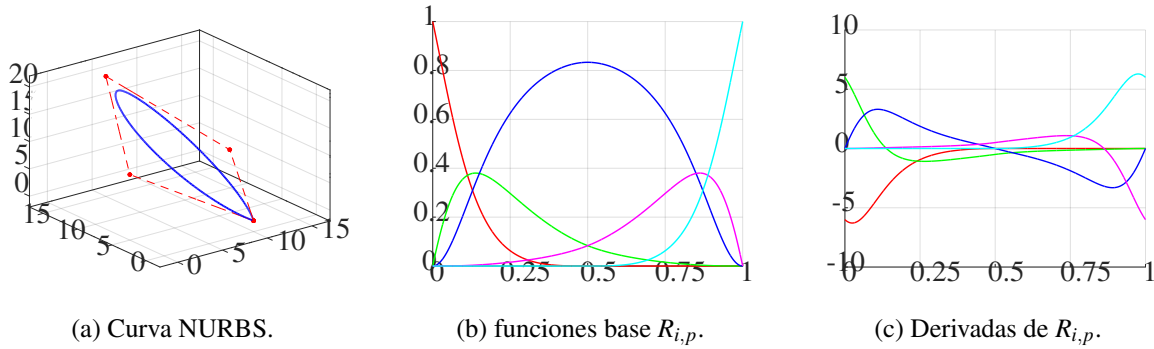
**Definición.** Podemos ahora definir las curvas generadas por NURBS como:

$$C(u) = \sum_{i=0}^n R_{i,p}(u) \cdot \mathbf{P}_i = \frac{\sum_{i=0}^n N_{i,p}(u)w_i \mathbf{P}_i}{\sum_{j=0}^n N_{j,p}(u)w_j}.$$

**Proposición 2.7.** *Debido a la restricción impuesta anteriormente ( $w_i > 0, \forall i$ ), las **curvas NURBS** también mantienen las propiedades de las **curvas B-spline** dadas en 2.5.*

---

<sup>4</sup>La multiplicidad  $k$  de un nudo  $u_i$  es el número de veces que el valor  $u_i$  aparece en el vector de nudos  $U$ , debe cumplirse que  $k \leq p$  en los nudos interiores.



*Nota:* El código fuente para la generación de estas figuras se encuentra disponible en el anexo 6.3.4.

## 2.4. Creación de superficies mediante el producto tensorial.

El producto tensorial es el estándar de CAD para la creación de superficies, se define como:

$$\mathbf{B}_{i,j}(u, v) = N_{i,p}(u) \cdot M_{j,q}(v).$$

Donde  $N_{i,p}(u)$  son las funciones base en la dirección de  $u$  y  $M_{j,q}(v)$  son las funciones base en la dirección de  $v$ .

Mediante este producto definimos la superficie generada por **B-Splines** mediante la fórmula:

$$\mathbf{S}(u, v) = \sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u) M_{j,q}(v) \mathbf{P}_{i,j}.$$

Redefiniendo la estructura en la que se encuentran los puntos de control, pasando a formar ahora una matriz.

Podemos ahora racionalizar estas curvas para obtener superficies generadas mediante **NURBS**, para ello utilizamos una matriz de pesos  $w_{i,j}$  y la fórmula:

$$\mathbf{S}(u, v) = \frac{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u) M_{j,q}(v) w_{i,j} \mathbf{P}_{i,j}}{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u) M_{j,q}(v) w_{i,j}}.$$

**Proposición 2.8.** Las superficies generadas mediante el producto tensorial mantienen la suavidad de las curvas que la generan en las direcciones de estas.

*Demostración.* Tomaremos una dirección arbitraria,  $v$  en este caso, y fijaremos  $u = u_0$ , la demostración es análoga para la dirección  $u$ .

$$\mathbf{S}(u_0, v) = \frac{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u_0) M_{j,q}(v) w_{i,j} \mathbf{P}_{i,j}}{\sum_{i=0}^n \sum_{j=0}^m N_{i,p}(u_0) M_{j,q}(v) w_{i,j}}.$$

Ahora reagrupamos los valores que dependen de  $v$  y los que no:

$$\mathbf{S}(u_0, v) = \frac{\sum_{j=0}^m M_{j,q}(v) (\sum_{i=0}^n N_{i,p}(u_0) w_{i,j} \mathbf{P}_{i,j})}{\sum_{j=0}^m M_{j,q}(v) (\sum_{i=0}^n N_{i,p}(u_0) w_{i,j})}.$$

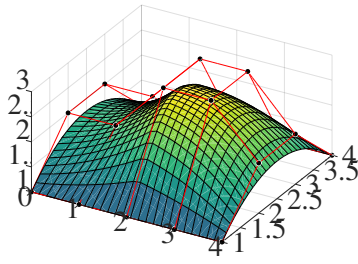
Definimos ahora los siguientes elementos:

$$\mathbf{w}_j^* = \sum_{i=0}^n N_{i,p}(u_0) w_{i,j}, \quad \mathbf{Q}_j = \frac{\sum_{i=0}^n N_{i,p}(u_0) \mathbf{P}_{i,j} w_{i,j}}{\mathbf{w}_j^*}.$$

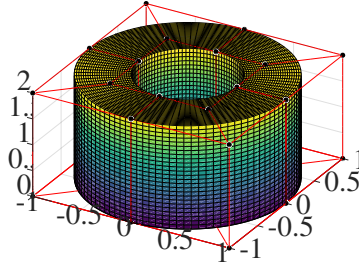
Obteniendo al final una definición

$$\mathbf{C}(v) = \mathbf{S}(u_0, v) = \frac{\sum_{j=0}^m M_{j,q}(v) \mathbf{w}_j^* \mathbf{Q}_j}{\sum_{j=0}^m M_{j,q}(v) \mathbf{w}_j^*}.$$

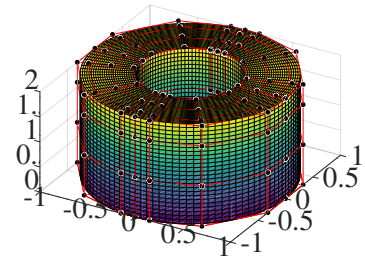
Obteniendo así la curva de NURBS asociada a la dirección  $v$  en el punto  $u_0$ . □



(a) Superficie NURBS ejemplo 1.



(b) Superficie NURBS ejemplo 2.



(c) Ejemplo 2 refinada.

*Nota:* El código fuente para la generación de estas figuras se encuentra disponible en el anexo 6.3.5.

## 2.5. Refinamiento de nudos (h-refinement)

En las figuras anteriores podemos observar un refinamiento de la superficie del ejemplo 2, este procedimiento se basa en insertar nuevos valores en un vector de nudos, introduciendo nuevos puntos de control y pesos para mantener la misma figura geométrica.

El refinamiento de nudos tiene como finalidad aumentar el control local donde se añade el nudo, obteniendo un mayor número de funciones base donde se inserta y decreciendo el tamaño de los elementos sobre los que se trabaja.

**Definición.** Definiremos los elementos que necesitamos para aplicar el refinamiento:

- **Puntos de control ponderados,  $\mathbf{P}_i^w$ :** Sea  $\mathbf{P}_i$  un punto de control y  $w_i$  su peso asociado, definimos  $\mathbf{P}_i^w = (w_i \mathbf{P}_i, w_i)^T$ .
- **Nuevo nudo,  $\xi$ :** aunque ya tenemos el concepto de nudo definido es necesario declarar en que intervalo se encuentra, tomaremos que  $\xi \in [u_k, u_{k+1})$ .
- **Nuevo vector de nodos,  $U^*$ :**  $U^* = \{u_0, u_1, \dots, u_k, \xi, u_{k+1}, \dots, u_m\}$ .
- **Nuevos puntos de control ponderados,  $\mathbf{Q}_i^w$ :** estarán definidos mediante  $\mathbf{Q}_i^w = \alpha_i \mathbf{P}_i^w + (1 - \alpha_i) \mathbf{P}_{i-1}^w$ , donde  $\alpha_i$  se define mediante la fórmula:

$$\alpha_i = \begin{cases} 1 & \text{si } i \leq k - p, \\ \frac{\xi - u_i}{u_{i+p} - u_i} & \text{si } k - p + 1 \leq i \leq k, \\ 0 & \text{si } i > k. \end{cases}$$

- **Nuevas funciones base,  $N_{i,p}^*(u)$ ,** asociadas al nuevo vector de nodos  $U^*$ , que cumplen la igualdad:  $N_{i,p}(u) = \alpha_i N_{i,p}^*(u) + (1 - \alpha_{i+1}) N_{i+1,p}^*(u)$ <sup>5</sup>.

**Proposición 2.9.** La curva original ponderada es equivalente a la curva ponderada refinada,

$$\mathbf{C}^w(u) = \sum_{i=0}^m N_{i,p}(u) \mathbf{P}_i^w = \sum_{j=0}^{m+1} N_{j,p}^*(u) \mathbf{Q}_j^w$$

*Demostración.* La demostración puede encontrarse en el anexo 6.2.9. □

La inserción de nudos en superficies se realiza de manera direccional, introduciendo unicamente elementos a uno de los dos vectores de nudos, modificando así una de las dos curvas que generan la superficie, pero al ser invariante bajo la inserción de nudos esta, la superficie generada final es idéntica a la original, los pesos y puntos de control en forma matricial son tratados como en la demostración de la proposición 2.8, trabajando con ellos de forma vectorial, relacionándolos con el vector de nudos que recibe la inserción.

<sup>5</sup>Demostración en el anexo 6.2.8



### 2.5.1. Implementación de la Inserción de Nudos

Durante el trabajo se utilizarán las siguientes funciones para la inserción de nudos en las estructuras correspondientes:

**Caso curvas (1D):** La función `CurveKnotsIns` calcula los nuevos puntos de control tras insertar un nudo  $u$  con multiplicidad  $r$ .

---

```
iMatrix<double> CurveKnotsIns(
    int np,                // n (num. ptos control)
    int p,                // p (grado)
    std::vector<double> &KnotVector, // Vector de Nudos (U)
    const iMatrix<double> &CtrlPtsW, // Ptos Control Ponderados (Pw)
    double u,             // Valor del nudo a insertar
    int index,            // Span index (k)
    int s,                // Multiplicidad inicial
    int r                 // Veces a insertar
);
```

---

Listing 2.1: Parámetros para la función de inserción de nudos en curvas.

**Caso superficies (2D):** Para superficies NURBS, la función se generaliza para operar sobre las direcciones  $u$  (índice 1) o  $v$  (índice 2) según el parámetro booleano `dir`.

---

```
std::vector<iMatrix<double>> SurfaceKnotIns(
    int np1, int p1, std::vector<double> &KnotVector1, // Dir U
    int np2, int p2, std::vector<double> &KnotVector2, // Dir V
    const std::vector<iMatrix<double>> &CtrlPtsW,      // Red de control
    bool dir,                // true = Dir U, false = Dir V
    double uv,              // Valor del nudo a insertar
    int index,              // Span index
    int r,                  // Veces a insertar
    int s                    // Multiplicidad inicial
);
```

---

Listing 2.2: Parámetros para la función de inserción de nudos en superficies.

La correspondencia matemática de los parámetros es:

- `np`, `np1`, `np2`: Corresponden al número de puntos de control ( $n$ ).
- `CtrlPtsW`: Representa los puntos de control en coordenadas homogéneas  $\mathbf{P}_{i,j}^w = (w_i x_i, w_i y_i, w_i z_i, w_i)$ .
- `index`: Corresponde al índice del intervalo nodal o *span* donde se encuentra el nudo a insertar.

El código fuente de estas implementaciones puede encontrarse en el anexo 6.3.3.



## Capítulo 3

# Análisis isogeométrico.

No estoy seguro del orden que debería seguir este capítulo, si debería poner antes las velocidades de convergencia o de tiempo de computación o mejor al final.

Resumen de los casos que se van a tratar en este capítulo (1D y 2D), y formulaciones débiles de estos para trabajar posteriormente sobre ellas.

### 3.1. Ensamblaje de matrices y vector de carga.

#### 3.1.1. Caso 1D.

Aplicar aquí los apuntes que hice para aclararme que os mande sobre el ensamblado en 1D.

#### 3.1.2. Caso 2D.

Aplicar aquí los apuntes que hice para aclararme que os mande sobre el ensamblado en 2D.

### 3.2. Imposición de condiciones de contorno.

En este capítulo no sé si es necesario que les de una explicación a por qué son necesarias o puedo asumir que partiendo de elementos finitos puedo asumir que el lector está familiarizado.

#### 3.2.1. Tipo Dirichlet.

#### 3.2.2. Tipo Neumann.

#### 3.2.3. Tipo Robin o mixtas.

### 3.3. Convergencia en $L^2$ y $H^1$ y tiempo de computación teórico.



## Capítulo 4

# Implementación computacional.

Resumen breve de la intención del capítulo en ilustrar ejemplos con código funcional que se ha utilizado para obtener resultados numéricos.

Así como el uso de diferentes estructuras como matrices sparse o tensores y su necesidad para el correcto funcionamiento del código.

### 4.1. Lenguaje y librerías utilizadas.

Se mencionará C++ como lenguaje principal usando la librería Eigen para el uso de matrices sparse, así como clases y funciones que se añadirán en el anexo para tema NURBS. también habrá mención a Octave para visualizar los resultados.

### 4.2. Metodos numéricos empleados.

Explicación de integración numérica para las integrales que aparecen en las matrices de ensamblado y vector de carga mediante la cuadratura de Gauss-Legendre, el metodo LU para resolver el sistema 1D y el método del gradiente conjugada con preconditionador de Choleski incompleto para resolver el sistema de 2D.

### 4.3. Estructura del programa.

Se explicaran con detalle y en orden como es la ejecución del programa y que funciones/operaciones se ejecutan en cada paso.

- Creación de la estructura mediante NURBS.

- Refinamiento del mallado.

- Ensamblado elemento a elemento (dependiendo de cuanto tome este apartado se fragmentará).

- Imposición de las condiciones de frontera.

- Resolver el sistema final.

- Construcción y evaluación de la solución a partir de las soluciones del sistema.

- Cálculo de la norma L2 y H1.

- Representación Gráfica en Octave.

(Con el fin de no superar el máximo de hojas permitidas, el código se mostrará en el anexo y no en este apartado)



## **Capítulo 5**

# **Resultados.**

En este capítulo se mostrarán tanto los resultados de convergencia como las gráficas de las soluciones, la frecuencia de estas respecto al refinamiento dependerá de cuantas páginas hayan sido escritas en total. En ambas secciones se verificará que el código se ha ejecutado correctamente y su validación.

### **5.1. Resultados en 1D**

### **5.2. Resultados en 2D**





## **Capítulo 6**

### **Conclusiones.**



# Bibliografía

- [1] M.G. Larson and F. Bengzon. *The Finite Element Method: Theory, Implementation, and Applications*. Texts in Computational Science and Engineering. Springer Berlin Heidelberg, 2013. ISBN 978-3-642-33287-6. URL [https://books.google.es/books?id=Vek\\_AAAAQBAJ](https://books.google.es/books?id=Vek_AAAAQBAJ).
- [2] H. Brezis. *Functional Analysis, Sobolev Spaces and Partial Differential Equations*. Universitext. Springer New York, 2010. ISBN 978-0-387-70913-0. URL <https://books.google.es/books?id=GAA2Xq0IIGoC>.
- [3] L.C. Evans. *Partial Differential Equations*. Graduate studies in mathematics. American Mathematical Society, 1991. URL <https://books.google.es/books?id=TF8EzQEACAAJ>.
- [4] L. Piegl and W. Tiller. *The NURBS Book*. Monographs in Visual Communication. Springer Berlin Heidelberg, 2012. ISBN 978-3-642-59223-2. URL <https://books.google.es/books?id=58KqCAAAQBAJ>.
- [5] J.A. Cottrell, T.J.R. Hughes, and Y. Bazilevs. *Isogeometric Analysis: Toward Integration of CAD and FEA*. Wiley, 2009. ISBN 978-0-470-74873-2. URL <https://books.google.es/books?id=SfOUdAEACAAJ>.
- [6] G.E. Farin. *Curves and Surfaces for CAGD: A Practical Guide*. Computer graphics and geometric modeling. Elsevier Science, 2002. ISBN 978-1-55860-737-8. URL <https://books.google.es/books?id=5HYTP1dIAp4C>.
- [7] J.H. Ahlberg, E.N. Nilson, J.L. Walsh, and R. Bellman. *The Theory of Splines and Their Applications: Mathematics in Science and Engineering: A Series of Monographs and Textbooks, Vol. 38*. Number v. 38. Academic Press, 2016. ISBN 978-1-4832-2295-0. URL <https://books.google.es/books?id=3bZ1DAAAQBAJ>.
- [8] Ulrich Langer, Stephen E Moore, and Martin Neumüller. Space-time isogeometric analysis of parabolic evolution equations. *Computer Methods in Applied Mechanics and Engineering*, 306: 342–363, 2016. doi: 10.1016/j.cma.2016.03.042. Publisher: Elsevier.
- [9] Y. Bazilevs, V.M. Calo, J.A. Cottrell, J.A. Evans, T.J.R. Hughes, S. Lipton, M.A. Scott, and T.W. Sederberg. Isogeometric analysis using T-splines. *Computer Methods in Applied Mechanics and Engineering*, 199(5-8):229–263, 2010. doi: 10.1016/j.cma.2009.02.036. Publisher: Elsevier.
- [10] David J. Benson, Yuri Bazilevs, Ming-Chen Hsu, and Thomas J.R. Hughes. A large deformation, rotation-free, isogeometric shell. *Computer Methods in Applied Mechanics and Engineering*, 200(13-16):1367–1378, 2011. doi: 10.1016/j.cma.2010.12.003. Publisher: Elsevier.
- [11] Yuri Bazilevs, Jeffrey Gohean, Thomas Hughes, R. Moser, and Yongjie Zhang. Patient-specific isogeometric fluid-structure interaction analysis of thoracic aortic blood flow due to implantation of the Jarvik 2000 left ventricular assist device. *Computer Methods in Applied Mechanics and Engineering*, 198:3534–3550, September 2009. doi: 10.1016/j.cma.2009.04.015.

- [12] Josef Kiendl, Kai-Uwe Bletzinger, J. Linhard, and Roland Wüchner. Isogeometric shell analysis with Kirchhoff-Love elements. *Computer Methods in Applied Mechanics and Engineering - COMPUT METHOD APPL MECH ENG*, 198:3902–3914, November 2009. doi: 10.1016/j.cma.2009.08.013.
- [13] Eric Brechner. Developing and Using CAD/CAM/CAE Systems in Boeing. *IEEE Annals of the History of Computing*, 46(1):38–49, January 2024. ISSN 1058-6180. doi: 10.1109/MAHC.2024.3468340.
- [14] R.A. Liming. *Practical Analytic Geometry with Applications to Aircraft*. Macmillan, 1944. ISBN 978-0-598-45557-4. URL <https://books.google.es/books?id=x99EAAAAAAAJ>.
- [15] K.J. Versprille. *Computer Aided Design Applications of the Rational B-spline Approximation Form*. University Microfilms, 1975. URL <https://books.google.es/books?id=D0wvjwEACAAJ>.
- [16] Dalton Fazanaro, Paulo Amorim, Thiago Franco de Moraes, Jorge Silva, and Helio Pedrini. NURBS Parameterization for Medical Surface Reconstruction. *Applied Mathematics*, 07:137–144, January 2016. doi: 10.4236/am.2016.72013.
- [17] Gerald Farin. A History of Curves and Surfaces in CAGD. *Computer Aided Geometric Design - CAGD*, December 2002. doi: 10.1016/B978-044451104-1/50002-2.

# Anexos

## 6.1. Método de los elementos finitos.

Partiendo de la formulación variacional, con la garantía de la existencia y unicidad de la solución  $u \in V(\Omega) \subset H^1(\Omega)$ , en el FEM buscamos una solución aproximada  $u^h$  de esta en un espacio de dimensión finita.

Para ello se realizan dos pasos previos:

- **Discretización del dominio ( $\Omega$ ):** este se divide en una colección de subdominios principales denominados **elementos**, estos elementos son politopos cerrados y simples (en 2D normalmente son triángulos, pero pueden ser también otro tipo de polígonos). Los denotaremos como  $K_j$ , de manera que se cumple que  $\Omega \cup \Gamma = \overline{\Omega} = \bigcup_{j=1}^M K_j$ . Esta subdivisión se conoce como malla ( $T_h = \{K_1, K_2, \dots, K_M\}$ ).
- **Discretización del espacio ( $V(\Omega)$ ):** el subespacio de dimensión finita construido a base de funciones polinómicas  $V^h = \{v \in H^1(\Omega) \mid v = g \text{ en } \Gamma_D \text{ y } v|_{K_j} \in P_p(K_j), \forall K_j \in T_h\}$ , donde  $p \in \mathbb{N}$  es el grado polinomial elegido y  $h = \max_{K_j \in \mathcal{T}_h} h_j$ ,  $h_j = \text{diam}(K_j) = \sup_{x,y \in K_j} \|x - y\|$ , de manera que podemos definir una base de este de la forma  $\{\phi_1, \phi_2, \dots, \phi_N\}$ .

Definimos adicionalmente los nodos  $x_i \in \overline{\Omega}$  y el conjunto nodal  $\mathcal{N} = \{x_1, x_2, \dots, x_N\} \subset \overline{\Omega}$ , donde  $N$  es la dimensión de  $V^h$ . El lugar que ocupa cada  $x_i$  depende tanto del tipo de elementos utilizados para crear la malla como del grado polinomial  $p$  elegido, además, cada uno de estos está asociado a una única función base, de manera que se cumplen las siguientes propiedades:

- $\phi_i(x_j) = \delta_{ij} = \begin{cases} 1 & \text{si } i = j \\ 0 & \text{si } i \neq j \end{cases}$
- $\text{supp}(\phi_i) = \overline{\bigcup \{K_j \in T_h \mid x_i \in K_j\}}$

Tras esto definimos la solución aproximada y su gradiente como:

$$u^h(x) = \sum_{i=1}^N U_i \phi_i(x), \quad U_i \in \mathbb{R}, \quad \forall i = 1, 2, \dots, N.$$

$$\nabla \mathbf{u}^h(\mathbf{x}) = \nabla \left( \sum_{i=1}^N U_i \phi_i(\mathbf{x}) \right) = \sum_{i=1}^N U_i \nabla \phi_i(\mathbf{x}).$$

Luego el problema se resume en encontrar  $u \in v^h$  tal que  $a(u^h, v^h) - L(v^h) = 0, \forall v^h \in V^h$ . sustituyendo y aplicando la **linealidad de la forma bilineal<sup>1</sup> y de la forma lineal<sup>2</sup>** nos queda:

$$\sum_{i=1}^N V_i \left[ \sum_{j=1}^N a(\phi_j, \phi_i) U_j - L(\phi_i) \right] = 0, \quad \forall V_i \in \mathbb{R}, \quad \forall i = 1, 2, \dots, N.$$

Finalizando por tanto con el sistema dado por  $\sum_{j=1}^N a(\phi_j, \phi_i) U_j = L(\phi_i), \quad \forall i = 1, 2, \dots, N$ .

<sup>1</sup>  $a(\sum_{j=1}^N U_j \phi_j, \sum_{i=1}^N U_i \phi_i) = \sum_{i=1}^N V_i \sum_{j=1}^N U_j a(\phi_j, \phi_i)$ .

<sup>2</sup>  $L(\sum_{i=1}^N V_i \phi_i) = \sum_{i=1}^N V_i L(\phi_i)$ .

**Definición.** Los principales elementos del sistema que se resolverá para obtener la solución son:

- **La matriz de rigidez  $\mathbf{A}$ :**  $A_{ij} = a(\phi_j, \phi_i) = \int_{\Omega} \nabla \phi_j \cdot \nabla \phi_i d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} \phi_j \phi_i d\Gamma$ .
- **El vector de carga  $\mathbf{F}$ :**  $F_i = L(\phi_i) = \int_{\Omega} f \phi_i d\Omega + \int_{\Gamma_N} h \phi_i d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} \phi_i d\Gamma$ .

<sup>3</sup> Finalizando con el sistema  $AU = F$ , donde  $U = (U_1, U_2, \dots, U_N)^T$ , adicionalmente  $u^h(x_i) = U_i$ .

**Definición.** Definimos ahora la norma inducida por la forma bilineal  $a(\cdot, \cdot)$ , que denotaremos por  $\|\cdot\|_V$ :

$$\|v\|_V^2 = a(v, v) = \int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma.$$

**Lema 6.1.1.** Las normas  $\|\cdot\|_V$  y  $\|\cdot\|_{H^1(\Omega)}$  son equivalentes, es decir, existen  $C_1, C_2 \in \mathbb{R}$  tal que:

$$C_1 \|v\|_{H^1(\Omega)}^2 \leq \|v\|_V^2 \leq C_2 \|v\|_{H^1(\Omega)}^2.$$

*Demostración omitida.* La demostración puede consultarse en el Anexo 6.2.4.

**Teorema 6.1.** El error  $e = u - u^h$  del FEM converge en las normas  $\|\cdot\|_{L^2(\Omega)}$  y  $\|\cdot\|_{H^1(\Omega)}$ , tal que:

- $\|e\|_{H^1(\Omega)}$ , se reduce con orden  $\mathcal{O}(h^p)$ .
- $\|e\|_{L^2(\Omega)}$ , se reduce con orden  $\mathcal{O}(h^{p+1})$ .

Siempre que  $u \in H^{p+1}(\Omega)$ .

*Demostración.* Separaremos la demostración en cada una de las normas:

- **Convergencia en  $\|\cdot\|_{H^1(\Omega)}$ :**

Asumiendo la coercividad y continuidad de la forma bilineal en  $V(\Omega)^4$ , nos encontramos en las condiciones del **Lema de Céa** por tanto se cumple:

$$\|u - u_h\|_V \leq \frac{C_{cont}}{C_{coer}} \inf_{v_h \in V_h} \|u - v_h\|_V,$$

donde  $C_{cont}$  es la constante de continuidad y  $C_{coer}$  es la constante de coercividad.

Aplicando ahora la equivalencia entre normas  $\|\cdot\|_V$  y  $\|\cdot\|_{H^1(\Omega)}$  obtenemos que  $\exists C_{Céa} \in \mathbb{R}^+$  tal que:

$$\|u - u_h\|_{H^1(\Omega)} \leq C_{Céa} \inf_{v_h \in V_h} \|u - v_h\|_{H^1(\Omega)}. \quad (6.1)$$

Por último usaremos la **propiedad de aproximación del espacio de Sobolev**, la cual nos garantiza que  $\exists C_{Sob} \in \mathbb{R}^+$  tal que:

$$\inf_{v_h \in V_h} \|u - v_h\|_{H^1(\Omega)} \leq C_{Sob} h^p \|u\|_{H^{p+1}(\Omega)}. \quad (6.2)$$

Finalmente uniendo todo y definiendo  $C_{Cota} = C_{Céa} C_{Sob}$  tenemos:

$$\|u - u_h\|_{H^1(\Omega)} \leq C_{Cota} h^p,$$

obteniendo así un error en  $H^1(\Omega)$  que converge con orden  $\mathcal{O}(h^p)$ , siempre que  $u \in H^{p+1}(\Omega)$ .

<sup>3</sup>Si se desea conocer más sobre algoritmos de ensamblado de  $\mathbf{A}$  y  $\mathbf{F}$ , consúltese [1], especialmente los capítulos 4 y 5.

<sup>4</sup>La demostración de estas puede ser encontrada en el anexo 6.2.5

■ **Convergencia en  $\|\cdot\|_{L^2(\Omega)}$ :**

Para la cota del error  $e = u - u_h$  en  $L^2(\Omega)$  consideramos el problema dual asociado:

Encontrar  $w \in V(\Omega)$  tal que:

$$a(v, w) = (e, v)_{L^2(\Omega)} = \int_{\Omega} e v d\Omega, \quad \forall v \in V(\Omega).$$

Debido a la coercividad y continuidad de la forma bilineal y la continuidad de la forma lineal<sup>5</sup>, debido al **Lema de Lax-Millgram**, la solución  $w$  existe y es única, su regularidad viene dada por las condiciones de regularidad elípticas asumidas en el anexo 6.2.3, luego  $\exists C_{\text{Reg}} \in \mathbb{R}^+$  tal que:

$$\|w\|_{H^2(\Omega)} \leq C_{\text{reg}} \|e\|_{L^2(\Omega)}$$

Tomamos ahora  $v = e$ , obteniendo así la identidad

$$a(e, w) = (e, e)_{L^2(\Omega)} = \|e\|_{L^2(\Omega)}^2.$$

Como la solución discreta  $u_h$  satisface la **propiedad de ortogonalidad de Galerkin**:

$$a(u - u_h, v_h) = a(e, v_h) = 0, \quad \forall v_h \in V_h.$$

Nos permite introducir cualquier función interpolante  $w_h \in V_h$  tal que se cumple la siguiente igualdad:

$$\|e\|_{L^2(\Omega)}^2 = a(e, w) - a(e, w_h) = a(e, w - w_h).$$

Aplicando ahora la **continuidad de la forma bilineal**,

$$\|e\|_{L^2(\Omega)}^2 \leq C_{\text{cont}} \|e\|_{H^1(\Omega)} \|w - w_h\|_{H^1(\Omega)}.$$

Usamos ahora el resultado de convergencia para  $\|e\|_{H^1(\Omega)}$  dadas por 6.1 y 6.2

$$\|e\|_{L^2(\Omega)}^2 \leq C_{\text{cont}} C_{\text{Céa}} C_{\text{Sob}} h^p \|u\|_{H^{p+1}(\Omega)} \|w - w_h\|_{H^1(\Omega)}.$$

Terminamos utilizando la **propiedad de aproximación de los espacios de Sobolev**, y tomando que  $w \in H^2(\Omega)$  debido a la regularidad elíptica tenemos:

$$\|w - w_h\|_{H^1(\Omega)} \leq C_2 h^{2-1} \|w\|_{H^2(\Omega)} = C_2 h^1 \|w\|_{H^2(\Omega)}.$$

Culminando en la desigualdad:

$$\|e\|_{L^2(\Omega)}^2 \leq \left( C_{\text{cont}} C_{\text{Céa}} C_{\text{Sob}} C_2 \|u\|_{H^{p+1}(\Omega)} \|w\|_{H^2(\Omega)} \right) h^{p+1}.$$

obteniendo así un error en  $L^2(\Omega)$  que converge con orden  $\mathcal{O}(h^{p+1})$ , siempre que  $u \in H^{p+1}(\Omega)$ .

□

<sup>5</sup>La demostración de la continuidad de la forma lineal se encuentra en el anexo 6.2.6

## 6.2. Demmostraciones omitidas

### 6.2.1. Paso de solución fuerte a solución variacional

*Demostración.* Recurriremos al **Principio de Galerkin**, el cual nos garantiza que:

$$\int_{\Omega} (-\Delta u - f)v d\Omega = 0 \quad \text{para todo } v \in V_0(\Omega).$$

Reordenando y utilizando la identidad  $\Delta u \cdot v = \nabla \cdot (\nabla u \cdot v) - \nabla u \cdot \nabla v$ :

$$\int_{\Omega} (-\Delta u)v d\Omega = \int_{\Omega} -[\nabla \cdot (\nabla u \cdot v) - \nabla u \cdot \nabla v] d\Omega = \int_{\Omega} \nabla u \cdot \nabla v d\Omega - \int_{\Omega} \nabla \cdot (\nabla u \cdot v) d\Omega. \quad (6.3)$$

Al aplicar el **Teorema de Gauss-Ostrogradsky** al último término:

$$\int_{\Omega} \nabla \cdot (\nabla u \cdot v) d\Omega = \int_{\Gamma} (\nabla u \cdot \mathbf{n})v d\Gamma.$$

Sustituyendo esta conversión en la ecuación (6.4) obtenemos la **Formulación variacional preliminar**:

$$\int_{\Omega} \nabla u \cdot \nabla v d\Omega - \int_{\Gamma} (\nabla u \cdot \mathbf{n})v d\Gamma = \int_{\Omega} f v d\Omega.$$

Separando ahora  $\Gamma$  en sus componentes disjuntas:

$$\int_{\Gamma} (\nabla u \cdot \mathbf{n})v d\Gamma = \int_{\Gamma_D} (\nabla u \cdot \mathbf{n})v d\Gamma + \int_{\Gamma_N} (\nabla u \cdot \mathbf{n})v d\Gamma + \int_{\Gamma_R} (\nabla u \cdot \mathbf{n})v d\Gamma.$$

Utilizamos ahora  $v \in V_0(\Omega)$  y las condiciones de contorno:

$$\int_{\Gamma} (\nabla u \cdot \mathbf{n})v d\Gamma = 0 + \int_{\Gamma_N} h v d\Gamma + \int_{\Gamma_R} \left( \frac{r - \alpha u}{\beta} \right) v d\Gamma.$$

Por último sustituimos y reordenamos en forma bilineal  $a(u, v)$  igualada a la forma lineal  $L(v)$ , obteniendo así:

$$\begin{aligned} a(u, v) &= \int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} u v d\Gamma = \\ &= \int_{\Omega} f v d\Omega + \int_{\Gamma_N} h v d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma = L(v). \end{aligned}$$

□

### 6.2.2. Existencia y unicidad de la solución variacional (Lema de Lax-Millgram)

**Teorema 6.2.** *La existencia y unicidad de la solución al problema variacional  $u \in V(\Omega)$  está garantizada gracias al **Lema de Lax-Milgram**, pues se cumplen las tres siguientes condiciones de este:*

- **Continuidad de la Forma Bilineal:**  $|a(u, v)| \leq (1 + C_{Tr}^2 \left| \frac{\alpha}{\beta} \right|) \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$
- **Coercividad de la Forma Bilineal:**  $a(v, v) \geq \|\nabla v\|_{L^2(\Omega)}^2 \geq \frac{1}{C_P + 1} \|v\|_{H^1(\Omega)}^2.$
- **Continuidad de la Forma Lineal:**  $|L(v)| \leq C \|v\|_{H^1(\Omega)}, \quad C \in \mathbb{R}^+.$
- **Continuidad de la Forma Bilineal:**



*Demostración.* aplicaremos la **desigualdad triangular**,

$$|a(u, v)| = \left| \int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma \right| \leq \left| \int_{\Omega} \nabla u \cdot \nabla v d\Omega \right| + \left| \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma \right|.$$

Continuaremos con la **desigualdad de Cauchy-Schwartz** para el primer término,

$$\left| \int_{\Omega} \nabla u \cdot \nabla v d\Omega \right| \leq \int_{\Omega} |\nabla u| |\nabla v| d\Omega \leq \|\nabla u\|_{L^2(\Omega)} \|\nabla v\|_{L^2(\Omega)} \leq \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$$

Buscamos ahora acotar el segundo término utilizando primero la **desigualdad de Cauchy-Schwartz**:

$$\left| \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma \right| \leq \left| \frac{\alpha}{\beta} \right| \int_{\Gamma_R} |u| |v| d\Gamma \leq \left| \frac{\alpha}{\beta} \right| \|u\|_{L^2(\Gamma_R)} \|v\|_{L^2(\Gamma_R)}.$$

Usamos ahora el **teorema de la traza** que nos garantiza la existencia de una constante  $C_{Tr}$  tal que,

$$\left| \frac{\alpha}{\beta} \right| \|u\|_{L^2(\Gamma_R)} \|v\|_{L^2(\Gamma_R)} \leq C_{Tr}^2 \left| \frac{\alpha}{\beta} \right| \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$$

Por lo que obtenemos:

$$|a(u, v)| \leq (1 + C_{Tr}^2 \left| \frac{\alpha}{\beta} \right|) \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$$

□

#### ■ Coercividad de la Forma Bilineal:

*Demostración.* partimos de la condición impuesta  $\frac{\alpha}{\beta} \geq 0$ :

$$a(v, v) = \int_{\Omega} \nabla v \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} v^2 d\Gamma = \int_{\Omega} |\nabla v|^2 d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} v^2 d\Gamma \geq \int_{\Omega} |\nabla v|^2 d\Omega = \|\nabla v\|_{L^2(\Omega)}^2.$$

Partiendo de que  $\Omega$  es acotado y  $v \in V_0(\Omega)$  aplicamos la **desigualdad de Poincaré** la cual nos dice:

$$\text{Existe } C_P > 0 \text{ tal que: } \|v\|_{L^2(\Omega)}^2 \leq C_P \|\nabla v\|_{L^2(\Omega)}^2.$$

Recordamos ahora la definición de la norma  $H^1$  y aplicamos la **desigualdad de Poincaré**:

$$\|v\|_{H^1(\Omega)}^2 = \|v\|_{L^2(\Omega)}^2 + \|\nabla v\|_{L^2(\Omega)}^2 \leq (C_P + 1) \|\nabla v\|_{L^2(\Omega)}^2.$$

Dividiendo ambos lados por  $(C_P + 1)$  y sustituyendo en la desigualdad de la forma bilinear anterior obtenemos:

$$a(v, v) \geq \|\nabla v\|_{L^2(\Omega)}^2 \geq \frac{1}{C_P + 1} \|v\|_{H^1(\Omega)}^2.$$

□

#### ■ Continuidad de la Forma Lineal:

*Demostración.* partimos de definición de  $L(v)$  y aplicamos la **desigualdad triangular**,

$$|L(v)| \leq \left| \int_{\Omega} f v d\Omega \right| + \left| \int_{\Gamma_N} h v d\Gamma \right| + \left| \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma \right|.$$

Acotaremos ahora término a término,

$$\left| \int_{\Omega} f v d\Omega \right| \leq \|f\|_{L^2(\Omega)} \|v\|_{L^2(\Omega)} \leq \|f\|_{L^2(\Omega)} \|v\|_{H^1(\Omega)}.$$

$$\left| \int_{\Gamma_N} h v d\Gamma \right| \leq \|h\|_{L^2(\Gamma_N)} \|v\|_{L^2(\Gamma_N)} \leq \|h\|_{L^2(\Gamma_N)} C_{Tr} \|v\|_{H^1(\Omega)}, \text{ (Teorema de la traza).}$$

$$\left| \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma \right| \leq \left| \frac{1}{\beta} \right| \|r\|_{L^2(\Gamma_R)} \|v\|_{L^2(\Gamma_R)} \leq \left| \frac{1}{\beta} \right| \|r\|_{L^2(\Gamma_R)} C_{Tr} \|v\|_{H^1(\Omega)}, \text{ (Teorema de la traza).}$$

Finalmente, la continuidad se demuestra al acotar:

$$|L(v)| \leq \left( \|f\|_{L^2(\Omega)} + \|h\|_{L^2(\Gamma_N)} C_{Tr} + \left| \frac{1}{\beta} \right| \|r\|_{L^2(\Gamma_R)} C_{Tr} \right) \|v\|_{H^1(\Omega)}.$$

□

El cumplimiento de estas condiciones para el operador Laplaciano asegura que el problema variacional está **bien planteado**.

### 6.2.3. Equivalencia de la solución de la formulación variacional y fuerte

*Demostración.* Asumimos  $\Gamma \in C^2$ ,  $f \in L^2(\Omega)$ ,  $g \in H^{3/2}(\Gamma_D)$ ,  $h \in H^{1/2}(\Gamma_N)$ ,  $r \in H^{1/2}(\Gamma_R)$  y  $u \in H^2(\Omega)$ , recordamos ahora la formulación variacional:

$$\int_{\Omega} \nabla u \cdot \nabla v d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} u v d\Gamma = a(u, v) = L(v) = \int_{\Omega} f v d\Omega + \int_{\Gamma_N} h v d\Gamma + \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma. \quad (6.4)$$

Como  $u \in H^2(\Omega)$  podemos aplicar el teorema de divergencia inverso al primer término de la forma bilineal:

$$\int_{\Omega} \nabla u \cdot \nabla v d\Omega = - \int_{\Omega} (\Delta u) v d\Omega + \int_{\Gamma} (\nabla u \cdot \mathbf{n}) v d\Gamma.$$

Sustituyendo ahora en (6.4) y agrupando todo en un lado de la igualdad obtenemos:

$$- \int_{\Omega} (\Delta u) v d\Omega + \int_{\Gamma} (\nabla u \cdot \mathbf{n}) v d\Gamma + \int_{\Gamma_R} \frac{\alpha}{\beta} u v d\Gamma - \int_{\Omega} f v d\Omega - \int_{\Gamma_N} h v d\Gamma - \int_{\Gamma_R} \frac{r}{\beta} v d\Gamma = 0.$$

Separamos  $\Gamma$  en sus componentes disjuntas  $\Gamma_N$ ,  $\Gamma_D$ ,  $\Gamma_R$  y como  $v \in V_0(\Omega)$ ,  $v = 0$  en  $\Gamma_D$  luego:

$$\int_{\Gamma} (\nabla u \cdot \mathbf{n}) v d\Gamma = \int_{\Gamma_N} (\nabla u \cdot \mathbf{n}) v d\Gamma + \int_{\Gamma_R} (\nabla u \cdot \mathbf{n}) v d\Gamma.$$

Reagrupamos ahora los términos que se integran en las mismas regiones:

$$\int_{\Omega} (-\Delta u - f) v d\Omega + \int_{\Gamma_N} (\nabla u \cdot \mathbf{n} - h) v d\Gamma + \int_{\Gamma_R} (\nabla u \cdot \mathbf{n} + \frac{\alpha}{\beta} u - \frac{r}{\beta}) v d\Gamma = 0.$$

Aplicando ahora el **teorema fundamental del cálculo de variaciones** tenemos que todos los elementos entre paréntesis equivalen a 0 luego:

$$\begin{cases} -\Delta u = f & \text{en } \Omega, \\ u = g & \text{en } \Gamma_D \quad (u \in V(\Omega)), \\ \nabla u \cdot \mathbf{n} = h & \text{en } \Gamma_N, \\ \alpha u + \beta \nabla u \cdot \mathbf{n} = r & \text{en } \Gamma_R. \end{cases}$$

□

### 6.2.4. Equivalencia de normas

*Demostración.* Demostraremos ambas cotas por separado:

- **Acotación superior** ( $\|v\|_V^2 \leq C_2 \|v\|_{H^1(\Omega)}^2$ ), iniciamos recordando la definición de cada norma:

- $\|v\|_V^2 = \int_{\Omega} |\nabla v|^2 d\Omega + \int_{\Gamma_R} \frac{\alpha}{\beta} |v|^2 d\Gamma$
- $\|v\|_{H^1(\Omega)}^2 = \int_{\Omega} |\nabla v|^2 d\Omega + \int_{\Omega} |v|^2 d\Omega$

Aplicaremos ahora la **desigualdad de la traza** obteniendo:

$$\int_{\Gamma_R} |v|^2 d\Gamma \leq C_{Tr} \|v\|_{H^1(\Omega)}^2.$$

Usaremos también la desigualdad trivial:

$$\int_{\Omega} |\nabla v|^2 d\Omega \leq \|v\|_{H^1(\Omega)}^2.$$

Obteniendo finalmente:

$$\|v\|_V^2 \leq \|v\|_{H^1(\Omega)}^2 + C_{Tr} \|v\|_{H^1(\Omega)}^2 = (1 + C_{Tr}) \|v\|_{H^1(\Omega)}^2.$$

Luego tomaremos  $C_2 = 1 + C_{Tr}$ .

- **Acotación inferior** ( $C_1 \|v\|_{H^1(\Omega)}^2 \leq \|v\|_V^2$ ), partimos de la **desigualdad de Poincaré generalizado**, el cual nos dice que existe  $C_P \in \mathbb{R}^+$  tal que:

$$\int_{\Omega} |v|^2 d\Omega \leq C_P \left( \int_{\Omega} |\nabla v|^2 d\Omega + \int_{\Gamma_R} |v|^2 d\Gamma \right)$$

Sustituyendo ahora en  $\|v\|_{H^1(\Omega)}$ :

$$\|v\|_{H^1(\Omega)}^2 \leq \int_{\Omega} |\nabla v|^2 d\Omega + C_P \left( \int_{\Omega} |\nabla v|^2 d\Omega + \int_{\Gamma_R} |v|^2 d\Gamma \right) = (1 + C_P) \int_{\Omega} |\nabla v|^2 d\Omega + C_P \int_{\Gamma_R} |v|^2 d\Gamma.$$

Buscamos ahora una constante  $C_R$  tal que  $C_R \cdot \|v\|_V^2 \geq \|v\|_{H^1(\Omega)}^2$  y cumpla las siguientes propiedades:

- $C_R \geq (1 + C_P) \implies C_R \geq 1 + C_P.$
- $C_R \cdot \frac{\alpha}{\beta} \geq C_P \implies C_R \geq C_P \cdot \frac{\beta}{\alpha}$

Luego tomamos:

$$C_R = \max \left( 1 + C_P, \frac{\beta C_P}{\alpha} \right).$$

Obteniendo así la siguiente desigualdad:

$$C_R \|v\|_V^2 = C_R \int_{\Omega} |\nabla v|^2 d\Omega + C_R \frac{\alpha}{\beta} \int_{\Gamma_R} |v|^2 d\Gamma \geq (1 + C_P) \int_{\Omega} |\nabla v|^2 d\Omega + C_P \int_{\Gamma_R} |v|^2 d\Gamma.$$

Tomando finalmente  $C_1 = \frac{1}{C_R}$ :

$$\|v\|_V^2 \geq C_1 \|v\|_{H^1(\Omega)}^2.$$

□

### 6.2.5. Condiciones del Lema de Céa

*Demostración.* Probaremos la coercividad y la continuidad en  $\|\cdot\|_V$ :

- **Coercividad:**  $\exists C_1 \in \mathbb{R}^+$  tal que  $C_1 \|v\|_{H^1(\Omega)}^2 \leq \|v\|_V^2$ , Probado en la equivalencia de normas.
- **Continuidad:**  $\exists C_2 \in \mathbb{R}^+$  tal que  $|a(u, v)| \leq C_2 \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}$ : Iniciamos aplicando la **desigualdad triangular** a la definición de  $|a(u, v)|$ :

$$|a(u, v)| \leq \left| \int_{\Omega} \nabla u \cdot \nabla v d\Omega \right| + \left| \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma \right|.$$

Aplicamos ahora la **desigualdad de Cauchy-Schwartz** en  $L^2(\Omega)$  al primer término:

$$\left| \int_{\Omega} \nabla u \cdot \nabla v d\Omega \right| \leq \int_{\Omega} |\nabla u| |\nabla v| d\Omega \leq \|\nabla u\|_{L^2(\Omega)} \|\nabla v\|_{L^2(\Omega)}.$$

Usamos ahora la propiedad trivial  $\|v\|_{H^1(\Omega)}^2 \geq \|\nabla v\|_{L^2(\Omega)}^2, \forall v \in H^1(\Omega)$ ,

$$|a(u, v)| \leq \|\nabla u\|_{L^2(\Omega)} \|\nabla v\|_{L^2(\Omega)} \leq \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$$

Pasamos ahora al segundo término e empezamos aplicando la **desigualdad de Cauchy-Schwartz**:

$$\left| \int_{\Gamma_R} \frac{\alpha}{\beta} uv d\Gamma \right| \leq \frac{\alpha}{\beta} \int_{\Gamma_R} |u| |v| d\Gamma \leq \frac{\alpha}{\beta} \|u\|_{L^2(\Gamma_R)} \|v\|_{L^2(\Gamma_R)}.$$

Seguiremos aplicando ahora la **desigualdad de la traza**:

$$\|u\|_{L^2(\Gamma_R)} \|v\|_{L^2(\Gamma_R)} \leq C_{Tr}^2 \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$$

Luego uniendo ambas cotas de los términos individuales obtenemos:

$$|a(u, v)| \leq \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)} + \frac{\alpha}{\beta} C_{Tr}^2 \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)} = (1 + \frac{\alpha}{\beta} C_{Tr}^2) \|u\|_{H^1(\Omega)} \|v\|_{H^1(\Omega)}.$$

Por último tomamos  $C_2 = 1 + \frac{\alpha}{\beta} C_{Tr}^2$ .

□

### 6.2.6. Continuidad de la forma lineal dual

*Demostración.* Recordamos que la forma lineal del problema dual es  $L_{\text{dual}}(v) = (e, v)_{L^2(\Omega)}$ .

Iniciamos usando la **desigualdad de Cauchy-Schwartz** en  $L^2(\Omega)$ ,

$$|(e, v)_{L^2(\Omega)}| \leq \|e\|_{L^2(\Omega)} \|v\|_{L^2(\Omega)}.$$

Aplicamos ahora la **propiedad trivial** de la acotación superior  $\|\cdot\|_{L^2(\Omega)}$  por  $\|\cdot\|_{H^1(\Omega)}$ ,

$$|(e, v)_{L^2(\Omega)}| \leq \|e\|_{L^2(\Omega)} \|v\|_{H^1(\Omega)}.$$

Como  $e = u - u^h$  es constante en este contexto,  $e \in L^2(\Omega)$ , luego tomamos  $C_L = \|e\|_{L^2(\Omega)}$ .

□

### 6.2.7. Propiedades de los B-Splines

*Demostración.* Se demostrarán las propiedades en el orden en el que han sido establecidas:

1. **Soporte Local:** procederemos mediante inducción sobre  $p$ :

- **Caso Base** ( $p = 0$ ): tenemos,  $N_{i,0}(u) = 1$  si  $u \in [u_i, u_{i+1})$  por definición.
- **Paso de inducción sobre  $p$ :** partiendo de la definición mediante el algoritmo de Cox-de Boor:

$$N_{i,p}(u) = \frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u). \quad (6.5)$$

Asumimos que  $N_{i,p-1}(u)$  tiene soporte en  $[u_i, u_{i+p})$  y  $N_{i+1,p-1}(u)$  tiene soporte en  $[u_{i+1}, u_{i+p+1})$ , luego  $N_{i,p}(u)$  tiene soporte  $[u_i, u_{i+p+1})$ .

2. **No Negatividad:** procederemos mediante inducción sobre  $p$ :

- **Caso Base** ( $p = 0$ ): tenemos por definición que  $N_i = 0$  o  $1$ , luego  $N_i \geq 0$ .
- **Paso de inducción sobre  $p$ :** asumimos cierta la hipótesis para  $p - 1$ , tomando  $u \in [u_i, u_{i+p+1})$  y aplicando el algoritmo 6.5, la demostración se reduce a probar que:
  - $\frac{u - u_i}{u_{i+p} - u_i} \geq 0$ , trivial pues por como esta definido el vector de nodos,  $u_{i+p} \geq u_i$  y como  $u \in [u_i, u_{i+p+1})$ ,  $u \geq u_i$
  - $\frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} \geq 0$ , de manera análoga tenemos  $u_{i+p+1} \geq u_{i+1}$  y  $u_{i+p+1} \geq u$ .

3. **Número de Bases Activas:** procederemos mediante inducción sobre  $p$ :

- **Caso Base** ( $p = 0$ ): por definición del caso  $p = 0$  de la funciones base y por la propiedad de no decrecimiento del vector de nudos, si  $u \in [u_i, u_{i+1})$ , existe unicamente una función base  $N_{i,0}$  no nula en ese intervalo, luego el único índice  $j$  que satisface la condición de tener una función base no nula asociada es  $i = j$ .
- **Paso de inducción sobre  $p$ :** asumimos que para el intervalo  $[u_i, u_{i+1})$  existen a lo sumo  $p$  funciones base  $N_{j,p-1}$  no nulas con índices  $j \in \{i - (p - 1), \dots, i\}$ , debido al algoritmo de Cox-de Boor 6.5 y a la no negatividad de los coeficientes y de las funciones base provada en el apartado anterior, si  $u \in [u_i, u_{i+1})$ ,  $N_{j,p}(u)$  será no nula en el intervalo  $[u_i, u_{i+1})$  si alguna de las funciones base  $N_{j,p-1}(u)$  y  $N_{j+1,p-1}(u)$  no lo son, luego:
  - $N_{j,p-1}(u)$  es no nula a lo sumo cuando  $j \in \{i - p + 1, i - p + 2, \dots, i\}$
  - $N_{j+1,p-1}(u)$  es no nula a lo sumo cuando  $j \in \{i - p, i - p + 1, \dots, i - 1\}$

Luego  $N_{j,p}$  es no nula cuando  $j \in [i - p, i]$ , probando así las posibles  $p + 1$  funciones base no nulas con sus índices posibles correspondientes.

Para demostrar la reducción, consideramos un nudo de valor  $\mathbf{u}^*$  con multiplicidad  $k$ , tal que la secuencia de nudos coincidentes es  $\mathbf{u}_j = \mathbf{u}_{j+1} = \dots = \mathbf{u}_i$ , donde  $\mathbf{i} = \mathbf{j} + \mathbf{k} - \mathbf{1}$ .

Una función base  $\mathbf{N}_{m,p}(\mathbf{u})$  es nula en el intervalo  $[u_i, u_{i+1})$  si su soporte  $(\mathbf{u}_m, \mathbf{u}_{m+p+1})$  no interseca dicho intervalo, es decir, si  $\mathbf{u}_{m+p+1} \leq \mathbf{u}_i$ .

Las  $k$  funciones base con los índices más bajos en el rango máximo, dadas por el conjunto  $\mathbf{J}_{\text{nulo}} = \{\mathbf{i} - \mathbf{p}, \mathbf{i} - \mathbf{p} + \mathbf{1}, \dots, \mathbf{i} - \mathbf{p} + \mathbf{k} - \mathbf{1}\}$ , se anulan. Analizamos el índice final del soporte para el último elemento de este conjunto,  $\mathbf{m} = \mathbf{i} - \mathbf{p} + \mathbf{k} - \mathbf{1}$ :

$$\mathbf{u}_{m+p+1} = u_{(i-p+k-1)+p+1} = \mathbf{u}_{i+k}$$

. Puesto que  $\mathbf{u}_i, \mathbf{u}_{i+1}, \dots, \mathbf{u}_{i+k-1}$  son todos iguales a  $\mathbf{u}^*$ , la secuencia de  $k$  nudos coincidentes fuerza a estas  $k$  bases a ser nulas. Por lo tanto, el número de funciones base nulas es  $k$ . Consecuentemente, el número de funciones base activas o no nulas es:  $\mathbf{p} + \mathbf{1} - k$ .

#### 4. Partición de la Unidad:

- **Caso Base** ( $p = 0$ ): como hemos probado en el apartado anterior en  $p = 0$  por cada intervalo  $[u_i, u_{i+1})$  hay una sola función base no nula, luego  $\forall u \in [u_0, u_m), \exists j \in R^+$  tal que  $u \in [u_j, u_{j+1})$ , luego,

$$\sum_{i=0}^n N_{i,0}(u) = N_{j,0}(u) = 1.$$

- **Paso de inducción sobre  $p$** : asumimos ahora que se cumple  $\sum_{i=0}^n N_{i,p}(u) = 1$ , empezamos entonces con el algoritmo de Cox-de Boor 6.5 talque,

$$\begin{aligned} \sum_{i=0}^n N_{i,p}(u) &= \sum_{i=0}^n \left[ \frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u) \right] = \\ &= \sum_{i=0}^n \left[ \frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) \right] + \sum_{i=0}^n \left[ \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u) \right] = \\ &= \sum_{i=0}^n \left[ \frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) \right] + \sum_{i=1}^{n+1} \left[ \frac{u_{i+p} - u}{u_{i+p} - u_i} N_{i,p-1}(u) \right]. \end{aligned}$$

Tomando que el soporte de  $N_{i,p-1}(u)$  es  $[u_i, u_{i+p})$  y que  $U$  es abierto, luego  $u_0 = u_1 = \dots = u_p$ , por lo que  $N_{0,p-1}(u)$ , al tener soporte degenerado, es siempre nula, y  $N_{n+1,p-1}$  es nula  $\forall u \in [u_0, u_n]$  nos queda:

$$\begin{aligned} \sum_{i=0}^n N_{i,p}(u) &= \sum_{i=0}^n \left[ \frac{u - u_i}{u_{i+p} - u_i} N_{i,p-1}(u) \right] + \left[ \frac{u_{i+p} - u}{u_{i+p} - u_i} N_{i,p-1}(u) \right] = \\ &= \sum_{i=0}^n \left[ \frac{u - u_i}{u_{i+p} - u_i} + \frac{u_{i+p} - u}{u_{i+p} - u_i} \right] N_{i,p-1}(u) = \sum_{i=0}^n N_{i,p-1}(u) = 1. \end{aligned}$$

5. **Invarianza bajo Transformaciones Afines**: sea una transformación afín  $T(\mathbf{P}) = \mathbf{M}\mathbf{P} + \mathbf{t}$ , aplicamos esta a la curva obteniendo:

$$\begin{aligned} T(\mathbf{C}(u)) &= \mathbf{M}\mathbf{C}(u) + \mathbf{t} = \mathbf{M} \left( \sum_{i=0}^n N_{i,p}(u) \mathbf{P}_i \right) + \mathbf{t} = \\ &= \sum_{i=0}^n \mathbf{M}(N_{i,p}(u) \mathbf{P}_i) + 1 \cdot \mathbf{t} = \sum_{i=0}^n N_{i,p}(u) (\mathbf{M}\mathbf{P}_i) + \mathbf{t} \left( \sum_{i=0}^n N_{i,p}(u) \right) = \\ &= \sum_{i=0}^n N_{i,p}(u) (\mathbf{M}\mathbf{P}_i + \mathbf{t}). \end{aligned}$$

Siendo esta última serie la curva generada por los puntos de control transformados mediante la transformación afín.

6. **Continuidad Controlable**: Suponemos que  $u_j$  tiene multiplicidad  $k \leq p$ , aplicamos ahora la definición de la derivada de la curva:

$$\mathbf{C}'(u) = \frac{d}{du} \left( \sum_{i=0}^n N_{i,p}(u) \mathbf{P}_i \right) = \sum_{i=0}^n N'_{i,p}(u) \mathbf{P}_i = \sum_{i=0}^n \left[ \frac{p}{u_{i+p} - u_i} N_{i,p-1}(u) - \frac{p}{u_{i+p+1} - u_{i+1}} N_{i+1,p-1}(u) \right] \mathbf{P}_i.$$

Reordenamos ahora los índices del término de la derecha de la suma, aplicando las mismas propiedades que en el apartado anterior, obteniendo así:

$$\mathbf{C}'(u) = \sum_{i=0}^{n-1} N_{i,p-1}(u) \underbrace{\left[ p \frac{\mathbf{P}_{i+1} - \mathbf{P}_i}{u_{i+p+1} - u_{i+1}} \right]}_{\mathbf{P}_i^{(1)}} = \sum_{i=0}^{n-1} N_{i,p-1}(u) \mathbf{P}_i^{(1)}.$$

Así, de manera recursiva definimos la derivada  $r$ -ésima de la curva:

$$\mathbf{C}^{(r)}(u) = \sum_i N_{i,p-r}(u) \mathbf{P}_i^{(r)}, \quad \left( \mathbf{P}_i^{(r)} = p \frac{\mathbf{P}_{i+1}^{(r-1)} - \mathbf{P}_i^{(r-1)}}{u_{i+p+1} - u_{i+1}} \right)$$

Obteniendo así que para  $r = p - k$ , si  $u = u_j$ ,  $N_{j-1,k} = 1$ , luego:

$$\begin{aligned} \lim_{u \rightarrow u_j^-} \mathbf{C}^{(r)}(u) &= \sum_i \mathbf{P}_i^{(r)} \lim_{u \rightarrow u_j^-} N_{i,k}(u) = \mathbf{P}_{j-1}^{(r)} \cdot \lim_{u \rightarrow u_j^-} N_{j-1,k}(u) = \mathbf{P}_{j-1}^{(r)}. \\ \lim_{u \rightarrow u_j^+} \mathbf{C}^{(r)}(u) &= \sum_i \mathbf{P}_i^{(r)} \lim_{u \rightarrow u_j^+} N_{i,k}(u) = \mathbf{P}_{j-1}^{(r)} \cdot \lim_{u \rightarrow u_j^+} N_{j-1,k}(u) = \mathbf{P}_{j-1}^{(r)}. \end{aligned}$$

Para  $r = p - k + 1$ , tenemos que:

$$\begin{aligned} \lim_{u \rightarrow u_j^-} N_{j-1,k-1}(u) &= 1, \quad \lim_{u \rightarrow u_j^+} N_{j-1,k-1}(u) = 0. \\ \lim_{u \rightarrow u_j^-} N_{j,k-1}(u) &= 0, \quad \lim_{u \rightarrow u_j^+} N_{j,k-1}(u) = 1. \end{aligned}$$

Por lo que en este caso:

$$\begin{aligned} \lim_{u \rightarrow u_j^-} \mathbf{C}^{(r)}(u) &= \sum_i \mathbf{P}_i^{(r)} \lim_{u \rightarrow u_j^-} N_{i,k-1}(u) = \mathbf{P}_{j-1}^{(r)} \cdot \lim_{u \rightarrow u_j^-} N_{j-1,k-1}(u) = \mathbf{P}_{j-1}^{(r)}. \\ \lim_{u \rightarrow u_j^+} \mathbf{C}^{(r)}(u) &= \sum_i \mathbf{P}_i^{(r)} \lim_{u \rightarrow u_j^+} N_{i,k-1}(u) = \mathbf{P}_j^{(r)} \cdot \lim_{u \rightarrow u_j^+} N_{j,k-1}(u) = \mathbf{P}_j^{(r)}. \end{aligned}$$

Luego como la continuidad de  $\mathbf{C}^{(r)}(u)$  depende de los puntos de control,  $\mathbf{C}(u) \notin L^{p-k+1}(\Omega)$ .

□

### 6.2.8. Igualdad de las nuevas funciones base bajo refinamiento

*Demostración.* Procederemos mediante inducción sobre  $p$ :

- **Caso base** ( $p = 0$ ): Sea  $\xi$  con valor dentro del intervalo de nudos  $[u_k, u_{k+1})$ , separamos los casos:
  - $i < k$ : En este caso  $N_{i,0}(u)$  comparte soporte con  $N_{i,0}^*(u)$ .
  - $i > k$ : En este caso  $N_{i,0}(u)$  comparte soporte con  $N_{i+1,0}^*(u)$ .
  - $i = k$ : En este caso  $N_{k,0}(u)$  tiene como soporte a la unión de ambos soportes de  $N_{k,0}^*(u)$  y  $N_{k+1,0}^*(u)$ , obteniendo la igualdad:

$$\begin{aligned} N_{k,0}(u) &= 1_{[u_k, u_{k+1})}(u) = 1_{[u_k, \xi)}(u) + 1_{[\xi, u_{k+1})}(u) = \\ &= N_{k,0}^*(u) + N_{k+1,0}^*(u) = \alpha_k N_{k,0}^*(u) + (1 - \alpha_{k+1}) N_{k+1,0}^*(u). \end{aligned}$$

- **Paso de inducción sobre  $p$** : iniciamos asumiendo cierta la hipótesis para  $p - 1$  y sustituimos en el algoritmo de Cox-de Boor,

$$\begin{aligned} N_{i,p}(u) &= \frac{u - u_i}{u_{i+p} - u_i} [\alpha_{i,p-1} N_{i,p-1}^*(u) + (1 - \alpha_{i+1,p-1}) N_{i+1,p-1}^*(u)] + \\ &+ \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} [\alpha_{i+1,p-1} N_{i+1,p-1}^*(u) + (1 - \alpha_{i+2,p-1}) N_{i+2,p-1}^*(u)]. \end{aligned}$$

Donde

$$\alpha_{i,p} = \begin{cases} 1 & \text{si } i \leq k - p, \\ \frac{\xi - u_i}{u_{i+p} - u_i} & \text{si } k - p < i \leq k, \\ 0 & \text{si } i > k. \end{cases}$$

Reorganizamos ahora la igualdad agrupando los términos asociados a cada función base distinta:

- Término asociado a  $N_{i,p-1}^*$ :

$$\frac{u - u_i}{u_{i+p} - u_i} \alpha_{i,p-1} = \alpha_{i,p} \frac{u - u_i^*}{u_{i+p}^* - u_i^*}.$$

- Término asociado a  $N_{i+1,p-1}^*$ :

$$\frac{u - u_i}{u_{i+p} - u_i} (1 - \alpha_{i+1,p-1}) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} \alpha_{i+1,p-1} = \alpha_{i,p} \frac{u_{i+p+1}^* - u}{u_{i+p+1}^* - u_{i+1}^*} + (1 - \alpha_{i+1,p}) \frac{u - u_{i+1}^*}{u_{i+1+p}^* - u_{i+1}^*}.$$

- Término asociado a  $N_{i+2,p-1}^*$ :

$$\frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} (1 - \alpha_{i+2,p-1}) = (1 - \alpha_{i+1,p}) \frac{u_{i+p+2}^* - u}{u_{i+p+2}^* - u_{i+2}^*}.$$

Sustituyendo ahora en la igualdad original obtenemos:

$$\begin{aligned} N_{i,p}(u) &= \alpha_{i,p} \underbrace{\left[ \frac{u - u_i^*}{u_{i+p}^* - u_i^*} N_{i,p-1}^* + \frac{u_{i+p+1}^* - u}{u_{i+p+1}^* - u_{i+1}^*} N_{i+1,p-1}^* \right]}_{N_{i,p}^*(u)} + \\ &+ (1 - \alpha_{i+1,p}) \underbrace{\left[ \frac{u - u_{i+1}^*}{u_{i+1+p}^* - u_{i+1}^*} N_{i+1,p-1}^* + \frac{u_{i+p+2}^* - u}{u_{i+p+2}^* - u_{i+2}^*} N_{i+2,p-1}^* \right]}_{N_{i+1,p}^*(u)}. \end{aligned}$$

□

### 6.2.9. Equivalencia de curvas bajo refinamiento

*Demostración.* Separaremos entre índices donde  $\alpha_j$  toma valores 0 o 1 y en los que no son tan triviales

- Para los índices  $j \leq k - p$ ,  $\alpha_j = 1$ ,  $\mathbf{Q}_j^w = (1)\mathbf{P}_j^w + (1-1)\mathbf{P}_{j-1}^w = \mathbf{P}_j^w$ ,  $N_{j,p} = N_{j,p}^*$ .
- Ahora para los índices  $j > k$ ,  $\alpha_j = 0$ ,  $\mathbf{Q}_j^w = (0)\mathbf{P}_j^w + (1-0)\mathbf{P}_{j-1}^w = \mathbf{P}_{j-1}^w$ ,  $N_{j-1,p} = N_{j,p}^*$ .
- Por último para los índices  $i \in \{k-p, k-p+1, \dots, k\}$ , obtenemos la definición de la influencia de estos índices mediante:

$$\mathbf{C}^w(u) = \sum_{i=k-p}^k N_{i,p}(u) \mathbf{P}_i^w.$$

descomponemos ahora las funciones base en las nuevas funciones base:

$$\mathbf{C}^w(u) = \sum_{i=k-p}^k [\alpha_i N_{i,p}^*(u) + (1 - \alpha_{i+1}) N_{i+1,p}^*(u)] \mathbf{P}_i^w.$$

Separamos en la suma en sus dos términos principales y reindexamos:

$$\begin{aligned} &\sum_{i=k-p}^k \alpha_i N_{i,p}^*(u) \mathbf{P}_i^w. \\ &\sum_{i=k-p}^k (1 - \alpha_{i+1}) N_{i+1,p}^*(u) \mathbf{P}_i^w = \sum_{j=k-p+1}^{k+1} (1 - \alpha_j) N_{j,p}^*(u) \mathbf{P}_{j-1}^w. \end{aligned}$$



Como hemos mostrado anteriormente la equivalencia de los términos de la serie para  $j = k - p$ ,  $j = k + 1$  y ambos términos aparecen con su función base y punto de control correspondiente una única vez, podemos eliminarlos de la suma y centrarnos en los índices de los que desconocemos si se cumple.

$$\begin{aligned} \mathbf{C}^w(u) &= \sum_{j=k-p+1}^k [\alpha_j N_{j,p}^*(u) \mathbf{P}_j^w + (1 - \alpha_j) N_{j,p}^*(u) \mathbf{P}_{j-1}^w] = \\ &= \sum_{j=k-p+1}^k N_{j,p}^*(u) [\alpha_j \mathbf{P}_j^w + (1 - \alpha_j) \mathbf{P}_{j-1}^w] = \sum_{j=k-p+1}^k N_{j,p}^*(u) \mathbf{Q}_j^w. \end{aligned}$$

□

## 6.3. Código omitido

### 6.3.1. FEM

**Código de las funciones base FEM**

---

```

1  clear; clc; close all;
2
3  % --- 1. PARÁMETROS CONFIGURABLES ---
4  L = 1;
5  num_elementos = 6;
6  p = 1;
7
8  % --- 2. GENERACIÓN DE LA MALLA ---
9  num_nodos = num_elementos * p + 1;
10 x_nodos = linspace(0, L, num_nodos);
11 colores = lines(num_nodos);
12
13 % Configuración de la figura
14 hold on; grid on;
15 axis([-0.1 L+0.1 -0.2 1.2]);
16
17 % --- 3. BUCLE DE FUNCIONES BASE ---
18 for i = 1:num_nodos
19     valores_nodales = zeros(1, num_nodos);
20     valores_nodales(i) = 1;
21     x_plot_total = [];
22     y_plot_total = [];
23     for e = 1:num_elementos
24         idx_ini = (e-1)*p + 1;
25         idx_fin = e*p + 1;
26         indices_locales = idx_ini:idx_fin;
27         x_elem = x_nodos(indices_locales);
28         y_elem = valores_nodales(indices_locales);
29         coeficientes = polyfit(x_elem, y_elem, p);
30         xx_elem = linspace(x_elem(1), x_elem(end), 50);
31         yy_elem = polyval(coeficientes, xx_elem);
32         x_plot_total = [x_plot_total, xx_elem];
33         y_plot_total = [y_plot_total, yy_elem];
34     end
35     % --- DIBUJO ---
36     plot(x_plot_total, y_plot_total, 'Color', colores(i,:), 'LineWidth', 2);
37     %try
38     %fill(x_plot_total, y_plot_total, colores(i,:), 'FaceAlpha', 0.1, 'EdgeColor', 'none');
39     %catch
40     %end
41     plot(x_nodos(i), 1, 'o', 'MarkerFaceColor', colores(i,:), 'MarkerEdgeColor', 'k');
42     if p == 1
43         offset = 0.05;
44     else
45         offset = 0.1 + 0.05*mod(i,2);
46     end
47     text(x_nodos(i), 1 + offset, sprintf('\phi_{%d}', i), ...
48         'Color', colores(i,:), 'HorizontalAlignment', 'center', 'FontWeight', 'bold');
49 end
50 for e = 0:num_elementos
51     x_borde = e * (L/num_elementos);
52     line([x_borde x_borde], [-0.2 1.2], 'Color', [0.7 0.7 0.7], 'LineStyle', '--');
53 end
54
55 hold off;
56
57 nombre_archivo = sprintf('bases_fem_p%d.svg', p);
58 set(gcf, 'PaperPositionMode', 'auto');
59 print(gcf, nombre_archivo, '-dsvg');

```

---

### 6.3.2. Figuras de Bézier

#### Código de las bases de Bernstein

---

```

1 function plot_bernstein_bases(p)
2
3     m = 100;
4     u = linspace(0, 1, m+1);
5
6     figure;
7     hold on;
8     grid on;
9
10    colors = hsv(p + 1);
11
12    for i = 0:p
13
14        binom_coeff = nchoosek(p, i);
15        poly_term = u.^i .* (1 - u).^(p - i);
16        B_ip = binom_coeff * poly_term;
17        plot(u, B_ip, 'LineWidth', 2, 'Color', colors(i + 1, :), ...
18             'DisplayName', ['B_{', num2str(i), ', ', num2str(p), '}(u)']);
19    end
20
21
22    xlim([0, 1]);
23    ylim([0, 1.1]);
24    hold off;
25
26    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
27    % EXPORTACIÓN EN FORMATO SVG
28    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
29
30    filename = ['bernstein_bases_p', num2str(p), '.svg'];
31    print(filename, '-dsvg');
32
33    disp(['Figura guardada como: ', filename]);
34 end

```

---

### Código de las curvas racionalizadas de Bernstein

---

```

1
2 m = 100;
3 u = 0:1/m:1;
4
5 P = [0, 1, 2, 3;
6       0, 5, 0, 4.5;
7       0, 1, 2, 4];
8
9 W = [1.0, 10.0, 7.0, 1.0];
10
11 [aux, n_puntos] = size(P);
12
13 sol = zeros(aux, m+1);
14
15 for k = 1:m+1
16
17     Bold = zeros(n_puntos, 1);
18     Bold(1) = 1;
19
20     for j = 2:n_puntos
21         Bnew = zeros(n_puntos, 1);
22         for i = 1:j-1
23             % B(i,j) = (1-u)*B(i,j-1) + u*B(i-1,j-1)
24             Bnew(i) = Bnew(i) + (1 - u(k)) * Bold(i);
25             Bnew(i+1) = Bnew(i+1) + u(k) * Bold(i);
26         endfor
27         Bold = Bnew;
28     endfor
29
30     %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
31     % RACIONALIZACIÓN
32     %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
33
34     Numerator = zeros(aux, 1);
35     Denominator = 0;
36
37     for j = 1:n_puntos
38
39         WeightedBasis = Bold(j) * W(j);
40         Numerator = Numerator + WeightedBasis * P(:, j);
41         Denominator = Denominator + WeightedBasis;
42     endfor
43
44     sol(:,k) = Numerator / Denominator;
45
46 endfor
47
48 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
49 % PLOTEO Y AJUSTE DE EJES DINÁMICOS
50 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
51
52 min_coords = min([P, sol], [], 2);
53 max_coords = max([P, sol], [], 2);
54
55 padding = 0.2;
56
57 min_X = min_coords(1) - padding;
58 max_X = max_coords(1) + padding;
59 min_Y = min_coords(2) - padding;
60 max_Y = max_coords(2) + padding;
61 min_Z = min_coords(3) - padding;
62 max_Z = max_coords(3) + padding;
63

```

```

64 figure
65 hold on
66
67 % Plotear el polígono de control (rojo discontinuo con círculos)
68 plot3(P(1,:), P(2,:), P(3,:), 'r--o', 'LineWidth', 1, 'MarkerSize', 5);
69
70 % Plotear la curva racional (azul sólido)
71 plot3(sol(1,:), sol(2,:), sol(3,:), 'b', 'LineWidth', 2);
72
73 % ** Establecer límites dinámicos **
74 xlim([min_X, max_X]);
75 ylim([min_Y, max_Y]);
76 zlim([min_Z, max_Z]);
77
78 xlabel('X');
79 ylabel('Y');
80 zlabel('Z');
81 grid on
82 view(3);
83 hold off
84
85 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
86 % EXPORTACIÓN DE LA FIGURA
87 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
88 print('curva_bezier_racional.svg', '-dsvg');

```

### 6.3.3. Código fundamental para el funcionamiento en C++

#### Clase iMatrix

```

1  /**
2  *AUTHOR: Carlos Palomera Oliva
3  *DATE: MARCH 2025
4  */
5  #include <iostream>
6  #include <stdexcept>
7  #include <algorithm>
8  #include "C:\Librerias\eigen-3.4.0\Eigen\Dense" //windows version
9  // #include "/home/carlos/Librerias/eigen-3.4.0/Eigen/Dense" //linux version
10 // #include "/home/carlos/Librerias/eigen-3.4.0/Eigen/Sparse" //linux version
11 #include <iomanip>
12 #include <vector>
13 #include <cmath>
14 #include <numeric>
15 #include <type_traits>
16
17 #ifndef IMATRIX_H
18 #define IMATRIX_H
19
20 // This file is part of the qbLinAlg linear algebra library.
21
22 /*
23 MIT License
24 Copyright (c) 2023 Michael Bennett
25
26 Permission is hereby granted, free of charge, to any person obtaining a copy of this software
27 and associated documentation files (the "Software"), to deal in the Software without restriction,
28 including without limitation the rights to use, copy, modify, merge, publish, distribute,
29 sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is
30 furnished to do so, subject to the following conditions:
31
32 The above copyright notice and this permission notice shall be included in all copies or
33 substantial portions of the Software.
34
35 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING
36 BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
37 NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM,
38 DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
39 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
40 */
41
42
43
44
45 template <class T>
46 class iMatrix{
47 public:
48     // Define the various constructors.
49     iMatrix();
50     iMatrix(int nRows, int nCols);
51     iMatrix(int nRows, int nCols, const T *inputData);

```

```

52 iMatrix(int nRows, int nCols, const Eigen::VectorXd &inputData);
53 iMatrix(const iMatrix<T> &inputMatrix);
54
55 // And the destructor.
56 ~iMatrix();
57
58 // Configuration methods.
59 bool Resize(int numRows, int numCols);
60 void SetToIdentity();
61
62 // Element access methods.
63 T GetElement(int row, int col) const;
64 std::vector<T> GetRow(int row) const;
65 std::vector<T> GetCol(int col) const;
66 bool SetElement(int row, int col, T elementValue);
67 bool SetCol(int col, std::vector<T> elements);
68 bool SetRow(int row, std::vector<T> elements);
69 int GetNumRows() const;
70 int GetNumCols() const;
71 iMatrix<T> GetSubMat(int rowIni, int rowEnd, int colIni, int colEnd) const;
72
73
74 // Overload the assignment operator.
75 iMatrix<T> operator= (const iMatrix<T> &rhs);
76
77
78 // Write
79 inline T& operator()(int i, int j) noexcept {
80     return m_matrixData[i * m_nCols + j];
81 };
82
83 // Read
84 inline const T& operator()(int i, int j) const noexcept {
85     return m_matrixData[i * m_nCols + j];
86 };
87
88
89 iMatrix<T> invert() const { //Uses LU factorization with partial pivoting
90     if (m_nRows != m_nCols) {
91         throw std::runtime_error("Matrix must be square for inversion.");
92     }
93
94     int n = m_nRows;
95     iMatrix<T> A(*this);
96     std::vector<int> piv(n);
97     for (int i = 0; i < n; ++i) piv[i] = i;
98
99     for (int k = 0; k < n; ++k) {
100         T maxVal = std::abs(A(k, k));
101         int pivotRow = k;
102         for (int i = k + 1; i < n; ++i) {
103             T val = std::abs(A(i, k));
104             if (val > maxVal) {
105                 maxVal = val;
106                 pivotRow = i;
107             }
108         }
109         if (maxVal == T(0)) {
110             throw std::runtime_error("Matrix is singular.");
111         }
112
113         if (pivotRow != k) {
114             for (int j = 0; j < n; ++j) {
115                 std::swap(A(k, j), A(pivotRow, j));
116             }
117             std::swap(piv[k], piv[pivotRow]);
118         }
119
120         for (int i = k + 1; i < n; ++i) {
121             A(i, k) /= A(k, k);
122             T factor = A(i, k);
123             for (int j = k + 1; j < n; ++j) {
124                 A(i, j) -= factor * A(k, j);
125             }
126         }
127     }
128
129     iMatrix<T> inv(n, n);
130     for (int col = 0; col < n; ++col) {
131         std::vector<T> b(n, T(0));
132         b[piv[col]] = T(1);
133
134         for (int i = 0; i < n; ++i) {
135             for (int j = 0; j < i; ++j) {
136                 b[i] -= A(i, j) * b[j];
137             }
138         }
139
140         for (int i = n - 1; i >= 0; --i) {
141             for (int j = i + 1; j < n; ++j) {
142                 b[i] -= A(i, j) * b[j];
143             }
144             b[i] /= A(i, i);
145         }
146
147         for (int i = 0; i < n; ++i) {
148             inv(i, col) = b[i];
149         }
150     }

```

```

151         return inv;
152     }
153     // Overload +, - and * operators (friends).
154     template <class U> friend iMatrix<U> operator+ (const iMatrix<U>& lhs, const iMatrix<U>& rhs);
155     template <class U> friend iMatrix<U> operator+ (const U& lhs, const iMatrix<U>& rhs);
156     template <class U> friend iMatrix<U> operator+ (const iMatrix<U>& lhs, const U& rhs);
157
158     template <class U> friend iMatrix<U> operator- (const iMatrix<U>& lhs, const iMatrix<U>& rhs);
159     template <class U> friend iMatrix<U> operator- (const U& lhs, const iMatrix<U>& rhs);
160     template <class U> friend iMatrix<U> operator- (const iMatrix<U>& lhs, const U& rhs);
161
162     template <class U> friend iMatrix<U> operator* (const iMatrix<U>& lhs, const iMatrix<U>& rhs);
163     template <class U> friend iMatrix<U> operator* (const U& lhs, const iMatrix<U>& rhs);
164     template <class U> friend iMatrix<U> operator* (const iMatrix<U>& lhs, const U& rhs);
165
166     template <class U> friend iMatrix<U> operator* (const std::vector<U>& v1, const std::vector<U>& v2);
167
168     iMatrix<T>& operator+= (const iMatrix<T>& rhs);
169     iMatrix<T>& operator-= (const T& rhs);
170
171     void PrintMatrix();
172     void PrintMatrix(int precision);
173     std::vector<T> Vectorize() const;
174     iMatrix<T> Transpose() const;
175
176     bool IsSquare();
177     T Determinant() const;
178
179     // Función estática para calcular el Producto Punto entre dos std::vector<T>.
180     static T dot_product (const std::vector<T>& v1, const std::vector<T>& v2);
181
182 private:
183     int Sub2Ind(int row, int col) const;
184 private:
185     std::vector<T> m_matrixData;
186     int m_nRows, m_nCols;
187 };
188
189
190
191
192 /* ***** CONSTRUCTOR / DESTRUCTOR FUNCTIONS ***** */
193
194 template <class T>
195 iMatrix<T>::iMatrix() : m_nRows(1), m_nCols(1)
196 {
197     // Inicializamos el vector con 1 elemento (por defecto 0 o T())
198     m_matrixData.resize(1);
199 }
200
201 // Construct empty matrix (all elements 0)
202 template <class T>
203 iMatrix<T>::iMatrix(int nRows, int nCols) : m_nRows(nRows), m_nCols(nCols)
204 {
205     int nElements = m_nRows * m_nCols;
206     if constexpr (std::is_arithmetic_v<T>) {
207         // Redimensionar e inicializar a 0.0f
208         m_matrixData.resize(nElements, static_cast<T>(0.0f));
209     } else {
210         // Redimensionar e inicializar con el constructor por defecto T()
211         m_matrixData.resize(nElements);
212     }
213 }
214
215 // Construct from Eigen::VectorXd
216 template <class T>
217 iMatrix<T>::iMatrix(int nRows, int nCols, const Eigen::VectorXd& inputData)
218 : m_nRows(nRows), m_nCols(nCols)
219 {
220     int nElements = m_nRows * m_nCols;
221     if (inputData.size() != nElements)
222         throw std::invalid_argument("Eigen::VectorXd no tiene el tamaño correcto para llenar la matriz.");
223     m_matrixData.resize(nElements);
224     for (int i = 0; i < nElements; ++i)
225         m_matrixData[i] = static_cast<T>(inputData[i]);
226 }
227
228 // Construct from const linear array.
229 template <class T>
230 iMatrix<T>::iMatrix(int nRows, int nCols, const T *inputData) : m_nRows(nRows), m_nCols(nCols)
231 {
232     int nElements = m_nRows * m_nCols;
233     // Usamos el constructor de rango de std::vector para la copia eficiente.
234     m_matrixData = std::vector<T>(inputData, inputData + nElements);
235 }
236
237 // The copy constructor.
238 template <class T>
239 iMatrix<T>::iMatrix(const iMatrix<T> &inputMatrix)
240 : m_nRows(inputMatrix.m_nRows),
241   m_nCols(inputMatrix.m_nCols),
242   m_matrixData(inputMatrix.m_matrixData)
243 {
244     // Cuerpo del constructor vacío (RAII).
245 }
246
247 template <class T>
248 iMatrix<T>::~iMatrix()
249 {

```

```

250 // El destructor de std::vector se llama automáticamente (RAII).
251 }
252 /* *****
253 CONFIGURATION FUNCTIONS
254 /* *****
255 template <class T>
256 bool iMatrix<T>::Resize(int numRows, int numCols)
257 {
258     m_nRows = numRows;
259     m_nCols = numCols;
260     int nElements = (m_nRows * m_nCols);
261
262     if constexpr (std::is_arithmetic_v<T>) {
263         m_matrixData.resize(nElements, static_cast<T>(0.0f));
264     } else {
265         m_matrixData.resize(nElements);
266     }
267
268     return true;
269 }
270
271 template <class T>
272 void iMatrix<T>::SetToIdentity()
273 {
274     if (!IsSquare())
275         throw std::invalid_argument("Cannot form an identity matrix that is not square.");
276
277     for (int row=0; row<m_nRows; ++row)
278     {
279         for (int col=0; col<m_nCols; ++col)
280         {
281             if (col == row)
282                 m_matrixData[Sub2Ind(row,col)] = 1.0;
283             else
284                 m_matrixData[Sub2Ind(row,col)] = 0.0;
285         }
286     }
287 }
288
289 template <class T>
290 int iMatrix<T>::GetNumRows() const
291 {
292     return m_nRows;
293 }
294
295 template <class T>
296 int iMatrix<T>::GetNumCols() const
297 {
298     return m_nCols;
299 }
300 /* *****
301 ELEMENT FUNCTIONS
302 /* *****
303 template <class T>
304 T iMatrix<T>::GetElement(int row, int col) const
305 {
306     int linearIndex = Sub2Ind(row, col);
307     if (linearIndex >= 0)
308         return m_matrixData[linearIndex];
309     else{
310         if constexpr (std::is_arithmetic_v<T>) {
311             return 0; // Para tipos numericos (int, float, double)
312         } else {
313             return T(); // Para contenedores (std::vector<float>, etc.)
314         }
315     }
316 }
317
318 template <class T>
319 std::vector<T> iMatrix<T>::GetRow(int row) const
320 {
321     int linearIndex = Sub2Ind(row, 0);
322     std::vector<T> AuxRow;
323     for(unsigned int i=0; i<this->GetNumCols(); i++){
324         AuxRow.push_back(m_matrixData[linearIndex]);
325         linearIndex++;
326     }
327     return AuxRow;
328 }
329
330 template <class T>
331 bool iMatrix<T>::SetElement(int row, int col, T elementValue)
332 {
333     int linearIndex = Sub2Ind(row, col);
334     if (linearIndex >= 0)
335     {
336         m_matrixData[linearIndex] = elementValue;
337         return true;
338     }
339     else
340     {
341         return false;
342     }
343 }
344
345 template <class T>
346 bool iMatrix<T>::SetCol(int col, std::vector<T> elements){
347     if (col < 0 || col >= m_nCols || static_cast<int>(elements.size()) != m_nRows)
348 
```



```

349     return false;
350     for (int i = 0; i < m_nRows; ++i) {
351         m_matrixData[i * m_nCols + col] = elements[i];
352     }
353     return true;
354 }
355
356
357 template <class T>
358 bool iMatrix<T>::SetRow(int row, std::vector<T> elements){
359     int linearIndex = Sub2Ind(row, 0);
360     int aux =this->GetNumCols();
361     if (linearIndex >= 0){
362         for(unsigned int i=0;i<aux;i++){
363             (*this)(row, i) = elements[i];
364             //this->SetElement(row,i,elements[i]);
365             linearIndex=linearIndex+aux;
366         }
367         return true;
368     }
369     else{
370         return false;
371     }
372 }
373
374
375 template <class T>
376 std::vector<T> iMatrix<T>::GetCol(int col) const
377 {
378     std::vector<T> result(m_nRows);
379     for (int i = 0; i < m_nRows; ++i) {
380         result[i] = m_matrixData[i * m_nCols + col];
381     }
382     return result;
383 }
384
385
386 template <class T>
387 iMatrix<T> iMatrix<T>::GetSubMat(int rowIni, int rowEnd, int colIni, int colEnd) const
388 {
389     int numRows= rowEnd-rowIni+1;
390     int numCols= colEnd-colIni+1;
391
392     int aux =this->GetNumCols();
393     iMatrix<T> AuxMat(numRows, numCols);
394
395     for(unsigned int i=0; i< numRows; i++){
396         for(unsigned int j=0; j<numCols; j++){
397             AuxMat(i,j)=(*this)(i+rowIni, j+colIni);
398         }
399     }
400     return AuxMat;
401 }
402
403
404 /* *****
405 THE + OPERATOR
406 /* *****
407 // matrix + matrix.
408 template <class T>
409 iMatrix<T> operator+ (const iMatrix<T>& lhs, const iMatrix<T>& rhs)
410 {
411     int numRows = lhs.m_nRows;
412     int numCols = lhs.m_nCols;
413     int numElements = lhs.m_matrixData.size();
414     T *tempResult = new T[numElements];
415     for (int i=0; i<numElements; i++)
416         tempResult[i] = lhs.m_matrixData[i] + rhs.m_matrixData[i];
417
418     iMatrix<T> result(numRows, numCols, tempResult);
419     delete[] tempResult;
420     return result;
421 }
422
423 // scaler + matrix
424 template <class T>
425 iMatrix<T> operator+ (const T& lhs, const iMatrix<T>& rhs)
426 {
427     int numRows = rhs.m_nRows;
428     int numCols = rhs.m_nCols;
429     int numElements = rhs.m_matrixData.size();
430     T *tempResult = new T[numElements];
431     for (int i=0; i<numElements; ++i)
432         tempResult[i] = lhs + rhs.m_matrixData[i];
433
434     iMatrix<T> result(numRows, numCols, tempResult);
435     delete[] tempResult;
436     return result;
437 }
438
439 // matrix + scaler
440 template <class T>
441 iMatrix<T> operator+ (const iMatrix<T>& lhs, const T& rhs)
442 {
443     int numRows = lhs.m_nRows;
444     int numCols = lhs.m_nCols;
445     int numElements = lhs.m_matrixData.size();
446     T *tempResult = new T[numElements];
447     for (int i=0; i<numElements; ++i)
448         tempResult[i] = lhs.m_matrixData[i] + rhs;

```

```

448     iMatrix<T> result(numRows, numCols, tempResult);
449     delete[] tempResult;
450     return result;
451 }
452
453 /* *****
454 THE - OPERATOR
455 /* *****
456 // matrix - matrix
457 template <class T>
458 iMatrix<T> operator- (const iMatrix<T>& lhs, const iMatrix<T>& rhs)
459 {
460     int numRows = lhs.m_nRows;
461     int numCols = lhs.m_nCols;
462     int numElements = lhs.m_matrixData.size();
463     T *tempResult = new T[numElements];
464     for (int i=0; i<numElements; i++)
465         tempResult[i] = lhs.m_matrixData[i] - rhs.m_matrixData[i];
466
467     iMatrix<T> result(numRows, numCols, tempResult);
468     delete[] tempResult;
469     return result;
470 }
471
472 // scaler - matrix
473 template <class T>
474 iMatrix<T> operator- (const T& lhs, const iMatrix<T>& rhs)
475 {
476     int numRows = rhs.m_nRows;
477     int numCols = rhs.m_nCols;
478     int numElements = rhs.m_matrixData.size();
479     T *tempResult = new T[numElements];
480     for (int i=0; i<numElements; ++i)
481         tempResult[i] = lhs - rhs.m_matrixData[i];
482
483     iMatrix<T> result(numRows, numCols, tempResult);
484     delete[] tempResult;
485     return result;
486 }
487
488 // matrix - scaler
489 template <class T>
490 iMatrix<T> operator- (const iMatrix<T>& lhs, const T& rhs)
491 {
492     int numRows = lhs.m_nRows;
493     int numCols = lhs.m_nCols;
494     int numElements = lhs.m_matrixData.size();
495     T *tempResult = new T[numElements];
496     for (int i=0; i<numElements; ++i)
497         tempResult[i] = lhs.m_matrixData[i] - rhs;
498
499     iMatrix<T> result(numRows, numCols, tempResult);
500     delete[] tempResult;
501     return result;
502 }
503
504 // scaler * matrix
505 template <class T>
506 iMatrix<T> operator* (const T& lhs, const iMatrix<T>& rhs)
507 {
508     iMatrix<T> result(rhs.m_nRows, rhs.m_nCols);
509     std::transform(
510         rhs.m_matrixData.begin(),
511         rhs.m_matrixData.end(),
512         result.m_matrixData.begin(),
513         [&lhs](const T& element) { return lhs * element; }
514     );
515     return result;
516 }
517
518 /* Old product operator for the scaler
519
520 template <class T>
521 iMatrix<T> operator* (const T& lhs, const iMatrix<T>& rhs)
522 {
523     int numRows = rhs.m_nRows;
524     int numCols = rhs.m_nCols;
525     int numElements = rhs.m_matrixData.size();
526     T *tempResult = new T[numElements];
527     for (int i=0; i<numElements; ++i)
528         tempResult[i] = lhs * rhs.m_matrixData[i];
529
530     iMatrix<T> result(numRows, numCols, tempResult);
531     delete[] tempResult;
532     return result;
533 }
534
535
536 template <class T>
537 iMatrix<T> operator* (const iMatrix<T>& lhs, const T& rhs)
538 {
539     int numRows = lhs.m_nRows;
540     int numCols = lhs.m_nCols;
541     int numElements = lhs.m_matrixData.size();
542     T *tempResult = new T[numElements];
543     for (int i=0; i<numElements; ++i)
544         tempResult[i] = lhs.m_matrixData[i] * rhs;
545
546     iMatrix<T> result(numRows, numCols, tempResult);
547     delete[] tempResult;
548     return result;
549 }

```

```

547     iMatrix<T> result(numRows, numCols, tempResult);
548     delete[] tempResult;
549     return result;
550 }
551
552 // matrix * scalar
553 template <class T>
554 iMatrix<T> operator* (const iMatrix<T>& lhs, const T& rhs)
555 {
556     iMatrix<T> result(lhs.m_nRows, lhs.m_nCols);
557
558     std::transform(
559         lhs.m_matrixData.begin(),
560         lhs.m_matrixData.end(),
561         result.m_matrixData.begin(),
562         [&rhs](const T& element) { return element * rhs; }
563     );
564
565     return result;
566 }
567
568 // matrix * matrix
569 template <class T>
570 iMatrix<T> operator* (const iMatrix<T>& lhs, const iMatrix<T>& rhs)
571 {
572     int r_numRows = rhs.m_nRows;
573     int r_numCols = rhs.m_nCols;
574     int l_numRows = lhs.m_nRows;
575     int l_numCols = lhs.m_nCols;
576
577     if (l_numCols == r_numRows)
578     {
579         iMatrix<T> rhs_T(r_numCols, r_numRows);
580         for (int i = 0; i < r_numRows; ++i) {
581             for (int j = 0; j < r_numCols; ++j) {
582                 rhs_T(j, i) = rhs(i, j);
583             }
584         }
585
586         T *tempResult = new T[lhs.m_nRows * rhs.m_nCols];
587
588         for (int lhsRow=0; lhsRow<l_numRows; lhsRow++)
589         {
590             for (int rhsCol=0; rhsCol<r_numCols; rhsCol++)
591             {
592                 T elementResult = static_cast<T>(0.0);
593                 int lhsLinearIndex = (lhsRow * l_numCols);
594                 int rhsTLinearIndex = (rhsCol * r_numRows);
595
596                 for (int k=0; k<l_numCols; k++)
597                 {
598                     elementResult += (lhs.m_matrixData[lhsLinearIndex + k] * rhs_T.m_matrixData[rhsTLinearIndex + k]);
599                 }
600                 int resultLinearIndex = (lhsRow * r_numCols) + rhsCol;
601                 tempResult[resultLinearIndex] = elementResult;
602             }
603         }
604
605         iMatrix<T> result(l_numRows, r_numCols, tempResult);
606         delete[] tempResult;
607         return result;
608     }
609     else
610     {
611         iMatrix<T> result(1, 1);
612         return result;
613     }
614 }
615
616 // matrix assignment
617 template <class T>
618 iMatrix<T> iMatrix<T>::operator= (const iMatrix<T> &rhs)
619 {
620     if (this != &rhs)
621     {
622         m_nRows = rhs.m_nRows;
623         m_nCols = rhs.m_nCols;
624         m_matrixData = rhs.m_matrixData;
625     }
626
627     return *this;
628 }
629
630 // matrix vector multiplication
631 template <class T>
632 iMatrix<T> operator* (const std::vector<T>& v1, const std::vector<T>& v2)
633 {
634     int N = v1.size();
635     int M = v2.size();
636
637     iMatrix<T> result(N, M);
638     for (int i = 0; i < N; ++i)
639     {
640         T v1_i = v1[i];
641
642         for (int j = 0; j < M; ++j)

```

```

646     {
647         result(i, j) = v1_i * v2[j];
648     }
649 }
650
651
652     return result;
653 }
654
655 template <class T>
656 iMatrix<T>& iMatrix<T>::operator+= (const iMatrix<T>& rhs)
657 {
658     if (m_nRows != rhs.m_nRows || m_nCols != rhs.m_nCols)
659     {
660         throw std::invalid_argument("Error: Las matrices deben tener las mismas dimensiones para la suma in-place.");
661     }
662
663     int numElements = m_matrixData.size();
664     T* data_lhs = m_matrixData.data();
665     const T* data_rhs = rhs.m_matrixData.data();
666     for (int i = 0; i < numElements; ++i)
667     {
668         data_lhs[i] += data_rhs[i];
669     }
670     return *this;
671 }
672
673 template <class T>
674 iMatrix<T>& iMatrix<T>::operator*= (const T& rhs)
675 {
676     int numElements = m_matrixData.size();
677     T* data_lhs = m_matrixData.data();
678     for (int i = 0; i < numElements; ++i)
679     {
680         data_lhs[i] *= rhs;
681     }
682
683     return *this;
684 }
685
686 // A simple function to print a matrix to stdout.
687 template <class T>
688 void iMatrix<T>::PrintMatrix()
689 {
690     int nRows = this->GetNumRows();
691     int nCols = this->GetNumCols();
692     for (int row = 0; row < nRows; ++row)
693     {
694         for (int col = 0; col < nCols; ++col)
695         {
696             std::cout << std::fixed << std::setprecision(3) << (*this)(row, col) << " ";
697         }
698         std::cout << std::endl;
699     }
700 }
701
702 // A simple function to print a matrix to stdout, with specified precision.
703 template <class T>
704 void iMatrix<T>::PrintMatrix(int precision)
705 {
706     int nRows = this->GetNumRows();
707     int nCols = this->GetNumCols();
708     for (int row = 0; row < nRows; ++row)
709     {
710         for (int col = 0; col < nCols; ++col)
711         {
712             std::cout << std::fixed << std::setprecision(precision) << (*this)(row, col) << " ";
713         }
714         std::cout << std::endl;
715     }
716 }
717
718 // Function to test whether the matrix is square.
719 template <class T>
720 bool iMatrix<T>::IsSquare()
721 {
722     if (m_nCols == m_nRows)
723         return true;
724     else
725         return false;
726 }
727
728 template <class T>
729 T iMatrix<T>::Determinant() const
730 {
731     if (!this->IsSquare()){
732         return static_cast<T>(0.0);
733     }
734
735     int N = this->m_nRows;
736     if (N == 1){
737         return (*this)(0, 0);
738     }
739     iMatrix<T> tempMatrix = *this;
740
741     T det = static_cast<T>(1.0);
742     int sign = 1;
743
744

```

```

745     for (int k = 0; k < N; ++k){
746         int pivotRow = k;
747         T maxValue = std::abs(tempMatrix(k, k));
748
749         for (int i = k + 1; i < N; ++i)
750         {
751             if (std::abs(tempMatrix(i, k)) > maxValue)
752             {
753                 maxValue = std::abs(tempMatrix(i, k));
754                 pivotRow = i;
755             }
756         }
757
758         if (pivotRow != k){
759             for (int j = 0; j < N; ++j){
760                 std::swap(tempMatrix(k, j), tempMatrix(pivotRow, j));
761             }
762
763             sign = -sign;
764         }
765
766         if (std::abs(tempMatrix(k, k)) < static_cast<T>(1e-9)){
767             return static_cast<T>(0.0);
768         }
769
770         T pivotValue = tempMatrix(k, k);
771         for (int i = k + 1; i < N; ++i) {
772             T factor = tempMatrix(i, k) / pivotValue;
773
774             for (int j = k; j < N; ++j){
775                 tempMatrix(i, j) -= factor * tempMatrix(k, j);
776             }
777         }
778
779         det *= pivotValue;
780     }
781     return sign * det;
782 }
783
784 template <class T>
785 std::vector<T> iMatrix<T>::Vectorize() const
786 {
787     return m_matrixData;
788 }
789
790 template <class T>
791 iMatrix<T> iMatrix<T>::Transpose() const
792 {
793     iMatrix<T> result(m_nCols, m_nRows); // ← filas y columnas intercambiadas
794
795     // Recorremos todos los elementos de la matriz original
796     for (int i = 0; i < m_nRows; ++i)
797     {
798         for (int j = 0; j < m_nCols; ++j)
799         {
800             // Elemento (i,j) en la original → posición (j,i) en la transpuesta
801             result(j, i) = (*this)(i, j);
802         }
803     }
804
805     return result;
806 }
807
808
809
810 template <class T>
811 T dot_product (const std::vector<T>& v1, const std::vector<T>& v2)
812 {
813     if (v1.size() != v2.size())
814     {
815         throw std::invalid_argument("Error: Los vectores deben tener la misma dimensión para el Producto Punto.");
816     }
817
818     int N = v1.size();
819     T resultValue = static_cast<T>(0.0);
820
821     for (int i = 0; i < N; ++i)
822     {
823         resultValue += v1[i] * v2[i];
824     }
825
826     return resultValue;
827 }
828
829 /* *****
830 PRIVATE FUNCTIONS
831 /* *****
832 // Function to return the linear index corresponding to the supplied row and column values.
833 template <class T>
834 int iMatrix<T>::Sub2Ind(int row, int col) const
835 {
836     if ((row < m_nRows) && (row >= 0) && (col < m_nCols) && (col >= 0))
837         return (row * m_nCols) + col;
838     else
839         return -1;
840 }
841
842

```

843 #endif

## Fichero funciones.hpp

```

1  /*****
2  *AUTHOR: Carlos Palomera Oliva
3  *DATE: MARCH 2025
4  *****/
5  #define _USE_MATH_DEFINES
6  #include <iostream>
7  #include <unordered_map>
8  #include <utility>
9  #include <cmath>
10 #include <iomanip>
11 #include <vector>
12 #include <algorithm>
13 #include <fstream>
14 #include <string>
15 #include <limits>
16 #include "iMatrix.cpp"
17 #include "C:\Librerias\eigen-3.4.0\Eigen\Sparse" //windows version
18 #include "C:\Librerias\eigen-3.4.0\Eigen\Dense" //windows version
19 // #include "/home/carlos/Librerias/eigen-3.4.0/Eigen/Dense"
20 // #include "/home/carlos/Librerias/eigen-3.4.0/Eigen/Sparse"
21 #ifndef FUNCIONES_HPP
22 #define FUNCIONES_HPP
23
24
25
26
27 /*****
28 * Structure of the Data File for Matrix
29 * Nrow Ncol
30 * vector of size Ncol representing the first variable where elements are separated by space Ex(x1 x2 x3 ...)
31 * vector of size Ncol ...
32 * .
33 * . Nrow times in total
34 * .
35 *
36 *****/
37 /*
38 *Input: Direction of the data file
39 *Output: iMatrix filled with the control points in data file
40 */
41 iMatrix<double> ReadDataFile_Matrix(std::string direction);
42
43 /*****
44 * Structure of the Data File for Tensors
45 * Ncol NSplits
46 * vector of size Ncol representing the first variable where elements are separated by space Ex(x1 x2 x3 ...)
47 * vector of size Ncol ...
48 * .
49 * . 3 times in total
50 * .
51 * -----
52 * The previous repeated NSplits totals
53 *****/
54 /*
55 *Input: Direction of the data file
56 *Output: vector of iMatrixes filled with the control points in data file
57 */
58 std::vector<iMatrix<double>> ReadDatafile_CtrlPts(std::string direction);
59
60
61 /*
62 *Input: Value of the point where the polynomial is evaluated t, and degree of the polynomial n
63 *Condition: t∈[0,1]
64 *Output: Values of all the Bernstein polynomial of degree n evaluated at t
65 */
66 std::vector<double> BernsteinPolynomial(double t, int n);
67
68
69 /*
70 *Input: iMatrix of control points and parameter of the curve t
71 *Conditions: t∈[0,1]
72 *Output: Value of the Bezier curve of such control points in the point t
73 */
74 std::vector<double> BezierCurve_Point_t(const iMatrix<double> &PtosControl, double t);
75
76
77 /*
78 *Input: iMatrix of control points and number of steps for the curve
79 *Conditions: Steps must be >= 1
80 *Output: Value of the Bezier curve at every point from 0 to 1 separated in N_Steps steps
81 */
82 iMatrix<double> BezierCurve(const iMatrix<double> &PtosControl, double N_Steps);
83
84 /*
85 *Input: Tensor of control points and the 2 parameters of the curve t1, t2
86 *Conditions: t1,t2∈[0,1]
87 *Output: Value of the Bezier surface at of such control points in the points t1 t2
88 */
89 std::vector<double> BezierSurface_Point_t1_t2(const std::vector<iMatrix<double>> &PtosControl, double t1, double t2);
90
91 /*
92 *Input: iMatrix of control points and number of steps for the curve in both parameters

```

```

93  *Conditions: N_Steps must be >= 1
94  *Output: Value of the Bezier surface at every point from 0 to 1 separated in N_Steps along both parameters
95  */
96  std::vector<iMatrix<double>> BezierSurface(const std::vector<iMatrix<double>> &PtsControl, double N_Steps);
97
98  /*
99  *Input: Bezier surface in form of a tensor
100  *Output: Writes a file of 3 matrices one for each coordinate
101  */
102  void Write_BezierSurface(const std::vector<iMatrix<double>> &Tensor, std::string name);
103
104  /*
105  *Input: CtrlPts in format described before
106  *Output: Writes a file of 3 matrices one for each coordinate
107  */
108  void Write_CtrlPts(const std::vector<iMatrix<double>> &CtrlPts, std::string name);
109
110
111
112  /*****
113  *
114  * Classic funtions from the Nurbs Book
115  *
116  *****/
117
118  /*
119  *Input: A KnotVector, t the point of evaluation, its polynomial degree, and n=the size of the Nurbs vector-p-2
120  *Output: the span index
121  */
122  int FindSpan(const std::vector<double> &KnotVector, double t, int p, int n);
123
124  /*
125  *Input: i span index, t eval point, p polinomial degree and KnotVector
126  *Output: vector of basis funcs
127  */
128  std::vector<double> BasisFuns(int i, double t, int p, const std::vector<double> &KnotVector);
129
130  /*
131  *Input: i span index, t eval point, p polinomial degree and KnotVector
132  *Output: iMatrix where the i-th row is the i-th degree basis fun
133  */
134  iMatrix<double> AllBasisFuns(int i, double t, int p, const std::vector<double> &KnotVector);
135
136  /*
137  *Input: i span index, t eval point, p polinomial degree, k derivate degree, KnotVector and an output vector ders;
138  *Output: value of the k-derivate at point t in the ders vector on the k-row of the matrix
139  */
140  iMatrix<double> DerBasisFuns(int span, double t, int p, int k, const std::vector<double> &KnotVector);
141
142  /*
143  *Input: matrix of BasisFunctions or its derivatives and the path where they will be written
144  *Output: Writes the Functions as a matrix on selected path
145  */
146  void Write_BasisFunctions(const iMatrix<double> &BasisFunctions, std::string name);
147
148  /*****
149  *
150  * Classic funtions for p spline curves/surfaces
151  *
152  *****/
153
154  /*
155  *Input: n=the size of the Nurbs vector-p-2, p polynomial degree, KnotVector, the CtrlPts ,t the point of evaluation
156  *Output: Evaluation of the curve at point t
157  */
158  std::vector<double> CurvePoint(int n, int p, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPts, double t);
159
160  /*
161  *Input: n=the size of the Nurbs vector-p-2, p polynomial degree, KnotVector, the CtrlPts ,
162  *      t the point of evaluation and d highest derivate order to compute
163  *Output: iMatrix where the i-th row is the evaluation of the i-th derivate
164  */
165  iMatrix<double> CurveDerivsAlg1(int n, int p, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPts, double t, int d);
166
167  /*
168  *Input: n=the size of the Nurbs vector-p-2, p polynomial degree, KnotVector, the CtrlPts ,t the point of evaluation,
169  *      d highest derivate order to compute, r1 and r2 specify the intervals of CtrlPoints
170  *Output: iMatrix where the i-th row is the vector of ctrl points
171  */
172  iMatrix<std::vector<double>> CurveDerivsOpts(int n, int p, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPts,
173      int d, int r1, int r2);
174
175  /*
176  *Input: n=the size of the Nurbs vector-p-2, p polynomial degree, KnotVector, the CtrlPts ,
177  *      t the point of evaluation and d highest derivate order to compute
178  *Output: iMatrix where the i-th row is the evaluation of the i-th derivate
179  */
180  iMatrix<double> CurveDerivsAlg2(int n, int p, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPts, double t, int d);
181
182  /*
183  *Input: Curve and path
184  *Output: Writes the values of the Curve in the selected path
185  */
186  void Write_Curve(const iMatrix<double> &Curve, std::string filename);
187
188  /*
189  *Input: n_i=the size of the Nurbs vector_i-p-2, p_i polynomial degree, KnotVector_i, the CtrlPts ,t_i the point of evaluation
190  *Output: Evaluation of the surface at point (t1,t2)
191  */

```

```

192 std::vector<double> SurfacePoint(int n1, int p1,const std::vector<double> &KnotVector1, int n2, int p2,const std::vector<double> &KnotVector2,
193                                const std::vector<iMatrix<double>> &CtrlPts, double t1, double t2);
194
195 /*
196 *Input: n_i=the size of the Nurbs vector_i-p-2, p_i polynomial degree, KnotVector_i, the CtrlPts ,
197 *      t_i the point of evaluation and d maximum order of the derivatives
198 *Output: Evaluation of the derivatives up to degree d (maximum is p or q and would overwrite it if d>p,q) in (t1,t2) where Output[k_1][k_2] is been
199         derived k_i times respect to the ith variable
200 */
201 iMatrix<std::vector<double>> SurfaceDerivsAlg1(int n1, int p1,const std::vector<double> &KnotVector1, int n2, int p2,
202                                              const std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPts, double t1, double t2, int d);
203
204
205 /*****
206 *
207 * Classic and support funtions for NURBS curves/surfaces
208 *
209 *****/
210
211 /*
212 *Input: Vector of CtrlPts and their weights at each point in a separated vector
213 *Output: iMatrix that each column is each of the coordinates of each CtrlPt except its last component that is it's respective weight
214 */
215 iMatrix<double> WeightCtrlPts(const iMatrix<double> &CtrlPts,const std::vector<double> &Weights);
216
217 /*
218 *Input: CtrlPts multiplied by their Weight with the respective weight at the end of each column
219 *Output: The original CtrlPts
220 */
221 iMatrix<double> UnWeightCtrlPts(const iMatrix<double> &CtrlPtsW);
222
223 /*
224 *Input: Vector of CtrlPts and their weights at each point in a separated vector
225 *Output: std::vector<iMatrix<double>> that keeps the format of the CtrlPts for Surface
226         operations with and extra 4th dimension on each coord thats's occupied by the weight of such pt
227 */
228 std::vector<iMatrix<double>> WeightCtrlPtsSurf(const std::vector<iMatrix<double>> &CtrlPts,const iMatrix<double> &Weights);
229
230 /*
231 *Input: CtrlPts multiplied by their Weight with the respective weight at the end of each column
232 *Output: The original CtrlPts
233 */
234 std::vector<iMatrix<double>> UnWeightCtrlPtsSurf(const std::vector<iMatrix<double>> &CtrlPtsW);
235
236 /*
237 *Input: n=the size of the Nurbs vector-p-2, p polynomial degree, KnotVector, the CtrlPts weighted with theris weight in the last component ,
238 *      t the point of evaluation
239 *Output: Evaluation of the NURBS curve at point t
240 */
241 std::vector<double> CurvePointRational(int n, int p,const std::vector<double> &KnotVector,const iMatrix<double> &CtrlPtsWeighted, double t);
242
243 /*
244 *Input: binomial coeficients k, i
245 *Output: combinatorial number given by upper number k and lower number i
246 */
247 int Binomial(int k, int i);
248
249 /*
250 *Input: i span index, t eval point, p polinomial degree and KnotVector, weights vector
251 *Output: vector of rational basis funcs
252 */
253 std::vector<double> RationalBasisFuns(int i, double t, int p,const std::vector<double> &KnotVector,const std::vector<double> &WeightVector);
254
255 /*
256 *Input: i span index, t eval point, k derivate degree, p polinomial degree and KnotVector, weights vector
257 *Output: vector of rational basis funcs
258 */
259 iMatrix<double> RatDersBasisFuns(int i, double t, int p, int k, const std::vector<double> &KnotVector,const std::vector<double> &WeightVector);
260
261 /*
262 *Input: i span index, t eval point, k derivate degree, p polinomial degree and KnotVector, iMatrix of the derivatives of the non-rational
263 *      basis funcs and weights vector
264 *Output: vector of rational basis funcs
265 */
266 iMatrix<double> RationalizeDersBasisFuns(int i, double t, int p, int k, const std::vector<double> &KnotVector,const iMatrix<double> &Aders ,
267                                       const std::vector<double> &WeightVector);
268
269 /*
270 *Input: Aders: derivatives of the CtrlPtsWeighted using CurvDersAlg1 evaluated at some t, wders: same as Aders but with the weighted vector,
271 *      maximum degree of the derivate
272 *      -Aders = CurveDerivsAlg1(n, p,KnotVector, WeightedCtrlPts, t, d);
273 *      -wders = CurveDerivsAlg1(n, p,KnotVector, WeightVector, t, d);
274 *Output: iMatrix on which each row is the i-th derivate of the rational curve made of the weighted CtrlPts and the weight vector
275 */
276 iMatrix<double> RatCurveDerivs(const iMatrix<double> &Aders,const iMatrix<double> &wders, int d);
277
278 /*
279 *Input: n_i=the size of the Nurbs vector_i-p-2, p_i polynomial degree, KnotVector_i, the CtrlPtsWeighted ,t_i the point of evaluation
280 *Output: Evaluation of the surface at point (t1,t2)
281 */
282 std::vector<double> SurfacePointRational(int n1, int p1,const std::vector<double> &KnotVector1, int n2, int p2,
283                                         const std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPtsWeighted, double t1, double t2);
284
285 /*
286 *Input: derivatives of the CtrlPtsWeighted using CurvDersAlg1 evaluated at some t, wders: same as Aders but with the weighted vector,
287 *      maximum degree of the derivate
288 *Output: iMatrix on which each row is the i-th derivate of the rational surface made of the weighted CtrlPts and the weight vector
289 */
290

```



```

291 iMatrix<std::vector<double>> RatSurfaceDerivs(const iMatrix<std::vector<double>> &Aders,const iMatrix<double> &wders,int d);
292
293 /*****
294 *
295 * Fundamental Geometric Algorithms
296 *
297 *****/
298
299 /*
300 *Input: KnotVector and element you want to introduce
301 *Output: index i such that KnotVector[i-1] <= e < KnotVector[i]
302 *Note: Applying insertion with this index using KnotVector.insert(KnotVector.begin()+index+1, r, e) will yield the KnotVector
303 *       we want (r is number of insertions)
304 */
305 int findInsertionIndex(const std::vector<double> &KnotVector, double e);
306
307 /*
308 *Input: ordered vector and element you want to check it's number of occurrences
309 *Output: number of times e appears on the vector
310 */
311 int countOccurrences(const std::vector<double> &v, double e);
312
313 /*
314 *Input: A KnotVector, t the point of evaluation, its polynomial degree, and n=the size of the Nurbs vector-p-2
315 *Output: the span index at k, its multiplicity at s
316 */
317 void FindSpanMult(const std::vector<double> &KnotVector, double t, int p, int n,int &k, int &s);
318
319 /*
320 *Input: np number of initial CtrlPts-1, p polynomial degree, the initial KnotVector, the initial CtrlPts Weighted, u value of the inserted knot,
321 *       index such that KnotVector[k]<= u < KnotVector[k+1], s initial multiplicity of u in KnotVector, r number of insertions of u in KnotVector
322 *Output: New CtrlPts Weighted that preserves the initial curve
323 *Note: !! KnotVector will be updated !!
324 */
325 iMatrix<double> CurveKnotsIns(int np, int p, std::vector<double> &KnotVector,const iMatrix<double> &CtrlPtsW, double u, int index, int s, int r);
326
327 /*
328 *Input: n=the size of the Knot vector-p-2, p polynomial degree, KnotVector, CtrlPts Weigthed, u point of evaluation
329 *Output: Evaluation of the curve given by the KnotVector and the ControlPts at the given point of evaluation u
330 */
331 std::vector<double> CurvePntByCornerCut(int n, int p, std::vector<double> &KnotVector, iMatrix<double> &CtrlPtsW, double u);
332
333 /*
334 *Input: npi size of KnotVectori-pi-2, pi degree of KnotVectori, KnotVectori for i=1,2, CtrlPts Weigthed, dir(true= knot inserted in KnotVector1,
335 *       false= knot inserted in KnotVector2, uv knot we want to insert, index such that KnotVectori[k]<= u < KnotVectori[k+1], s initial
336 *       multiplicity of u in KnotVectori, r number of insertions of u in KnotVectori
337 *Output: the KnotVectori will be updated and it will return new CtrlPtsW such that they generate the same surface.
338 */
339 std::vector<iMatrix<double>> SurfaceKnotIns(int np1, int p1, std::vector<double> &KnotVector1, int np2, int p2,
340 *       std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPtsW, bool dir,
341 *       double uv, int index, int r, int s);
342
343 /*
344 *Input: n=Knotvector.size()-2-p, p polynomial degree, KnotVector, CtrlPtsW, Insertions: vector of new elements to insert into KnotVector,
345 *       r=Insertion.size()-1.
346 *Output: Update of the Knotvector with the new inserions, CtrlPtsW that generate the same curve with the new KnotVector.
347 */
348 iMatrix<double> RefineKnotVectCurve(int n, int p, std::vector<double> &KnotVector,const iMatrix<double> &CtrlPtsW,
349 *       const std::vector<double> &Insertions, int r);
350
351 /*
352 *Input: npi size of KnotVectori-i, pi degree of KnotVectori, KnotVectori for i=1,2, CtrlPts Weigthed, dir(true= knot inserted in KnotVector1,
353 *       false= knot inserted in KnotVector2, Insertions: vector of new elements to insert into KnotVector, r=Insertion.size()-1.
354 *Output: the KnotVectori will be updated and it will return new CtrlPtsW such that they generate the same surface.
355 */
356 std::vector<iMatrix<double>> RefineKnotVectSurface(int np1, int p1, std::vector<double> &KnotVector1, int np2, int p2,
357 *       std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPtsW, bool dir,
358 *       const std::vector<double> &Insertions, int r);
359
360 /*****
361 *
362 * Functions for the Assembly of the Matrix
363 *
364 *****/
365
366 /*
367 *Input: n: n² pts of integration
368 *Output: nodes will contain the roots of the legendre polynomial and weights its weights (all in double precision).
369 */
370 void legendre_pol(int n, Eigen::VectorXd &nodes, Eigen::VectorXd &weights);
371
372 /*
373 *Input: ordered vector
374 *Output: the original vector without duplicates
375 */
376 std::vector<double> removeDuplicates(const std::vector<double> &input);
377
378 /*
379 *Input: number of elements of evaluation
380 *Output: vector of nElements+1 uniformly distributed
381 */
382 std::vector<double> createSubIntervals(const double nElements, const double lower_limit, const double upper_limit);
383
384 /*
385 *Input: v1 vector where we remove the elements that appear in v2, v2 KnotVector
386 *Output: v1 without the elements in v2
387 */
388 std::vector<double> removeMatches(const std::vector<double> &v1, const std::vector<double> &v2);
389

```

```

390  /*
391  *Input: n=the size of the Knot vector-p-2, i span index, t eval point, p polynomial degree, number of points of evaluations in the integral,
392  *      KnotVector, upper and lower limits of the integration, weights vector, nodes of the legendre polynomial, CtrlPtsW the control
393  *      points weighted, the function f and the previous evalPoint.
394  *Output: BasisFunsEvals and DerBasisFunsEvals each row will store the evaluation of the value or derivate of the basis fun alongside each
395  *      evaluation point, same for the JacobianEvals, InverseJacobianEvals and funcEvals.
396  */
397  void D1_element_eval(int n, int i, int p, int nEvals, double lower_limit, double upper_limit, const std::vector<double> &KnotVector,
398  const std::vector<double> &Weights, const Eigen::VectorXd &nodes, const iMatrix<double> &CtrlPtsW, std::function<double(double)> f,
399  double &preEvalPoint, double &cumLenght, iMatrix<double> &BasisFunsEvals, iMatrix<double> &DerBasisFunsEvals, std::vector<double> &JacobianEvals,
400  std::vector<double> &InverseJacobianEvals, std::vector<double> &funcEvals, double Lenght=1.0);
401
402  /*
403  *Input: p polynomial degree, upper and lower limits of the integration, number of points of evaluations in the integral,
404  *      evaluations of RationalDerivates, evaluation of the Inverse Jacobian and the legendre weights.
405  *Output: Matrix of all the permutations of evaluations where the elements i,j of the matrix correspond to
406  *      the BasisFuns (init+i)*BasisFuns (init+j) where init=spanIndex()
407  */
408  iMatrix<double> gauss_legendre_cuadrature_integral_bilinearForm(int p, double lower_limit, double upper_limit, int nEvals,
409  const iMatrix<double> &DerBasisFunsEvals, const std::vector<double> &InverseJacobianEvals, const Eigen::VectorXd &weights);
410
411  /*
412  *Input: p polynomial degree, upper and lower limits of the integration, number of points of evaluations in the integral,
413  *      evaluations of RationalBasisFuns, evaluations of the Jacobian, evaluations of the function and the legendre weights.
414  *Output: Vector of evaluations which its components are l(index) where init=spanIndex()
415  */
416  std::vector<double> gauss_legendre_cuadrature_integral_linealForm(int p, double lower_limit, double upper_limit,
417  int nEvals, const iMatrix<double> &BasisFunsEvals, const std::vector<double> &JacobianEvals, const std::vector<double> &funcEvals,
418  const Eigen::VectorXd &weights);
419
420  /*
421  *Input: p polynomial degree, size=LinaerForm.size()-1, SparseMatrix with the system already assambled,
422  *      same with the LinearForm, Modify0(1) true if we have dirichlet condition at the start(end) of the boundary,
423  *      vValueAt0(1) value set for the boundary condition, default is set at 0.
424  *Output: Modification of the sparse matrix and the linear form to keep the system as symetric as possible.
425  */
426  void impose_Dirichlet_Condition(int p, int size, Eigen::SparseMatrix<double> &global, Eigen::VectorXd &LinearForm,
427  bool Modify0, bool Modify1, double ValueAt0=0.0, double ValueAt1=0.0);
428
429  /*
430  *Input: size=LinaerForm.size()-1, LinearForm, Modify0(1) true if we have dirichlet condition at the start(end) of the boundary,
431  *      vValueAt0(1) value set for the boundary condition, default is set at 0.
432  *Output: Modification of the linear form applying the conditions to the borders.
433  */
434  void impose_Newmann_Condition(int size, Eigen::VectorXd &LinearForm, bool Modify0, bool Modify1, double ValueAt0=0.0, double ValueAt1=0.0);
435
436  /*
437  *Input: p polynomial degree, size=LinaerForm.size()-1, SparseMatrix with the system already assambled, same with the LinearForm,
438  *      Modify0(1) true if we have dirichlet condition at the start(end) of the boundary,
439  *      vValueAt0(1) value set for the boundary condition, default is set at 0.
440  *      values alpha and beta come from the expresion alpha*u(x)+beta*u'(x)=ValueAtx.
441  *Output: Modification of the sparse matrix and the linear form to keep the system as symetric as possible.
442  */
443  void impose_Robin_Condition(int p, int size, Eigen::SparseMatrix<double> &global, Eigen::VectorXd &LinearForm, double alpha,
444  double beta, bool Modify0, bool Modify1, double ValueAt0=0.0, double ValueAt1=0.0);
445
446  /*
447  *Input: vector of evaluations of the function, name of hte file where we write the evaluations
448  *Output: writes the evaluation of the function in a txt
449  */
450  void writeNumericFunction(Eigen::VectorXd funEvals, const std::vector<double> &KnotVector, const std::vector<double> &WeightVector,
451  int n, int p, int nEvals, double lowerLimit, double upperLimit, std::string name,
452  std::vector<double> &Intervals, int nElements, double Lenght);
453
454  /*
455  *Input: the function we evaluate, lower and upper limits of evaluation and number of steps of the evaluation, name of the
456  *      file where we write the evaluations, for the evaluation of the curve point, CtrlPtsW, KnotVector, n = KnotVector.size()-p-2,
457  *      p polynomial degree.
458  *Output: writes the evaluation of the function in a txt
459  */
460  void writeAnalyticFunction(std::function<double(double)> f, std::function<double(double)> f_Ders, std::vector<double> &Intervals, double nEvals,
461  const iMatrix<double> &CtrlPtsW, const std::vector<double> &KnotVector, int n, int p, std::string name, double Lenght, double nElements);
462
463  /*
464  *Input: name of the file, p polynomial degree
465  *Output: vector of evaluations on each elements loaded
466  */
467  std::vector<std::vector<double>> leerArchivo(const std::string & name, int p);
468
469  /*
470  *Input: vector of evaluations of the function, name of hte file where we write the evaluations
471  *Output: writes the evaluation of the derivates of the function in a txt
472  */
473  /*
474  void writeFunctionDers(Eigen::VectorXd funEvals, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPtsW,
475  const std::vector<double> &WeightVector, int n, int p, int nEvals, double lowerLimit, double upperLimit, std::string name);
476  */
477  /*
478  *Input: the function we evaluate, lower and upper limits of evaluation and number of steps of the evaluation, name of hte file
479  *      where we write the evaluations, for the evaluation of the curve point, CtrlPtsW, KnotVector, n = KnotVector.size()-p-2, p polynomial degree.
480  *Output: writes the evaluation of the function in a txt
481  */
482  /*
483  void writeFunctionDers(std::function<double(double)> f, double lowerLimit, double upperLimit, double nEvals,
484  const iMatrix<double> &CtrlPtsW, const std::vector<double> &KnotVector,
485  int n, int p, std::string name, double Lenght=1.0);
486  */
487  /*
488  */

```

```

489 /*Input: lowerLimit of integration, upperLimit of integration, number of subintervals for the quadrature, CtrlPtsW, KnotVector, n, p
490 *Output: numerical value of the integral between lowerLimit and upperLimit of the norm of the derivate of the curve
491 */
492 double IntegrateNormDer(double lowerLimit, double upperLimit, int nEvals, const iMatrix<double> &CtrlPtsW,
493     const std::vector<double> &KnotVector, int n, int p);
494
495 /*
496 *Input: e fisical parameter, lowerLimit of integration, upperLimit of integration, number of subintervals for the quadrature,
497     CtrlPtsW, KnotVector, n, p
498 *Output: numerical value parametric space parameter such that the analytic solution of it its equal to the fisical solution at e
499 */
500 inline double Fisical_to_Parametric(double t2, double t1, double lowerLimit, double Lenght,
501     int nEvals, const iMatrix<double> &CtrlPtsW, const std::vector<double> &KnotVector, int n, int p){
502     return IntegrateNormDer(t2,t1,nEvals, CtrlPtsW, KnotVector, n, p)/Lenght;
503 }
504
505 /*
506 *Input: i span index, p polynomial degree, k maximum derivate order, KnotVector, Intervals, nodes and weights
507     of the gauss-legendre cuadrature, vector of storage with spanIndex(memory has to be reserved before execution).
508 *Output: vector that each element is a vector that evaluates the basis functions in each of the corresponding gauss-legendre
509     points of each interval, modification of the vector spanIndex.
510 */
511 std::vector<std::vector<iMatrix<double>>> PreBasis_and_Ders(int p, int k, std::vector<double> KnotVector, std::vector<double> Intervals,
512     Eigen::VectorXd nodes, std::vector<int> &spanIndex);
513
514 /*
515 *Input: span index, nEvals number of points of the gauss-legendre cuadrature alongside the corresponding variable, nodes,
516     vector of the Gauss-Legendre cuadrature, lower_limit, upper_limit of the interval, p polynomial degree, k maximum derivate order, KnotVector,
517     Basis_and_DersY vector of the precomputed Basis and its corresponding derivates, nodes and weights of the gauss-legendre cuadrature
518     and Weights vector of the corresponding weights alongside the corresponding variables.
519 *Output: vector that each element is thee evaluations of the basis functions and its derivates up to k-degree in each of the corresponding
520     gauss-legendre points of the interval.
521 */
522 std::vector<iMatrix<double>> RationalizeDersBasisFunsVec(int span, int nEvals, Eigen::VectorXd nodes, double lower_limit, double upper_limit,
523     int p, int k, int n, std::vector<double> KnotVector, std::vector<iMatrix<double>>> Basis_and_DersY, std::vector<double> Weights);
524
525 /*
526 *Input: vector of Intervals, and vector of the correlated indexes for each interval
527 *Output: unordered map such that given a span index returns its corresponding interval.
528 */
529 std::unordered_map<int, std::vector<std::pair<double, double>>> BuildSpanMap(const std::vector<double> &Intervals, const std::vector<int> &spanIndices);
530
531 /*
532 *Input: vector of Intervals, and vector of the correlated indexes for each interval
533 *Output: unordered map such that given a span index returns its corresponding interval.
534 */
535 iMatrix<iMatrix<double>> RationalizeDersBasisFuns2D(const iMatrix<double> &DersBasisX, const iMatrix<double> &DersBasisY,
536     const iMatrix<double> &activeWeights, unsigned int pX, unsigned int pY);
537
538 /*
539 *Input: SolEvals and Surface evaluations of the same points with the filename and the path
540 *Output: writes the corresponding .txt with the Surface and its evaluations.
541 */
542 void ExportToTxt(const iMatrix<double> &SolEvals, const iMatrix<std::vector<double>> &Surf, const std::string &filename);
543 #endif

```

## Fichero funciones.cpp

```

1  /******
2  *AUTHOR: Carlos Palomera Oliva
3  *DATE: MARCH 2025
4  *****/
5  #include "funciones.hpp"
6
7  /******
8
9  *****/
10 iMatrix<double> ReadDataFile_Matrix(std::string direction){
11     double Nrow, Ncol;
12     std::string line;
13     std::ifstream DataSet;
14
15     DataSet.open(direction);
16     if(DataSet.is_open()){
17
18         if(std::getline(DataSet, line, ' ')){
19             Nrow = stod(line);
20         }
21         if(std::getline(DataSet, line, '\n')){
22             Ncol = stod(line);
23         }
24
25         iMatrix<double> matrix(Nrow, Ncol);
26
27         for(unsigned int i=0; i<Nrow; i++){
28             for(unsigned int j=0; j<Ncol-1; j++){
29                 if(std::getline(DataSet, line, ' ')){
30                     matrix(i, j)=stod(line);
31                 }
32             }
33             if(std::getline(DataSet, line, '\n')){
34                 matrix(i, Ncol-1)=stod(line);
35             }
36         }
37         DataSet.close();
38     }

```

```

39     return matrix;
40 }
41 else{
42     throw std::invalid_argument("Cannot open Data File.");
43     iMatrix<double> m;
44     return m;
45 }
46
47
48 }
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137

```

```

return matrix;
}
else{
    throw std::invalid_argument("Cannot open Data File.");
    iMatrix<double> m;
    return m;
}

}

/***** */
/***** */
std::vector<iMatrix<double>> ReadDataFile_CtrlPts(std::string direction){
    double Ncol,NSplits;
    std::string line;
    std::ifstream DataSet;

    DataSet.open(direction);
    if(DataSet.is_open()){

        if(std::getline(DataSet,line, ' ')){
            Ncol = stod(line);
        }
        if(std::getline(DataSet,line, '\n')){
            NSplits = stod(line);
        }

        std::vector<iMatrix<double>> CtrlPts(NSplits);
        for(unsigned int k=0; k<NSplits;k++){

            CtrlPts[k].Resize(3,Ncol);

            for(unsigned int i=0; i<3; i++){

                for(unsigned int j=0; j<Ncol-1;j++){

                    if(std::getline(DataSet,line, ' ')){

                        CtrlPts[k](i,j)=stod(line);

                    }

                }
                if(std::getline(DataSet,line, '\n')){
                    CtrlPts[k](i,Ncol-1)=stod(line);
                }
            }
            getline(DataSet,line,'\n');

        }

        DataSet.close();
        return CtrlPts;
    }
    else{
        throw std::invalid_argument("Cannot open Data File.");
        std::vector<iMatrix<double>> T;
        return T;
    }
}

/***** */
/***** */
std::vector<double> BernsteinPolynomial(double t, int n){
    std::vector<double> Bnew(n);
    std::vector<double> Bold(n);
    std::fill(Bold.begin(), Bold.end(), 0);
    Bold[0]=1;
    double t1=1-t;
    for(unsigned int j=1;j<n;j++){

        std::fill(Bnew.begin(), Bnew.end(), 0);

        for(unsigned int i=0; i<j;i++){

            Bnew[i]=Bnew[i]+t1*Bold[i];
            Bnew[i+1]=Bnew[i+1]+t*Bold[i];

        }

        Bold=Bnew;

    }
    return Bnew;
}

/***** */
/***** */
std::vector<double> BezierCurve_Point_t(const iMatrix<double> &PtsControl, double t){
    int dim = PtsControl.GetNumRows();
    std::vector<double> Curve(dim);
    int n=PtsControl.GetNumCols();
    std::vector<double> B = BernsteinPolynomial(t,n);
    for(unsigned int j=0; j<dim; j++){
        for(unsigned int i=0; i<n;i++){
            Curve[j]=Curve[j]+B[i]*PtsControl(j,i);

```

```

138     }
139     }
140     return Curve;
141 }
142
143 /***** */
144
145 /***** */
146 iMatrix<double> BezierCurve(const iMatrix<double> &PtosControl, double N_Steps){
147
148     iMatrix<double> Curve(PtosControl.GetNumRows(), N_Steps+1);
149     double Step=1/(N_Steps);
150     for(unsigned int i=0; i<N_Steps+1;i++){
151         Curve.SetCol(i,BezierCurve_Point_t(PtosControl,i*Step));
152     }
153     return Curve;
154 }
155
156 /***** */
157
158 /***** */
159 std::vector<double> BezierSurface_Point_t1_t2(const std::vector<iMatrix<double>> &PtosControl, double t1, double t2){
160     int dim = 3;
161     std::vector<double> Curve(dim);
162
163     int N_Splits = PtosControl.size();
164     int n=PtosControl[0].GetNumCols();
165     std::vector<double> B1 = BernsteinPolynomial(t1,N_Splits);
166     std::vector<double> B2 = BernsteinPolynomial(t2,n);
167
168     for(unsigned int k=0; k<N_Splits;k++){
169         for(unsigned int j=0; j<dim; j++){
170             for(unsigned int i=0; i<n;i++){
171                 Curve[j]=Curve[j]+B1[k]*B2[i]*PtosControl[k](j,i);
172             }
173         }
174     }
175
176     return Curve;
177 }
178
179 /***** */
180
181 /***** */
182 std::vector<iMatrix<double>> BezierSurface(const std::vector<iMatrix<double>> &PtosControl, double N_Steps){
183     std::vector<iMatrix<double>> Surface(N_Steps+1);
184     double Step=1/(N_Steps);
185
186     for(unsigned int k=0; k<N_Steps+1;k++){
187         Surface[k].Resize(3,N_Steps+1);
188         for(unsigned int i=0; i<N_Steps+1;i++){
189             Surface[k].SetCol(i,BezierSurface_Point_t1_t2(PtosControl,i*Step,k*Step));
190         }
191     }
192     return Surface;
193 }
194
195 /***** */
196
197 /***** */
198 void Write_BezierSurface(const std::vector<iMatrix<double>> &Tensor, std::string name){
199     std::ofstream fichero;
200     char coord[3]={'X','Y','Z'};
201     int iterj = Tensor[0].GetNumCols();
202     int iterk = Tensor.size();
203
204     for(unsigned int i=0; i<3;i++){
205         fichero.open(name+coord[i]+".txt");
206         for(unsigned int j=0; j<iterj;j++){
207             for(unsigned int k=0; k<iterk;k++){
208                 fichero<<Tensor[k](i,j)<<" ";
209             }
210             fichero<<std::endl;
211         }
212         fichero.close();
213     }
214 }
215
216 /***** */
217
218 /***** */
219
220 /***** */
221 void Write_CtrlPts(const std::vector<iMatrix<double>> &CtrlPts, std::string name){
222     std::ofstream fichero;
223     char coord[3]={'X','Y','Z'};
224     int iterj = CtrlPts[0].GetNumCols();
225     int iterk = CtrlPts.size();
226
227     for(unsigned int i=0; i<3;i++){
228         fichero.open(name+coord[i]+".txt");
229         for(unsigned int j=0; j<iterj;j++){
230             for(unsigned int k=0; k<iterk;k++){
231                 fichero<<CtrlPts[k](i,j)<<" ";
232             }
233             fichero<<std::endl;
234         }
235         fichero.close();
236     }

```

```

237 }
238
239 }
240
241
242
243 /*****
244 *
245 * Classic funtions from the Nurbs Book
246 *
247 *****/
248
249 int FindSpan(const std::vector<double> &KnotVector, double t, int p, int n){
250
251     if (KnotVector[n+1]==t){
252         return n;
253     }
254
255     else{
256         int low =p;
257         int high=n+1;
258         int mid=(low+high)/2;
259         while(t<KnotVector[mid] || t>=KnotVector[mid+1]){
260             if (t<double(KnotVector[mid])){
261                 //std::cout<<"Condición t<U[mid]"<< "t="<<t<< " mid="<<mid<< " , U[mid]="<< KnotVector[mid]<<std::endl;
262                 high=mid;
263             }
264             else{
265                 //std::cout<<"Condición t>=U[mid]"<< "t="<<t<< " mid="<<mid<< " , U[mid]="<< KnotVector[mid]<<std::endl;
266                 low=mid;
267             }
268             mid=(low+high)/2;
269         }
270         return mid;
271     }
272 }
273
274 /***** */
275
276 /***** */
277 std::vector<double> BasisFuns(int i, double t, int p, const std::vector<double> &KnotVector){
278     std::vector<double> sol(p+1);
279     std::vector<double> left(p+1);
280     std::vector<double> right(p+1);
281     double temp,saved;
282     //std::cout<<"Entramos en basisFuns"<<std::endl;
283     sol[0]=1.0f;
284     for(unsigned int j=1; j<=p; j++){
285         //std::cout<< "j: " << j<< std::endl;
286         left[j]=t-KnotVector[i+1-j];
287         right[j]=KnotVector[i+j]-t;
288         saved = 0.0f;
289         for(unsigned int r=0; r<j; r++){
290             //std::cout<< "r: " << r<< std::endl;
291             temp= sol[r]/(right[r+1]+left[j-r]);
292             sol[r]=saved+right[r+1]*temp;
293             saved=left[j-r]*temp;
294         }
295         sol[j]=saved;
296     }
297     //std::cout<<"Salimos de basisFuns"<<std::endl;
298     return sol;
299 }
300
301 /***** */
302
303 /***** */
304
305
306 iMatrix<double> AllBasisFuns(int i, double t, int p, const std::vector<double> &KnotVector){
307
308
309     iMatrix<double> sol(p+1,p+1);
310     std::vector<double> left(p+1);
311     std::vector<double> right(p+1);
312     sol(0,0)=1.0f;
313     double temp,saved;
314
315     for(unsigned int j=1; j<=p; j++){
316
317         left[j]=t-KnotVector[i+1-j];
318         right[j]=KnotVector[i+j]-t;
319         saved = 0.0f;
320         for(unsigned int r=0; r<j; r++){
321
322             temp= sol(j-1,r)/(right[r+1]+left[j-r]);
323             sol(j,r)=saved+right[r+1]*temp;
324             saved=left[j-r]*temp;
325         }
326         sol(j,j)=saved;
327     }
328
329 }
330
331     return sol;
332 }
333
334 /***** */
335

```

```

336 /*****
337 iMatrix<double> DerBasisFuns(int span, double t, int p, int k, const std::vector<double> &KnotVector){
338     iMatrix<double> ders(k+1,p+1);
339     iMatrix<double> ndu(p+1,p+1);
340     double temp,saved;
341     ndu(0,0)=1.0f;
342     std::vector<double> left(p + 1), right(p + 1);
343
344     for (int j = 1; j <= p; ++j) {
345         left[j] = t - KnotVector[span + 1 - j];
346         right[j] = KnotVector[span + j] - t;
347         double saved = 0.0;
348
349         for (int r = 0; r < j; ++r) {
350             ndu(j,r)= right[r + 1] + left[j - r];
351             double temp= ndu(r,j-1)/ndu(j,r);
352             ndu(r,j)=saved + right[r + 1] * temp ;
353             saved = left[j - r] * temp;
354         }
355         ndu(j,j)=saved;
356     }
357
358     for (int j = 0; j <= p; ++j) {
359         ders(0,j)=ndu(j,p);
360     }
361
362     for (int r = 0; r <= p; ++r) {
363         int s1 = 0, s2 = 1;
364         iMatrix<double> a(2,p+1);
365         a(0,0)=1.0f;
366
367         for (int i = 1; i <= k; ++i) {
368             double d = 0.0;
369             int rk = r - i, pk = p - i;
370             if (r >= i) {
371                 a(s2,0) = a(s1,0)/ndu(pk+1,rk);
372                 d = a(s2,0)*ndu(rk,pk);
373             }
374             int j1 = (rk >= -1) ? 1 : -rk;
375             int j2 = (r - 1 <= pk) ? i - 1 : p - r;
376
377             for (int j = j1; j <= j2; ++j) {
378                 a(s2,j) = (a(s1,j) - a(s1,j-1))/ndu(pk+1,rk+1) ;
379                 d += a(s2,j)*ndu(rk+j,pk);
380             }
381             if (r <= pk) {
382                 a(s2,i) = -a(s1,i-1)/ndu(pk+1,r) ;
383                 d += a(s2,i)*ndu(r,pk);
384             }
385             ders(i,r)=d;
386             std::swap(s1, s2);
387         }
388     }
389
390     }
391
392     // Ajustar multiplicadores
393     int r = p;
394     for (int i = 1; i <= k; ++i) {
395         for (int j = 0; j <= p; ++j) {
396             ders(i,j)= ders(i,j)*r;
397         }
398         r -= (p - i);
399     }
400
401     return ders;
402 }
403
404 /*****
405
406 /*****
407 void Write_BasisFunctions(const iMatrix<double> &BasisFunctions, std::string name){
408     std::ofstream fichero;
409     fichero.open(name+".txt");
410     for(unsigned int j=0; j<BasisFunctions.GetNumRows();j++){
411         for(unsigned int k=0; k<BasisFunctions.GetNumCols();k++){
412             fichero<<BasisFunctions(j,k)<<" ";
413         }
414         fichero<<std::endl;
415     }
416     fichero.close();
417 }
418
419 /*****
420 *
421 * Classic funtions for p spline curves/surfaces
422 *
423 *****/
424
425 std::vector<double> CurvePoint(int n, int p, const std::vector<double> &KnotVector, const iMatrix<double> &CtrlPts, double t){
426     int span =FindSpan(KnotVector, t, p, n);
427     std::vector<double> Funs = BasisFuns(span,t,p,KnotVector);
428
429     int itermax=CtrlPts.GetNumRows();
430     std::vector<double> C(itermax);
431     for(unsigned int i = 0; i<=p;i++){
432         double aux=Funs[i];
433
434         for(unsigned int j=0;j<itermax;j++){

```

```

435         C[j]=C[j]+ aux*(CtrlPts(j,span-p+i));
436     }
437 }
438
439 }
440
441 return C;
442 }
443
444
445 /***** */
446
447 /***** */
448 iMatrix<double> CurveDerivsAlg1(int n, int p,const std::vector<double> &KnotVector,const iMatrix<double> &CtrlPts, double t, int d){
449     int du= std::min(d,p);
450     int span =FindSpan(KnotVector, t, p, n);
451     int itermax=CtrlPts.GetNumRows();
452     //std::cout << "du " << du<< std::endl;
453     iMatrix<double> CK(du+1, itermax); //inicializa a 0 no hace falta dar valores
454     iMatrix<double> nders= DerBasisFuns(span, t, p, du,KnotVector);
455     for(unsigned int k=0; k<=du;k++){
456         for(unsigned int i=0; i<=p;i++){
457             double aux =nders(k,i);
458
459             //CK[k] = CK[k] + nders[k][j]*P[span-p+j]
460             for(unsigned int j=0;j<itermax;j++){
461
462                 CK(k,j)+=aux*(CtrlPts(j,span-p+i));
463
464             }
465         }
466     }
467     return CK;
468 }
469
470 /***** */
471
472 /***** */
473 iMatrix<std::vector<double>> CurveDerivsCpts(int n, int p,const std::vector<double> &KnotVector,const iMatrix<double> &CtrlPts, int d,
474     int r1, int r2){
475     int r= r2-r1;
476     iMatrix<std::vector<double>> sol(d+1,r+1);
477     for(unsigned int i=0;i<=r;i++){
478         sol(0,i)= CtrlPts.GetCol(r1+i);
479     }
480     for(unsigned int k=1;k<=d;k++){
481         int tmp = p-k+1;
482         for(unsigned int i=0;i<=r-k;i++){
483
484             int iter=CtrlPts.GetNumCols(); //dimension del espacio
485             std::vector<double> aux(iter);
486             double denominator=KnotVector[r1+i+p+1]-KnotVector[r1+i+k];
487
488             for(unsigned int j=0; j<iter; j++){
489                 aux[j]=tmp*(sol(k-1,i+1)[j]-sol(k-1,i)[j])/denominator;
490             }
491             sol(k,i)= aux ;
492         }
493     }
494     return sol;
495 }
496
497 /***** */
498
499 /***** */
500 iMatrix<double> CurveDerivsAlg2(int n, int p,const std::vector<double> &KnotVector,const iMatrix<double> &CtrlPts, double t, int d){
501     int du= std::min(d,p);
502     int span =FindSpan(KnotVector, t, p, n);
503     int itermax=CtrlPts.GetNumRows(); //dimension del espacio
504     iMatrix<double> CK(itermax, du); //inicializa a 0 no hace falta dar valores
505     iMatrix<double> nders= AllBasisFuns(span, t, p, KnotVector);
506     iMatrix<std::vector<double>> DervCpts=CurveDerivsCpts(n,p,KnotVector,CtrlPts,du,span-p,span);
507     for(unsigned int k=0; k<=du;k++){
508         for(unsigned int i=0; i<=p-k;i++){
509             double aux =nders(i,p-k);
510             for(unsigned int j=0;j<itermax;j++){
511
512                 CK(k,j)=CK(k,j)+aux*DervCpts(k,i)[j] ;
513
514             }
515         }
516     }
517     return CK;
518 }
519
520 /***** */
521
522 /***** */
523 void Write_Curve(const iMatrix<double> &Curve, std::string filename){
524     std::ofstream fichero;
525     int iterMaxi=Curve.GetNumCols();
526     int iterMaxj=Curve.GetNumRows();
527     fichero.open(filename);
528     for(unsigned int j=0; j<iterMaxj;j++){
529         for(unsigned int i=0; i<iterMaxi;i++){

```



```

534         fichero<<Curve(j,i)<<" ";
535     }
536     fichero<< std::endl;
537 }
538 fichero.close();
539 }
540
541 /***** */
542
543 /***** */
544 std::vector<double> SurfacePoint(int n1, int p1,const std::vector<double> &KnotVector1, int n2, int p2,
545     const std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPts, double t1, double t2){
546     int span1=FindSpan(KnotVector1,t1,p1,n1);
547     int span2=FindSpan(KnotVector2,t2,p2,n2);
548     std::vector<double> Basis1=BasisFuns(span1,t1,p1,KnotVector1);
549     std::vector<double> Basis2=BasisFuns(span2,t2,p2,KnotVector2);
550     int ind1 = span1-p1;
551     std::vector<double> Pt= {0,0,0};
552     for(unsigned int l=0; l<=p2; l++){
553
554         std::vector<double> temp= {0,0,0};
555
556         int ind2= span2 - p2+1;
557         for(unsigned int k=0;k<=p1;k++){
558             double auxBasis1= Basis1[k];
559             //std::vector<double> auxCtrlPt = CtrlPts(ind1+k,ind2);
560             for(unsigned int i=0; i<3;i++){
561                 //std::cout<<"i="<<i<<" ,k="<<k<<" ,l="<<l<<std::endl;
562                 temp[i]=temp[i]+auxBasis1*CtrlPts[ind1+k] (i,ind2);
563             }
564         }
565
566         double auxBasis2=Basis2[l];
567         for(unsigned int i=0; i<3;i++){
568             Pt[i]=Pt[i]+auxBasis2*temp[i];
569         }
570     }
571     return Pt;
572 }
573
574 /***** */
575
576 /***** */
577 iMatrix<std::vector<double>> SurfaceDerivsAlg1(int n1, int p1,const std::vector<double> &KnotVector1, int n2, int p2,
578     const std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPts, double t1, double t2, int d){
579     unsigned int du=std::min(d,p1);
580     unsigned int dv=std::min(d,p2);
581
582     unsigned int dim = CtrlPts[0].GetNumRows();
583     //std::cout<< "dim: " << dim << std::endl;
584     iMatrix<std::vector<double>> Sol(du+1,dv+1);
585     const std::vector<double> zero(dim);
586     for(unsigned int k=0; k<=du; k++){
587         for(unsigned int j=0; j<=dv;j++){
588             Sol(k,j)= zero ;
589         }
590     }
591     //std::cout<< "2" << std::endl;
592     int span1=FindSpan(KnotVector1,t1,p1,n1);
593     int span2=FindSpan(KnotVector2,t2,p2,n2);
594
595     iMatrix<double> Ders1 = DerBasisFuns(span1, t1, p1, du, KnotVector1);
596     iMatrix<double> Ders2 = DerBasisFuns(span2, t2, p2, dv, KnotVector2);
597     iMatrix<double> temp(dim,p2+1);
598     //std::cout<< "3" << std::endl;
599     for(unsigned int k=0; k<=du; k++){
600         //std::cout<< "4" << std::endl;
601
602         for(unsigned int s=0;s<=p2;s++){
603             //std::cout << "s: " << s << std::endl;
604             temp.SetCol(s,zero);
605             for(unsigned int r=0; r<=p1; r++){
606                 // std::cout<< "6" << std::endl;
607                 //temp[s] = temp[s] + Nu[k] [r]*P[uspan-p+r] [vspan-q+s];
608                 double aux= Ders1(k,r);
609                 for(unsigned int j=0; j<dim;j++){
610                     // std::cout<< "7" << std::endl;
611                     //std::cout<< CtrlPts[span1-p1+r] (j,span2 -p2+s) << "|";
612                     temp(j,s) = temp(j,s)+aux*CtrlPts[span1-p1+r] (j,span2 -p2+s);
613                 }
614             }
615
616             //std::cout << "-----" << std::endl;
617         }
618     }
619     //std::cout << "8" << std::endl;
620     int dd=std::min(d-k,dv);
621     for(unsigned int l=0; l<= dd; l++){
622         Sol(k,l)=zero;
623         for(unsigned int s=0;s<=p2;s++){
624             double aux = Ders2(l,s);
625             std::vector<double> auxSol= Sol(k,l);
626             std::vector<double> aux2(dim);
627             for(unsigned int j=0; j<dim; j++){
628                 aux2[j]=auxSol[j]+aux*temp(j,s);
629             }
630             Sol(k,l)= aux2;
631         }
632     }

```

```

633     }
634 }
635
636     return Sol;
637 }
638
639 /*****
640 *
641 * Classic and support funtions for NURBS curves/surfaces
642 *
643 *****/
644 iMatrix<double> WeightCtrlPts(const iMatrix<double> &CtrlPts,const std::vector<double> &Weights){
645     int dim= CtrlPts.GetNumRows()+1;
646     int size = CtrlPts.GetNumCols();
647     iMatrix<double> CtrlPtsWeighted(dim,size);
648     for(unsigned int i=0; i<size;i++){
649         for(unsigned int j=0; j<dim-1; j++){
650             CtrlPtsWeighted(j,i)=CtrlPts(j,i)*Weights[i];
651         }
652         CtrlPtsWeighted(dim-1,i)=Weights[i];
653     }
654
655     return CtrlPtsWeighted;
656 }
657
658 /***** */
659 /***** */
660
661 iMatrix<double> UnWeightCtrlPts(const iMatrix<double> &CtrlPtsW){
662     int NumPts = CtrlPtsW.GetNumCols();
663     int dim = CtrlPtsW.GetNumRows()-1;
664     iMatrix<double> CtrlPts(dim,NumPts);
665     for(unsigned int j=0; j< NumPts;j++){
666         for(unsigned int i=0; i<dim; i++){
667             if(CtrlPtsW(dim,j)==0){
668                 //nothing, the declararion automatically sets it to 0
669             }
670             else{
671                 CtrlPts(i,j)=CtrlPtsW(i,j)/CtrlPtsW(dim,j);
672             }
673         }
674     }
675
676     return CtrlPts;
677 }
678
679 /***** */
680 /***** */
681
682 std::vector<iMatrix<double>> WeightCtrlPtsSurf(const std::vector<iMatrix<double>> &CtrlPts,const iMatrix<double> &Weights){
683     int dim = CtrlPts.size();
684     std::vector<iMatrix<double>> WeightedCtrlPts(dim);
685     for(unsigned int i = 0; i< dim; i++){
686         WeightedCtrlPts[i]=WeightCtrlPts(CtrlPts[i], Weights.GetRow(i));
687     }
688     return WeightedCtrlPts;
689 }
690
691 /***** */
692 /***** */
693
694 std::vector<iMatrix<double>> UnWeightCtrlPtsSurf(const std::vector<iMatrix<double>> &CtrlPtsW){
695     int NumPts2= CtrlPtsW.size();
696     int NumPts1 = CtrlPtsW[0].GetNumCols();
697     int dim = CtrlPtsW[0].GetNumRows()-1;
698
699     std::vector<iMatrix<double>> CtrlPts(NumPts2);
700
701     for(unsigned int k=0; k<NumPts2; k++){
702         CtrlPts[k]=iMatrix<double>(dim, NumPts1);
703         //iMatrix<double> AuxMat (dim,NumPts1);
704
705         for(unsigned int j=0; j< NumPts1;j++){
706
707             for(unsigned int i=0; i<dim; i++){
708
709                 if(CtrlPtsW[k](dim,j)==0){
710
711                     //nothing, the declararion automatically sets it to 0
712                 }
713                 else{
714
715                     CtrlPts[k](i,j)=CtrlPtsW[k](i,j)/CtrlPtsW[k](dim,j);
716                 }
717             }
718         }
719
720     }
721
722     return CtrlPts;
723 }
724
725 /***** */
726 /***** */
727 /***** */
728
729 std::vector<double> CurvePointRational(int n, int p,const std::vector<double> &KnotVector,
730     const iMatrix<double> &CtrlPtsWeighted, double t){

```

```

732 int span = FindSpan(KnotVector, t, p, n);
733 std::vector<double> Funs = BasisFuns(span, t, p, KnotVector);
734 int dim = CtrlPtsWeighted.GetNumRows();
735 std::vector<double> Cw(dim, 0.0f);
736 for(unsigned int j=0; j<=p; j++){
737     //Cw=Cw + Funs[j]*CtrlPtsWeighted[span-p+j]
738     for(unsigned int i=0; i<dim; i++){
739         Cw[i]=Cw[i]+Funs[j]*CtrlPtsWeighted(i,span-p+j);
740     }
741 }
742 std::vector<double> C(dim-1);
743 double w = Cw[dim-1];
744
745 for(unsigned int i=0; i<dim-1; i++){
746     C[i]=Cw[i]/w;
747 }
748 return C;
749 }
750
751 /***** */
752
753 /***** */
754 int Binomial(int k, int i){
755     if (i < 0 || i > k) return 0;
756     if (i == 0 || i == k) return 1;
757     if (i > k - i) i = k - i; // simetría
758
759     long long res = 1;
760     for (int j = 1; j <= i; j++) {
761         res *= k - (i - j);
762         res /= j;
763     }
764     return (int)res;
765 }
766
767 /***** */
768
769 /***** */
770 std::vector<double> RationalBasisFuns(int i, double t, int p, const std::vector<double> &KnotVector,
771 const std::vector<double> &WeightVector){
772     std::vector<double> Funs = BasisFuns(i, t, p, KnotVector);
773     int itermax=Funs.size();
774     double coef=0.0f;
775     //std::cout<<itermax<<std::endl;
776     for(int j=0; j<itermax; j++){
777         //std::cout<<"a"<<std::endl;
778         coef=coef+Funs[j]*WeightVector[i-p+j];
779     }
780     for(int j=0; j<itermax; j++){
781         //std::cout<<"b"<<std::endl;
782         Funs[j]=Funs[j]*WeightVector[i-p+j]/coef;
783     }
784
785     return Funs;
786 }
787
788 /***** */
789
790 /***** */
791 iMatrix<double> RatDersBasisFuns(int i, double t, int p, int k, const std::vector<double> &KnotVector,
792 const std::vector<double> &WeightVector){
793     iMatrix<double> R_ders(k+1, p+1);
794     iMatrix<double> ders = DerBasisFuns(i, t, p, k, KnotVector);
795
796     std::vector<double> Wk(k+1, 0.0);
797     for (int j = 0; j <= k; ++j){
798         for (int iter = 0; iter <= p; iter++){
799             Wk[j] += WeightVector[i-p+iter] * ders(j, iter);
800             //std::cout << "Weight1: " << WeightVector[i-p+iter] << " ";
801         }
802     }
803     //std::cout << std::endl;
804
805     for (int a = 0; a < p+1; ++a) {
806         R_ders(0, a) = ders(0, a) * WeightVector[i-p+a]/Wk[0]; // (D(0, a) * weights[a]) / Wk[0];
807         for (int j = 1; j <= k; ++j) {
808             double sum = 0.0;
809             for (int m = 1; m <= j; ++m){
810                 sum += Binomial(j, m) * Wk[m] * R_ders(j - m, a);
811             }
812             double num = ders(j, a) * WeightVector[i-p+a] - sum;
813             R_ders(j, a) = num / Wk[0];
814         }
815     }
816
817     return R_ders;
818 }
819
820 }
821
822 /***** */
823
824 /***** */
825 iMatrix<double> RationalizeDersBasisFuns(int i, double t, int p, int k, const std::vector<double> &KnotVector,
826 const iMatrix<double> &ders, const std::vector<double> &WeightVector){
827     iMatrix<double> R_ders(k+1, p+1);
828
829     std::vector<double> Wk(k+1, 0.0);

```

```

831     for (int j = 0; j <= k; ++j){
832         for (int iter = 0; iter <= p; iter++){
833             Wk[j] += WeightVector[i-p+iter] * ders(j,iter);
834         }
835     }
836
837     for (int a = 0; a < p+1; ++a) {
838         R_ders(0, a) = ders(0,a)*WeightVector[i-p+a]/Wk[0]; // (D(0, a) * weights[a]) / Wk[0];
839         for (int j = 1; j <= k; ++j) {
840             double sum = 0.0;
841             for (int m = 1; m <= j; ++m){
842                 sum += Binomial(j, m) * Wk[m] * R_ders(j - m, a);
843             }
844             double num = ders(j, a) * WeightVector[i-p+a] - sum;
845             R_ders(j, a) = num / Wk[0];
846         }
847     }
848 }
849
850 return R_ders;
851 }
852
853
854 /***** */
855
856 /***** */
857 iMatrix<double> RatCurveDerivs(const iMatrix<double> &Aders,const iMatrix<double> &wders, int d){
858     int NumCols=Aders.GetNumCols();
859     iMatrix<double> CK(d+1,NumCols);
860     //CK[0]=Aders[0]/wders[0]
861     //std::vector<double> init(NumCols);
862     for(unsigned int k=0; k<NumCols; k++){
863         CK(0,k)=Aders(0,k)/wders(0,0);
864     }
865
866
867     for(unsigned int k =1; k<=d; k++){
868         std::vector<double> v = Aders.GetRow(k);
869         for(int i=1; i<=k; i++){
870             int Bin=Binomial(k,i);
871             //std::cout << "k: " << k <<std::endl;
872             //std::cout << "i: " << i <<std::endl;
873             //std::cout << "binomial: " << Bin <<std::endl;
874             //v = v - Binomial(k, i) * wders[i] * CK[k - i];
875             for(unsigned int j=0; j<NumCols; j++){
876                 v[j] -= Bin*wders(i,0)*CK(k-i,j);
877             }
878             //CK[k] = v / wders[0];
879             for(unsigned int j=0; j<NumCols; j++){
880                 v[j] /= wders(0,0);
881             }
882             CK.SetRow(k,v);
883         }
884     }
885
886     return CK;
887 }
888
889 /***** */
890
891 /***** */
892 std::vector<double> SurfacePointRational(int n1, int p1,const std::vector<double> &KnotVector1, int n2, int p2,
893     const std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPtsWeighted, double t1, double t2){
894     int span1=FindSpan(KnotVector1,t1,p1,n1);
895     int span2=FindSpan(KnotVector2,t2,p2,n2);
896     std::vector<double> Basis1=BasisFuns(span1,t1,p1,KnotVector1);
897     std::vector<double> Basis2=BasisFuns(span2,t2,p2,KnotVector2);
898     int ind1 = span1-p1;
899     int dim = CtrlPtsWeighted[0].GetNumRows();
900     std::vector<double> Pt(dim);
901     iMatrix<double> auxtemp(p2+1,dim);
902     for(unsigned int l=0; l<=p2; l++){
903         std::vector<double> temp(dim);
904
905         int ind2= span2 - p2+1;
906         for(unsigned int k=0;k<=p1;k++){
907             double auxBasis1= Basis1[k];
908             //std::vector<double> auxCtrlPt = CtrlPts(ind1+k,ind2);
909             for(unsigned int i=0; i<dim;i++){
910                 //std::cout<<"i="<<i<<"k="<<k<<"l="<<l<<std::endl;
911                 temp[i] +=auxBasis1*CtrlPtsWeighted[ind1+k](i,ind2);
912             }
913             auxtemp.SetRow(l, temp);
914         }
915     }
916
917     for(unsigned int l=0; l<=p2;l++){
918         double auxBasis2=Basis2[l];
919         for(unsigned int i=0; i<dim;i++){
920             Pt[i] =Pt[i]+auxBasis2*auxtemp(l,i);
921         }
922     }
923
924     std::vector<double> C(dim-1);
925     double w = Pt[dim-1];

```

```

930     for(unsigned int i=0; i<dim-1; i++){
931         C[i]=Pt[i]/v;
932     }
933     return C;
934 }
935
936
937 /*****
938
939 /*****
940 iMatrix<std::vector<double>> RatSurfaceDerivs(const iMatrix<std::vector<double>> &Aders,const iMatrix<double> &wders,int d){
941
942     int dim=Aders(0,0).size();
943     iMatrix<std::vector<double>> SKL(d+1,d+1);
944
945     for(unsigned int k=0; k<=d; k++){
946         for(unsigned int l=0;l<=d-k;l++){
947             //v = Aders [k] [l] ;
948             std::vector<double> v = Aders(k,l);
949
950             for(unsigned int j=1; j<=l;j++){
951                 double binaux=Binomial(l,j);
952                 double Wdersaux=wders(0,j);
953
954
955
956                 for(unsigned int iter=0; iter < dim; iter++){
957
958                     //v = v - Bin[l] [j]*wders[0] [j]*SKL[k] [l-j];
959                     v[iter]=v[iter]-binaux*Wdersaux*SKL(k,l-j)[iter];
960
961                 }
962
963             }
964
965
966             for(unsigned int i=1; i<=k; i++){
967                 double binaux=Binomial(k,i);
968                 double Wdersaux=wders(i,0);
969
970                 for(unsigned int iter=0; iter < dim; iter++){
971                     //v = v - Bin[k] [i]*wders[i] [0]*SKL[k-i] [l];
972                     v[iter]=v[iter]-binaux*Wdersaux*SKL(k-i,l)[iter];
973                 }
974
975                 //v2 = 0.0;
976                 std::vector<double> v2(Aders(0,0).size());
977                 for(unsigned int j=1; j<=l;j++){
978                     binaux=Binomial(l,j);
979                     Wdersaux=wders(i,j);
980
981                     for(unsigned int iter=0; iter < dim; iter++){
982                         //v2 = v2 + Bin[l] [j]*wders[i] [j]*SKL[k-i] [l-j];
983                         v2[iter]=v2[iter]+binaux*Wdersaux*SKL(k-i,l-j)[iter];
984                     }
985
986                     binaux=Binomial(k,i);
987                     for(unsigned int iter=0; iter < dim; iter++){
988                         //v = v - Bin[k] [i]*v2;
989                         v[iter]=v[iter]-binaux*v2[iter];
990                     }
991
992                 }
993
994                 //SKL[k] [l] = v/wders[0] [0];
995                 std::vector<double> auxvec(dim);
996                 for(unsigned int iter=0; iter < dim; iter++){
997                     auxvec[iter]=v[iter]/wders(0,0);
998                 }
999
1000                 SKL(k,l)=auxvec;
1001             }
1002         }
1003     }
1004
1005     return SKL;
1006 }
1007
1008 /*****
1009 *
1010 * Fundamental Geometric Algorithms and support functions
1011 *
1012 /*****
1013 int findInsertionIndex(const std::vector<double>& v, double e){
1014     auto it = std::upper_bound(v.begin(), v.end(), e);
1015     return it - v.begin()-1;
1016 }
1017
1018 /*****
1019
1020 /*****
1021
1022 int countOccurrences(const std::vector<double>& v, double e) {
1023     double eps = 1e-6f;
1024     int left = 0;
1025     int right = v.size() - 1;
1026     int index = -1;
1027
1028     //binary search

```

```

1029 while (left <= right) {
1030     int mid = left + (right - left) / 2;
1031     if (std::fabs(v[mid] - e) < eps) {
1032         index = mid;
1033         right = mid - 1;
1034     } else if (v[mid] < e) {
1035         left = mid + 1;
1036     } else {
1037         right = mid - 1;
1038     }
1039 }
1040
1041 if (index == -1) return 0; // Not found case
1042
1043 //linear count
1044 int count = 0;
1045 while (index + count < v.size() && std::fabs(v[index + count] - e) < eps) {
1046     count++;
1047 }
1048
1049 return count;
1050 }
1051
1052 /***** */
1053
1054 /***** */
1055 void FindSpanMult(const std::vector<double> &KnotVector, double t, int p, int n, int &k, int &s){
1056     double eps = 1e-6f;
1057     k=FindSpan(KnotVector,t,p,n);
1058     s=0;
1059     for (int i = k; i >= 0 && std::fabs(KnotVector[i] - t) < eps; --i) s++;
1060     for (int i = k + 1; i <= n + p + 1 && std::fabs(KnotVector[i] - t) < eps; ++i) s++;
1061 }
1062
1063 /***** */
1064
1065 /***** */
1066 iMatrix<double> CurveKnotsIns(int np, int p, std::vector<double> &KnotVector, const iMatrix<double> &CtrlPtsW,
1067     double u, int index, int s, int r){
1068     //int mp = np*p+1;
1069     int dim=CtrlPtsW.GetNumRows();
1070     int L;
1071     double alpha;
1072
1073     //New CtrlPts
1074     iMatrix<double> NewCtrlPtsW(dim, CtrlPtsW.GetNumCols()+r);
1075
1076     for(unsigned int i=0; i<=index-p; i++){
1077         NewCtrlPtsW.SetCol(i, CtrlPtsW.GetCol(i));
1078     }
1079
1080     for(unsigned int i=index-s; i<=np; i++){
1081         NewCtrlPtsW.SetCol(i+r, CtrlPtsW.GetCol(i));
1082     }
1083
1084     iMatrix<double> AuxCtrlPts(dim,p+1);
1085
1086     for(unsigned int i=0; i<=p-s; i++){
1087         AuxCtrlPts.SetCol(i, CtrlPtsW.GetCol(index-p+i));
1088     }
1089
1090     for(unsigned int j=1; j<=r; j++){
1091
1092         L=index-p+j;
1093
1094         for(int i=0; i<=p-j-s; i++){
1095             alpha= (u-KnotVector[L+i])/(KnotVector[i+index+1]-KnotVector[L+i]);
1096
1097             for(unsigned int iter=0; iter<dim; iter++){
1098
1099                 AuxCtrlPts(iter,i)=alpha*AuxCtrlPts(iter,i+1)+(1.0f-alpha)*AuxCtrlPts(iter,i);
1100             }
1101         }
1102
1103         NewCtrlPtsW.SetCol(L,AuxCtrlPts.GetCol(0));
1104
1105         NewCtrlPtsW.SetCol(index+r-j-s,AuxCtrlPts.GetCol(p-j-s));
1106
1107     }
1108     for(unsigned int i=L+1; i<=index-s; i++){
1109
1110         NewCtrlPtsW.SetCol(i,AuxCtrlPts.GetCol(i-L));
1111     }
1112
1113     //Update KnotVector
1114     KnotVector.insert(KnotVector.begin()+index+1, r, u);
1115
1116     return NewCtrlPtsW;
1117 }
1118
1119 /***** */
1120
1121 /***** */
1122 std::vector<double> CurvePntByCornerCut(int n, int p, std::vector<double> &KnotVector, iMatrix<double> &CtrlPtsW, double u){
1123     int dim = CtrlPtsW.GetNumRows()-1;
1124     std::vector<double> aux(dim+1);
1125
1126     if (u==KnotVector[0]){
1127

```

```

1128     std::vector<double> aux(dim);
1129     for(unsigned int i=0; i< dim; i++){
1130         aux[i]=CtrlPtsW(i,0)/CtrlPtsW(dim,0);
1131     }
1132     return aux;
1133 }
1134 if(u==KnotVector[np+1]){
1135     std::vector<double> aux(dim);
1136     for(unsigned int i=0; i< dim; i++){
1137         aux[i]=CtrlPtsW(i,n)/CtrlPtsW(dim,n);
1138     }
1139     return aux;
1140 }
1141 int k,s;
1142 FindSpanMult(KnotVector, u, p, n, k, s);
1143 int r = p-s;
1144 //std::cout<< "r: " << r << std::endl;
1145 iMatrix<double> AuxCtrlPts(dim+1,r+1);
1146 for(unsigned int i=0; i<=r; i++){
1147     AuxCtrlPts.SetCol(i,CtrlPtsW.GetCol(k-p+i));
1148 }
1149
1150
1151
1152 for(unsigned int j=1; j<=r; j++){
1153     for(unsigned int i=0; i<=r-j; i++){
1154         double alpha= (u-KnotVector[k-p+j+i])/(KnotVector[i+k+1]-KnotVector[k-p+j+i]);
1155
1156         for(unsigned int iter=0; iter< dim+1; iter++){
1157             aux[iter]=alpha*AuxCtrlPts(iter,i+1)+(1.0f-alpha)*AuxCtrlPts(iter,i);
1158         }
1159         AuxCtrlPts.SetCol(i,aux);
1160         //AuxCtrlPts.PrintMatrix();
1161     }
1162 }
1163
1164 std::vector<double> sol(dim);
1165
1166 for(unsigned int i=0; i<dim; i++){
1167     sol[i]=AuxCtrlPts(i,0)/AuxCtrlPts(dim,0);
1168 }
1169 return sol;
1170 }
1171
1172
1173 /***** */
1174 /***** */
1175
1176 std::vector<iMatrix<double>> SurfaceKnotIns(int np1, int p1, std::vector<double> &KnotVector1, int np2, int p2,
1177 std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPtsW, bool dir,
1178 double uv, int index, int r, int s){
1179     if(dir==true){//KnotVector1 will be updated
1180         int dim=CtrlPtsW[0].GetNumRows();
1181         iMatrix<double> Aux(dim,p1-s+1);
1182         int numCtrlPts1=CtrlPtsW.size()+1;
1183         int L;
1184         iMatrix<double> alpha(p1-s+1,r+1);
1185         std::vector<iMatrix<double>> NewCtrlPts(numCtrlPts1);
1186         for(unsigned int i=0; i<numCtrlPts1; i++){
1187             NewCtrlPts[i]=iMatrix<double>(dim, CtrlPtsW[0].GetNumCols());
1188         }
1189
1190         for(unsigned int j=1; j<=p1-j-s;j++){
1191             L=index-p1+j;
1192             for(unsigned int i=0;i<=p1-j-s;i++){
1193                 alpha(i,j)= (uv-KnotVector1[L+i])/(KnotVector1[i+index+1]-KnotVector1[L+i]);
1194             }
1195         }
1196
1197         for(unsigned int row=0; row<= np2; row++){
1198             for(unsigned int i=0; i<=index-p1; i++){
1199                 NewCtrlPts[i].SetCol(row,CtrlPtsW[i].GetCol(row));
1200             }
1201             for(unsigned int i=index-s; i<=np1; i++){
1202                 NewCtrlPts[i+r].SetCol(row,CtrlPtsW[i].GetCol(row));
1203             }
1204             for(unsigned int i=0; i<=p1-s;i++){
1205                 Aux.SetCol(i,CtrlPtsW[index-p1+i].GetCol(row));
1206             }
1207
1208             for(unsigned int j=1; j<=r; j++){
1209                 L=index-p1+j;
1210                 for(unsigned int i=0; i<= p1-j-s; i++){
1211                     std::vector<double> aux(dim);
1212                     for(unsigned int iter=0; iter<dim; iter++){
1213                         aux[iter]= alpha(i,j)*Aux(iter,i+1)+(1.0f-alpha(i,j))*Aux(iter,i);
1214                     }
1215                     Aux.SetCol(i,aux);
1216                 }
1217                 NewCtrlPts[L].SetCol(row,Aux.GetCol(0));
1218                 NewCtrlPts[index+r-j-s].SetCol(row,Aux.GetCol(p1-j-s));
1219             }
1220             for(unsigned int i=L+1;i<index-s;i++){
1221                 NewCtrlPts[i].SetCol(row,Aux.GetCol(i-L));
1222             }
1223         }
1224     }
1225     KnotVector1.insert(KnotVector1.begin()+index+1, r, uv);
1226     return NewCtrlPts;

```

```

1227     }
1228     else{//KnotVector2 will be updated
1229         int dim = CtrlPtsW[0].GetNumRows();
1230         iMatrix<double> Aux(dim, p2 - s + 1);
1231         int numCtrlPts2 = CtrlPtsW[0].GetNumCols() + 1;
1232         int L;
1233         iMatrix<double> alpha(p2 - s + 1, r + 1);
1234         std::vector<iMatrix<double>> NewCtrlPts(CtrlPtsW.size());
1235
1236         for (unsigned int i = 0; i < CtrlPtsW.size(); i++) {
1237             NewCtrlPts[i] = iMatrix<double>(dim, numCtrlPts2);
1238         }
1239
1240         for (unsigned int j = 1; j <= p2 - j - s; j++) {
1241             L = index - p2 + j;
1242             for (unsigned int i = 0; i <= p2 - j - s; i++) {
1243                 alpha(i, j) = (uv - KnotVector2[L + i]) / (KnotVector2[i + index + 1] - KnotVector2[L + i]);
1244             }
1245         }
1246
1247         for (unsigned int col = 0; col <= np1; col++) {
1248             for (unsigned int i = 0; i <= index - p2; i++) {
1249                 NewCtrlPts[col].SetCol(i, CtrlPtsW[col].GetCol(i));
1250             }
1251             for (unsigned int i = index - s; i <= np2; i++) {
1252                 NewCtrlPts[col].SetCol(i + r, CtrlPtsW[col].GetCol(i));
1253             }
1254
1255             for (unsigned int i = 0; i <= p2 - s; i++) {
1256                 Aux.SetCol(i, CtrlPtsW[col].GetCol(index - p2 + i));
1257             }
1258
1259             for (unsigned int j = 1; j <= r; j++) {
1260                 L = index - p2 + j;
1261                 for (unsigned int i = 0; i <= p2 - j - s; i++) {
1262                     std::vector<double> aux(dim);
1263                     for (unsigned int d = 0; d < dim; d++) {
1264                         aux[d] = alpha(i, j) * Aux(d, i + 1) +
1265                             (1.0f - alpha(i, j)) * Aux(d, i);
1266                     }
1267                     Aux.SetCol(i, aux);
1268                 }
1269
1270                 NewCtrlPts[col].SetCol(L, Aux.GetCol(0));
1271                 NewCtrlPts[col].SetCol(index + r - j - s, Aux.GetCol(p2 - j - s));
1272             }
1273
1274             for (unsigned int i = L + 1; i < index - s; i++) {
1275                 NewCtrlPts[col].SetCol(i, Aux.GetCol(i - L));
1276             }
1277         }
1278
1279         KnotVector2.insert(KnotVector2.begin() + index + 1, r, uv);
1280
1281         return NewCtrlPts;
1282     }
1283 }
1284
1285 }
1286
1287
1288 /***** */
1289
1290 /***** */
1291 iMatrix<double> RefineKnotVectCurve(int n, int p, std::vector<double> &KnotVector,
1292 const iMatrix<double> &CtrlPtsW, const std::vector<double> &Insertions, int r){
1293     int m=n+p+1;
1294     int dim=CtrlPtsW.GetNumRows();
1295     std::vector<double> NewKnotVector(m+r+2);
1296     //int numCtrlPts=CtrlPtsW.GetNumCols();
1297     iMatrix<double> NewCtrlPts(dim,n+r+2);
1298     int a=FindSpan(KnotVector,Insertions[0],p,n);
1299     int b=FindSpan(KnotVector,Insertions[r],p,n)+1;
1300
1301     for(unsigned int j=0;j<=a-p;j++){
1302         NewCtrlPts.SetCol(j,CtrlPtsW.GetCol(j));
1303     }
1304     for(unsigned int j=b-1;j<=n;j++){
1305         NewCtrlPts.SetCol(j+r+1,CtrlPtsW.GetCol(j));
1306     }
1307     for(unsigned int j=0; j<=a;j++){
1308         NewKnotVector[j]=KnotVector[j];
1309     }
1310     for(unsigned int j=b+p; j<=m;j++){
1311         NewKnotVector[j+r+1]=KnotVector[j];
1312     }
1313
1314
1315     int i=b+p-1;
1316     int k=b+p+r;
1317     for(int j=r; j>=0; j--){
1318         while(Insertions[j]<=KnotVector[i]&&i>a){
1319             NewCtrlPts.SetCol(k-p-1, CtrlPtsW.GetCol(i-p-1));
1320             NewKnotVector[k]=KnotVector[i];
1321             k--;
1322             i--;
1323         }
1324         NewCtrlPts.SetCol(k-p-1,NewCtrlPts.GetCol(k-p));
1325         for(unsigned int l=1; l<=p;l++){

```



```

1326     int ind = k-p+1;
1327     double alpha=NewKnotVector[k+1]-Insertions[j];
1328     if (alpha==0.0){
1329         NewCtrlPts.SetCol(ind-1,NewCtrlPts.GetCol(ind));
1330     }
1331     else{
1332         alpha=alpha/(NewKnotVector[k+1]-KnotVector[i-p+1]);
1333         std::vector<double> aux(dim);
1334         for(unsigned int iter=0; iter<dim; iter++){
1335             aux[iter]=alpha*NewCtrlPts(iter,ind-1)+(1.0f-alpha)*NewCtrlPts(iter,ind);
1336         }
1337         NewCtrlPts.SetCol(ind-1,aux);
1338     }
1339 }
1340 NewKnotVector[k]=Insertions[j];
1341 k--;
1342 }
1343 KnotVector=NewKnotVector;
1344 return NewCtrlPts;
1345 }
1346
1347
1348 /***** */
1349
1350 /***** */
1351 std::vector<iMatrix<double>> RefineKnotVectSurface(int np1, int p1, std::vector<double> &KnotVector1,
1352 int np2, int p2, std::vector<double> &KnotVector2,const std::vector<iMatrix<double>> &CtrlPtsW, bool dir,
1353 const std::vector<double> Insertions, int r){
1354     int dim=CtrlPtsW[0].GetNumRows();
1355     if (dir==true){
1356         //int L;
1357         int numCtrlPts1=CtrlPtsW.size()+r+1;
1358         int numCtrlPts2=CtrlPtsW[0].GetNumCols();
1359         int a=FindSpan(KnotVector1,Insertions[0],p1,np1);
1360         int b=FindSpan(KnotVector1,Insertions[r],p1,np1)+1;
1361         int m=np1+p1+1;
1362         std::vector<double> NewKnotVector(m+r+2);
1363         std::vector<iMatrix<double>> NewCtrlPts(numCtrlPts1);
1364         for(unsigned int i=0; i<numCtrlPts1; i++){
1365             NewCtrlPts[i]=iMatrix<double>(dim, numCtrlPts2);
1366         }
1367
1368         for(unsigned int row=0; row<=m;row++){
1369             for(unsigned int k=0; k<=a-p1;k++){
1370                 NewCtrlPts[k].SetCol(row,CtrlPtsW[k].GetCol(row));
1371             }
1372             for(unsigned int k=b-1; k<=np1; k++){
1373                 NewCtrlPts[k+r+1].SetCol(row,CtrlPtsW[k].GetCol(row));
1374             }
1375         }
1376
1377         for(unsigned int j=0; j<=a;j++){
1378             NewKnotVector[j]=KnotVector1[j];
1379         }
1380         for(unsigned int j=b+p1; j<=m; j++){
1381             NewKnotVector[j+r+1]=KnotVector1[j];
1382         }
1383     }
1384
1385     int i=b+p1-1;
1386     int k=b+p1+r;
1387
1388     for(int j=r; j>=0; j--){
1389         while(Insertions[j]<=KnotVector1[i]&&i>a){
1390             for(unsigned int row=0; row<=m;row++){
1391                 NewCtrlPts[k-p1-1].SetCol(row,CtrlPtsW[i-p1-1].GetCol(row));
1392             }
1393             NewKnotVector[k]=KnotVector1[i];
1394             k--;
1395             i--;
1396         }
1397     }
1398
1399     for(unsigned int row=0; row<=m;row++){
1400         NewCtrlPts[k-p1-1].SetCol(row,NewCtrlPts[k-p1].GetCol(row));
1401     }
1402
1403     for(unsigned int l=1; l<=p1; l++){
1404         int ind=k-p1+1;
1405         double alpha=NewKnotVector[k+1]-Insertions[j];
1406         if (alpha==0.0){
1407             //NewCtrlPts.SetCol(ind-1,NewCtrlPts.GetCol(ind));
1408             for(unsigned int row=0; row<=m;row++){
1409                 NewCtrlPts[ind-1].SetCol(row,NewCtrlPts[ind].GetCol(row));
1410             }
1411         }
1412         else{
1413             alpha=alpha/(NewKnotVector[k+1]-KnotVector1[i-p1+1]);
1414             for(unsigned int row=0; row<=m;row++){
1415                 std::vector<double> aux(dim);
1416                 for(unsigned int iter=0; iter<dim; iter++){
1417                     aux[iter]=alpha*NewCtrlPts[ind-1](iter,row)+(1.0f-alpha)*NewCtrlPts[ind](iter,row);
1418                 }
1419                 NewCtrlPts[ind-1].SetCol(row,aux);
1420             }
1421         }
1422     }
1423 }
1424 NewKnotVector[k]=Insertions[j];

```

```

1425     k--;
1426 }
1427
1428     KnotVector1=NewKnotVector;
1429     return NewCtrlPts;
1430 }
1431 else{
1432     int numCtrlPts1 = CtrlPtsW.size();
1433     int numCtrlPts2 = CtrlPtsW[0].GetNumCols() + r + 1;
1434     int a = FindSpan(KnotVector2, Insertions[0], p2, np2);
1435     int b = FindSpan(KnotVector2, Insertions[r], p2, np2) + 1;
1436     int m = np2 + p2 + 1;
1437
1438     std::vector<double> NewKnotVector(m + r + 2);
1439     std::vector<iMatrix<double>> NewCtrlPts(numCtrlPts1);
1440
1441     for (int i = 0; i < numCtrlPts1; i++) {
1442         NewCtrlPts[i] = iMatrix<double>(dim, numCtrlPts2);
1443     }
1444
1445     for (int col = 0; col <= a - p2; col++) {
1446         for (int i = 0; i < numCtrlPts1; ++i) {
1447             NewCtrlPts[i].SetCol(col, CtrlPtsW[i].GetCol(col));
1448         }
1449     }
1450
1451     for (int col = b - 1; col <= np2; col++) {
1452         for (int i = 0; i < numCtrlPts1; i++) {
1453             NewCtrlPts[i].SetCol(col + r + 1, CtrlPtsW[i].GetCol(col));
1454         }
1455     }
1456
1457     for (int j = 0; j <= a; j++) {
1458         NewKnotVector[j] = KnotVector2[j];
1459     }
1460
1461     for (int j = b + p2; j <= m; j++) {
1462         NewKnotVector[j + r + 1] = KnotVector2[j];
1463     }
1464
1465     int i = b + p2 - 1;
1466     int k = b + p2 + r;
1467
1468     for (int j = r; j >= 0; j--) {
1469         while (Insertions[j] <= KnotVector2[i] && i > a) {
1470             for (int row = 0; row < numCtrlPts1; ++row) {
1471                 NewCtrlPts[row].SetCol(k - p2 - 1, CtrlPtsW[row].GetCol(i - p2 - 1));
1472             }
1473             NewKnotVector[k] = KnotVector2[i];
1474             k--;
1475             i--;
1476         }
1477
1478         for (int row = 0; row < numCtrlPts1; row++) {
1479             NewCtrlPts[row].SetCol(k - p2 - 1, NewCtrlPts[row].GetCol(k - p2));
1480         }
1481
1482         for (int l = 1; l <= p2; l++) {
1483             int ind = k - p2 + l;
1484             double alpha = NewKnotVector[k + l] - Insertions[j];
1485             if (alpha == 0.0f) {
1486                 for (int row = 0; row < numCtrlPts1; row++) {
1487                     NewCtrlPts[row].SetCol(ind - 1, NewCtrlPts[row].GetCol(ind));
1488                 }
1489             } else {
1490                 alpha = alpha / (NewKnotVector[k + l] - KnotVector2[i - p2 + l]);
1491                 for (int row = 0; row < numCtrlPts1; row++) {
1492                     std::vector<double> aux(dim);
1493                     for (int iter = 0; iter < dim; iter++) {
1494                         aux[iter] = alpha * NewCtrlPts[row](iter, ind - 1) + (1.0f - alpha) * NewCtrlPts[row](iter, ind);
1495                     }
1496                     NewCtrlPts[row].SetCol(ind - 1, aux);
1497                 }
1498             }
1499         }
1500
1501         NewKnotVector[k] = Insertions[j];
1502         k--;
1503     }
1504
1505     KnotVector2 = NewKnotVector;
1506     return NewCtrlPts;
1507 }
1508 }
1509
1510 }
1511
1512 /*****
1513 *
1514 * Functions for the Assembly of the Matrix
1515 *
1516 *****/
1517 void legendre_pol(int n, Eigen::VectorXd &nodes, Eigen::VectorXd &weights) {
1518
1519     Eigen::VectorXd beta(n-1);
1520     for (int i = 0; i < n-1; ++i) {
1521         double k = i + 1;
1522         beta(i) = k / std::sqrt(4.0 * k * k - 1);
1523     }

```

```

1524 Eigen::MatrixXd J = Eigen::MatrixXd::Zero(n, n);
1525 for (int i = 0; i < n-1; ++i) {
1526     J(i, i+1) = beta(i);
1527     J(i+1, i) = beta(i);
1528 }
1529
1530 Eigen::SelfAdjointEigenSolver<Eigen::MatrixXd> solver(J);
1531 nodes = solver.eigenvalues();
1532
1533 Eigen::MatrixXd V = solver.eigenvectors();
1534 weights.resize(n);
1535 for (int i = 0; i < n; ++i) {
1536     weights(i) = 2.0 * V(0, i) * V(0, i);
1537 }
1538 }
1539
1540
1541
1542 /***** */
1543
1544 /***** */
1545 std::vector<double> removeDuplicates(const std::vector<double>& input) {
1546     std::vector<double> result = input;
1547     auto last = std::unique(result.begin(), result.end());
1548     result.erase(last, result.end());
1549     return result;
1550 }
1551
1552 /***** */
1553
1554 /***** */
1555 std::vector<double> createSubIntervals(const double nElements, const double lower_limit, const double upper_limit){
1556     std::vector<double> sol(nElements+1);
1557     double Lenght = upper_limit - lower_limit;
1558     double Step=Lenght/nElements;
1559     for(unsigned int i=0; i<nElements; i++){
1560         sol[i]=lower_limit + i*Step;
1561     }
1562     return sol;
1563 }
1564
1565 /***** */
1566
1567 /***** */
1568 std::vector<double> removeMatches(const std::vector<double>& v1, const std::vector<double>& v2 ) {
1569     if (v1.empty()) return {};
1570
1571     double start = v1.front();
1572     double step = v1.size() > 1 ? v1[1] - v1[0] : 1;
1573
1574     std::vector<char> toRemove(v1.size(), 0);
1575
1576     for (size_t i = 0; i < v2.size(); ++i) {
1577         double val = v2[i];
1578         if (val < start) continue;
1579
1580         double diff = val - start;
1581         double idxReal = diff / step;
1582         long long idx = llround(idxReal);
1583
1584         if (std::fabs(idxReal - idx) > 1e-9) continue;
1585
1586         if (idx >= 0 && static_cast<size_t>(idx) < v1.size()) {
1587             toRemove[idx] = 1;
1588         }
1589     }
1590
1591     std::vector<double> result;
1592     result.reserve(v1.size());
1593     for (size_t i = 0; i < v1.size(); ++i) {
1594         if (!toRemove[i]) result.push_back(v1[i]);
1595     }
1596
1597     return result;
1598 }
1599
1600
1601 /***** */
1602
1603 /***** */
1604 void D1_element_eval(int n, int i, int p, int nEvals, double lower_limit, double upper_limit, const std::vector<double> &KnotVector,
1605     const std::vector<double> &Weights, const Eigen::VectorXd &nodes, const iMatrix<double> &CtrlPtsW, std::function<double(double)> f,
1606     double &preEvalPoint, double &cumLenght, iMatrix<double> &BasisFunsEvals, iMatrix<double> &DerBasisFunsEvals,
1607     std::vector<double> &JacobianEvals, std::vector<double> &InverseJacobianEvals, std::vector<double> &funcEvals, double Lenght){
1608
1609     BasisFunsEvals.Resize(p+1,nEvals);
1610     DerBasisFunsEvals.Resize(p+1,nEvals);
1611     JacobianEvals.resize(nEvals);
1612     InverseJacobianEvals.resize(nEvals);
1613     funcEvals.resize(nEvals);
1614     double auxeval1= (upper_limit - lower_limit)/2;
1615     double auxeval2= (upper_limit + lower_limit)/2;
1616
1617     for(unsigned int k=0; k<nEvals; k++){
1618         //std::cout << k << std::endl;
1619         double EvalPoint = auxeval1*nodes(k)+auxeval2;
1620         //std::cout <<"EvalPoint: " << EvalPoint<< std::endl;
1621         iMatrix<double> AuxMat1 = RatDersBasisFuns(i,EvalPoint, p, 1, KnotVector, Weights);
1622         BasisFunsEvals.SetCol(k, AuxMat1.GetRow(0));

```

```

1623 DerBasisFunsEvals.SetCol(k, AuxMat1.GetRow(1));
1624 //std::cout << "i1" << std::endl;
1625 iMatrix<double> Aders, wders;
1626 iMatrix<double> Allders = CurveDerivsAlg1(n, p, KnotVector, CtrlPtsW, EvalPoint, 1);
1627 //std::cout << "i2" << std::endl;
1628 int auxInt=Allders.GetNumCols()-1;
1629 Aders = Allders.GetSubMat(0,1,0, auxInt-1);
1630 wders = Allders.GetSubMat(0,1,auxInt, auxInt);
1631 iMatrix<double> AuxMat2 = RatCurveDerivs(Aders, wders, 1);
1632 //std::cout << "i3" << std::endl;
1633
1634 std::vector<double> AuxVec= AuxMat2.GetRow(1);
1635 double AuxVal=0;
1636 for(unsigned int iter=0; iter< AuxVec.size(); iter++){
1637     AuxVal+= AuxVec[iter]*AuxVec[iter];
1638 }
1639 AuxVal=sqrt(AuxVal);
1640 //std::cout << "i4" << std::endl;
1641 JacobianEvals[k]=AuxVal;
1642 InverseJacobianEvals[k]=1/AuxVal;
1643 double FisicalP=Fisical_to_Parametric(preEvalPoint, EvalPoint, 0, Lenght, 20, CtrlPtsW, KnotVector, n, p);
1644 cumLenght += FisicalP;
1645 preEvalPoint=EvalPoint;
1646 //std::cout <<std::setprecision(7) << "EvalPoint= " << cumLenght << std::endl;
1647 //funcEvals[k]=f(EvalPoint)/(M_PI *M_PI ); //Case Jacobian is constant at 1
1648 //funcEvals[k] = 10*10*M_PI*M_PI*sin(10*M_PI*CurvePointRational(n, p, KnotVector, CtrlPtsW, EvalPoint)[0]);
1649 //funcEvals[k] = f(CurvePointRational(n, p, KnotVector, CtrlPtsW, EvalPoint)[0]); //Case unidimensional curve
1650 funcEvals[k]= f(cumLenght)/(M_PI*M_PI); //Generic case (optimization needed)
1651 }
1652 }
1653
1654
1655 /***** */
1656
1657 /***** */
1658 iMatrix<double> gauss_legendre_cuadrature_integral_bilinealForm(int p, double lower_limit, double upper_limit, int nEvals,
1659     const iMatrix<double> &DerBasisFunsEvals,const std::vector<double>& InverseJacobianEvals,const Eigen::VectorXd &weights){
1660     iMatrix<double> sol(p+1,p+1);
1661     double AuxConstant = (upper_limit-lower_limit)/2;
1662
1663     for(unsigned int i=0; i<=p; i++){
1664         for(unsigned int j=0; j<=p; j++){
1665             double AuxVal=0;
1666             for(unsigned int iter=0; iter<nEvals; iter++){
1667                 AuxVal+=weights[iter]*DerBasisFunsEvals(i, iter)*DerBasisFunsEvals(j,iter)*InverseJacobianEvals[iter];
1668             }
1669             AuxVal=AuxConstant*AuxVal;
1670
1671             sol(i,j)=AuxVal;
1672             //sol(j,i,AuxVal);
1673         }
1674     }
1675     return sol;
1676 }
1677
1678 /***** */
1679
1680 /***** */
1681 std::vector<double> gauss_legendre_cuadrature_integral_linealForm(int p, double lower_limit, double upper_limit,
1682     int nEvals,const iMatrix<double> &BasisFunsEvals,const std::vector<double>& JacobianEvals,
1683     const std::vector<double>& funcEvals,const Eigen::VectorXd &weights){
1684     std::vector<double> sol(p+1);
1685     double AuxConstant = (upper_limit-lower_limit)/2;
1686
1687     for(unsigned int i=0; i<=p; i++){
1688         double AuxVal=0;
1689         for(unsigned int iter=0; iter<nEvals; iter++){
1690             AuxVal+=weights[iter]*BasisFunsEvals(i, iter)*JacobianEvals[iter]*funcEvals[iter];
1691         }
1692         sol[i]=AuxConstant*AuxVal;
1693     }
1694     return sol;
1695 }
1696
1697 /***** */
1698
1699 /***** */
1700
1701 void impose_Dirichlet_Condition(int p, int size, Eigen::SparseMatrix<double> &global, Eigen::VectorXd &LinearForm,
1702     bool Modify0, bool Modify1, double ValueAt0, double ValueAt1){
1703     if(Modify0){
1704
1705         global.coeffRef(0, 0) = 1.0;
1706         LinearForm(0)=ValueAt0;
1707         for(unsigned int i=1; i<=p; i++){
1708             global.coeffRef(i, 0) = 0.0;
1709             LinearForm(i)-=ValueAt0*global.coeffRef(0, i);
1710             global.coeffRef(0, i) = 0.0;
1711         }
1712     }
1713
1714     if(Modify1){
1715
1716         global.coeffRef(size, size) = 1.0;
1717         LinearForm(size)=ValueAt1;
1718         for(int i=size-1; i>=size-p; i--){
1719             global.coeffRef(size, i) = 0.0;
1720             LinearForm(i)-=ValueAt1*global.coeffRef(i, size);
1721             global.coeffRef(i, size) = 0.0;
1722         }
1723     }
1724 }

```

```

1722     }
1723 }
1724
1725 /***** */
1726
1727 /***** */
1728
1729 void impose_Newmann_Condition(int size, Eigen::VectorXd &LinearForm, bool Modify0, bool Modify1, double ValueAt0, double ValueAt1){
1730     if (Modify0){
1731         LinearForm(0) -= ValueAt0;
1732     }
1733     if (Modify1){
1734         LinearForm(size) += ValueAt1;
1735     }
1736 }
1737
1738 /***** */
1739
1740 /***** */
1741 void impose_Robin_Condition(int p, int size, Eigen::SparseMatrix<double> &global, Eigen::VectorXd &LinearForm,
1742     double alpha, double beta, bool Modify0, bool Modify1, double ValueAt0, double ValueAt1){
1743
1744     double BetaInverse=1/beta;
1745
1746     if (Modify0){
1747
1748         global.coeffRef(0, 0) -= alpha/beta;
1749         LinearForm(0) -= ValueAt0/beta;
1750     }
1751
1752     if (Modify1){
1753
1754         global.coeffRef(size, size) += alpha/beta;
1755         LinearForm(0) += ValueAt1/beta;
1756     }
1757 }
1758
1759 /***** */
1760
1761 /***** */
1762 void writeNumericFunction(Eigen::VectorXd funEvals, const std::vector<double> &KnotVector, const std::vector<double> &WeightVector, int n,
1763     int p, int nEvals, double lowerLimit, double upperLimit, std::string name, std::vector<double> Intervals, int nElements, double Lenght){
1764
1765     std::ofstream fichero1, fichero2;
1766     fichero1.open(name+"_h=" + std::to_string(static_cast<int>(nElements)) + "_p=" + std::to_string(p) + "_testCircle.txt");
1767     fichero2.open(name+ "Ders_h=" + std::to_string(static_cast<int>(nElements)) + "_p=" + std::to_string(p) + "_testCircle.txt");
1768
1769     // Configurar alta precisión
1770     fichero1 << std::scientific << std::setprecision(15);
1771     fichero2 << std::scientific << std::setprecision(15);
1772     Eigen::VectorXd nodes, weights;
1773     legendre_pol(nEvals, nodes, weights);
1774
1775     for(unsigned int iter=0; iter<Intervals.size()-1; iter++){
1776
1777         double auxeval1= (Intervals[iter+1] - Intervals[iter])/2;
1778         double auxeval2= (Intervals[iter+1] + Intervals[iter])/2;
1779         //double preVal=Intervals[iter];
1780
1781
1782         for(unsigned int i=0; i<nEvals; i++){
1783             double t=0.5*(auxeval1)*nodes(i)+auxeval2;
1784
1785             int span= FindSpan(KnotVector, t, p, n);
1786             //std::cout << "span: " << span << std::endl;
1787             double AuxVal1=0;
1788             double AuxVal2=0;
1789             //std::vector<double> eval= RatDersBasisFuns(span, t, p, 0, KnotVector, WeightVector).GetRow(0);
1790             iMatrix<double> evalMat= RatDersBasisFuns(span, t, p, 1, KnotVector, WeightVector);
1791             std::vector<double> eval1=evalMat.GetRow(0);
1792             std::vector<double> eval2=evalMat.GetRow(1);
1793             //std::cout << "t=" << t << std::endl;
1794             for(int j=0; j<p; j++){
1795                 //std::cout << "funEvals[" << span-p+j << "]=" << funEvals[span-p+j] << std::endl;
1796                 //std::cout << "eval[" << j << "]=" << eval[j] << std::endl;
1797                 AuxVal1+=funEvals[span-p+j]*eval1[j];
1798                 AuxVal2+=funEvals[span-p+j]*eval2[j]*Lenght;
1799             }
1800             fichero1 << AuxVal1 << " ";
1801             fichero2 << AuxVal2 << " ";
1802         }
1803     }
1804     fichero1.close();
1805     fichero2.close();
1806 }
1807
1808 /***** */
1809
1810 /***** */
1811 void writeAnalyticFunction(std::function<double(double)> f, std::function<double(double)> f_Ders, std::vector<double> Intervals,
1812     double nEvals, const iMatrix<double> &CtrlPtsW, const std::vector<double> &KnotVector, int n, int p,

```

```

1821     std::string name, double Lenght, double nElements){
1822
1823     std::ofstream fichero1, fichero2;
1824     fichero1.open(name+ "_h=" + std::to_string(static_cast<int>(nElements)) + "_p=" + std::to_string(p) + ".Analytic_testCircle.txt");
1825     fichero2.open(name+ "_h=" + std::to_string(static_cast<int>(nElements)) + "_p=" + std::to_string(p) + ".Ders_Analytic_testCircle.txt");
1826
1827     // Configurar alta precisión
1828     fichero1 << std::scientific << std::setprecision(15);
1829     fichero2 << std::scientific << std::setprecision(15);
1830     double preVal=Intervals[0];
1831     double cumLenght=0;
1832     Eigen::VectorXd nodes, weights;
1833     legendre_pol(nEvals, nodes, weights);
1834
1835     for(unsigned int iter=0; iter<Intervals.size()-1; iter++){
1836
1837
1838         double auxeval1= (Intervals[iter+1] - Intervals[iter])/2;
1839         double auxeval2= (Intervals[iter+1] + Intervals[iter])/2;
1840         //double preVal=Intervals[iter];
1841
1842
1843         for(unsigned int i=0; i<nEvals; i++){
1844             iMatrix<double> Aders, wders;
1845             double t=0.5*(auxeval1)*nodes(i)+auxeval2;
1846             iMatrix<double> Allders = CurveDerivsAlg1(n, p, KnotVector, CtrlPtsW, t, 1);
1847             //std::cout << "i2" << std::endl;
1848             int auxInt=Allders.GetNumCols()-1;
1849             Aders = Allders.GetSubMat(0,1,0, auxInt-1);
1850             wders = Allders.GetSubMat(0,1,auxInt, auxInt);
1851             iMatrix<double> AuxMat2 = RatCurveDerivs(Aders, wders, 1);
1852             //std::cout << "i3" << std::endl;
1853
1854             std::vector<double> AuxVec= AuxMat2.GetRow(1);
1855             double Jacobian=0;
1856             for(unsigned int i=0; i< AuxVec.size(); i++){
1857                 Jacobian+= AuxVec[i]*AuxVec[i];
1858             }
1859             Jacobian=sqrt(Jacobian);
1860             double FisicalP=Fisical_to_Parametric(preVal, t, 0, Lenght, 20, CtrlPtsW, KnotVector, n, p);
1861             cumLenght+=FisicalP;
1862             //std::cout << "analytic = " << Fisical_to_Parametric(auxeval1*nodes(i)+auxeval2, 0, Lenght, 50, CtrlPtsW, KnotVector, n, p)<< " ,EvalPoint= "<< auxeval
1863             //double auxval = CurvePointRational(n, p, KnotVector, CtrlPtsW, lowerLimit + i*Step)[0];
1864             //std::cout << "point is: "<< lowerLimit + (auxeval1*nodes(i)+auxeval2) << std::endl;
1865             //fichero << f(auxeval1*nodes(i)+auxeval2) << " "; //case Jacobian is constant at value 1
1866             //fichero << f(CurvePointRational(n, p, KnotVector, CtrlPtsW, auxeval1*nodes(i)+auxeval2)[0]) << " "; //case jacobian is not constant but f only takes 1
1867             fichero1 << f(cumLenght)<< " "; //optimization needed, not necessary to compute the lagranje pol pts in each iteration
1868             fichero2 << f_Ders(cumLenght)*Jacobian<< " ";
1869             preVal=t;
1870             //fichero1 << f(auxeval1*nodes(i)+auxeval2)<< " "; //optimization needed, not necessary to compute the lagranje pol pts in each iteration
1871             //fichero2 << f_Ders(auxeval1*nodes(i)+auxeval2)*Jacobian<< " ";
1872             //std::cout << CurvePointRational(n, p, KnotVector, CtrlPtsW, time[i])[0] << std::endl;
1873             //fichero1 << f(CurvePointRational(n, p, KnotVector, CtrlPtsW, time[i])[0]) << " ";
1874             //fichero2 << f_Ders(CurvePointRational(n, p, KnotVector, CtrlPtsW, time[i])[0])*Jacobian << " ";
1875
1876         }
1877     }
1878     fichero1.close();
1879     fichero2.close();
1880     std::cout << "Longitud total arco recorrida: " << cumLenght << std::endl;
1881 }
1882
1883
1884 /*****
1885
1886 /*****
1887 std::vector<std::vector<double>> leerArchivo(const std::string& name, int p) {
1888     std::vector<std::vector<double>> resultado;
1889     std::ifstream archivo(name);
1890
1891     if (!archivo.is_open()) {
1892         std::cerr << "Error: No se pudo abrir el archivo " << name << std::endl;
1893         return resultado;
1894     }
1895
1896     std::string linea;
1897     std::vector<double> filaActual;
1898
1899     std::stringstream buffer;
1900     buffer << archivo.rdbuf();
1901     std::string contenido = buffer.str();
1902
1903     std::istringstream ss(contenido);
1904     double numero;
1905
1906     while (ss >> numero) {
1907         filaActual.push_back(numero);
1908
1909         if (filaActual.size() == p) {
1910             resultado.push_back(filaActual);
1911             filaActual.clear();
1912         }
1913     }
1914
1915     if (!filaActual.empty()) {
1916         resultado.push_back(filaActual);
1917     }
1918
1919     archivo.close();

```

```

1920     return resultado;
1921 }
1922 /*****
1923
1924 /*****
1925 */
1926 void writeFunctionDers(Eigen::VectorXd funEvals,const std::vector<double> &KnotVector,const iMatrix<double> &CtrlPtsW ,const std::vector<double> &WeightVector, int n
1927 {
1928     std::ofstream fichero;
1929     fichero.open(name);
1930     Eigen::VectorXd nodes, weights;
1931     legendre_pol(nEvals, nodes, weights);
1932     double auxeval1= (upperLimit - lowerLimit)/2;
1933     double auxeval2= (upperLimit + lowerLimit)/2;
1934
1935     //double Step=(upperLimit-lowerLimit)/(funEvals.size()-p);
1936     for(unsigned int i=0; i<nEvals; i++){
1937         double t=auxeval1*nodes(i)+auxeval2;
1938         //std::cout << "i4" << std::endl;
1939
1940         int span= FindSpan(KnotVector, t, p, n);
1941         double AuxVal=0;
1942         std::vector<double> eval= RatDersBasisFuns(span, t , p, 1, KnotVector, WeightVector).GetRow(1);
1943         for(int j=0; j<p; j++){
1944             AuxVal+=funEvals[span-p+j]*eval[j];
1945         }
1946
1947         fichero << AuxVal << " ";
1948     }
1949     fichero.close();
1950 }
1951 /****
1952 /*****
1953
1954 /*****
1955 */
1956 void writeFunctionDers(std::function<double(double)> f, double lowerLimit, double upperLimit, double nEvals,const iMatrix<double> &CtrlPtsW,const std::vector<double>
1957     int n, int p, std::string name, double Lenght){
1958     std::ofstream fichero;
1959     fichero.open(name);
1960     Eigen::VectorXd nodes, weights;
1961     legendre_pol(nEvals, nodes, weights);
1962     double auxeval1= (upperLimit - lowerLimit)/2;
1963     double auxeval2= (upperLimit + lowerLimit)/2;
1964     for(unsigned int i=0; i<nEvals; i++){
1965         double t= auxeval1*nodes(i)+auxeval2;
1966         iMatrix<double> Aders, wders;
1967         iMatrix<double> Allders = CurveDerivsAlg1(n, p, KnotVector, CtrlPtsW, t, 1);
1968         //std::cout << "i2" << std::endl;
1969         int auxInt=Allders.GetNumCols()-1;
1970         Aders = Allders.GetSubMat(0,1,0, auxInt-1);
1971         wders = Allders.GetSubMat(0,1,auxInt, auxInt);
1972         iMatrix<double> AuxMat2 = RatCurveDerivs(Aders, wders, 1);
1973         //std::cout << "i3" << std::endl;
1974
1975         std::vector<double> AuxVec= AuxMat2.GetRow(1);
1976         double Jacobian=0;
1977         for(unsigned int i=0; i< AuxVec.size(); i++){
1978             Jacobian+= AuxVec[i]*AuxVec[i];
1979         }
1980         Jacobian=sqrt(Jacobian);
1981         //double auxVal = f(CurvePointRational(n, p, KnotVector, CtrlPtsW, t)[0])*Jacobian; //Case curve is 1 dimensional
1982         //std::cout << "tders = " << Fisical_to_Parametric(t, 0, Lenght, p+2, CtrlPtsW, KnotVector, n, p) << std::endl;
1983         double auxVal = f(Fisical_to_Parametric(t, 0, Lenght, 20, CtrlPtsW, KnotVector, n, p))*Jacobian; //Generic case
1984
1985         //double auxval = CurvePointRational(n, p, KnotVector, CtrlPtsW, lowerLimit + i*Step)[0];
1986         //std::cout << "point is: "<< lowerLimit + (auxeval1*nodes(i)+auxeval2) << std::endl;
1987
1988         //fichero << f(auxeval1*nodes(i)+auxeval2) << " "; //case Jacobian is constant at value 1
1989         fichero << auxVal << " "; //case jacobian is not constant but f only takes 1 dimensional parameters
1990     }
1991
1992     fichero.close();
1993 }
1994 /****
1995
1996 /*****
1997
1998 /*****
1999
2000 double IntegrateNormDer(double lowerLimit, double upperLimit, int nEvals,const iMatrix<double> &CtrlPtsW,
2001     const std::vector<double> &KnotVector, int n, int p){
2002     Eigen::VectorXd nodes, weights;
2003     legendre_pol(nEvals, nodes, weights);
2004     double auxeval1= (upperLimit - lowerLimit)/2;
2005     double auxeval2= (upperLimit + lowerLimit)/2;
2006     double sol=0;
2007
2008     for(unsigned int i=0; i<nEvals; i++){
2009
2010         double t= auxeval1*nodes(i)+auxeval2;
2011         //std::cout << "t= " << t << std::endl;
2012         iMatrix<double> Aders, wders;
2013         //std::cout << "i0" << std::endl;
2014         iMatrix<double> Allders = CurveDerivsAlg1(n, p, KnotVector, CtrlPtsW, t, 1);
2015         //std::cout << "i1" << std::endl;
2016         //Allders.PrintMatrix();
2017         //std::cout << "i2" << std::endl;
2018         int auxInt=Allders.GetNumCols()-1;
2019         Aders = Allders.GetSubMat(0,1,0, auxInt-1);

```

```

2019     wders = Allders.GetSubMat(0,1,auxInt, auxInt);
2020     iMatrix<double> AuxMat2 = RatCurveDerivs(Aders, wders, 1);
2021     //std::cout << "i3" << std::endl;
2022
2023     std::vector<double> AuxVec= AuxMat2.GetRow(1);
2024     double Jacobian=0;
2025     for(unsigned int iter=0; iter< AuxVec.size(); iter++){
2026         Jacobian+= AuxVec[iter]*AuxVec[iter];
2027     }
2028     Jacobian=sqrt(Jacobian);
2029     sol+=Jacobian*weights[i];
2030
2031     //double auxval = CurvePointRational(n, p, KnotVector, CtrlPtsW, lowerLimit + i*Step)[0];
2032     //std::cout << "point is: " << lowerLimit + (auxeval1*nodes(i)+auxeval2) << std::endl;
2033     //fichero << f(auxeval1*nodes(i)+auxeval2) << " "; //case Jacobian is constant at value 1
2034
2035 }
2036
2037 return sol*auxeval1;
2038 }
2039
2040 /***** */
2041
2042 /***** */
2043 std::vector<std::vector<iMatrix<double>>> PreBasis_and_Ders(int p, int k, std::vector<double> KnotVector,
2044 std::vector<double> Intervals,Eigen::VectorXd nodes, std::vector<int>& spanIndex){
2045     int nIntervals =Intervals.size()-1;
2046     int nEvals=nodes.size();
2047     int n= KnotVector.size()-p-2;
2048     std::vector<std::vector<iMatrix<double>>> sol(nIntervals);
2049     for(unsigned int i=0; i< nIntervals; i++){
2050         double lower_limit=Intervals[i];
2051         double upper_limit=Intervals[i+1];
2052         double auxeval1= (upper_limit - lower_limit)/2;
2053         double auxeval2= (upper_limit + lower_limit)/2;
2054         int span= FindSpan(KnotVector, auxeval2, p, n);
2055         spanIndex[i]=span;
2056         std::vector<iMatrix<double>> auxVec(nEvals);
2057         for(unsigned int j=0; j<nEvals; j++){
2058             double EvalPoint = auxeval1*nodes(j)+auxeval2;
2059
2060             //std::cout <<"EvalPoint: " << EvalPoint<< std::endl;
2061             auxVec[j] = DerBasisFuns(span,EvalPoint, p, k, KnotVector);
2062         }
2063         sol[i]=auxVec;
2064     }
2065     return sol;
2066 }
2067
2068 /***** */
2069
2070 /***** */
2071 std::vector<iMatrix<double>> RationalizeDersBasisFunsVec(int span, int nEvals, Eigen::VectorXd nodes,
2072 double lower_limit, double upper_limit,int p, int k, int n, std::vector<double> KnotVector,
2073 std::vector<iMatrix<double>> Basis_and_DersY,std::vector<double> Weights){
2074     std::vector<iMatrix<double>> sol(nEvals);
2075     double auxeval1= (upper_limit - lower_limit)/2;
2076     double auxeval2= (upper_limit + lower_limit)/2;
2077     for(unsigned int iter=0; iter<nEvals; iter++){
2078         double EvalPoint = auxeval1*nodes(iter)+auxeval2;
2079         sol[iter]=RationalizeDersBasisFuns(span, auxeval1*nodes(iter)+auxeval2, p, k, KnotVector, Basis_and_DersY[iter], Weights);
2080     }
2081     return sol;
2082 }
2083
2084 /***** */
2085
2086 /***** */
2087 std::unordered_map<int, std::vector<std::pair<double,double>>> BuildSpanMap(const std::vector<double>& Intervals, const std::vector<int>& spanIndices){
2088     std::unordered_map<int, std::vector<std::pair<double,double>>> map;
2089     size_t nIntervals = spanIndices.size(); // debe ser nodes.size() - 1
2090
2091     for (size_t i = 0; i < nIntervals; ++i) {
2092         int span = spanIndices[i];
2093         std::pair<double,double> interval = { Intervals[i], Intervals[i+1] }; // extremos reales
2094         map[span].push_back(interval);
2095     }
2096
2097     return map;
2098 }
2099
2100 /***** */
2101
2102 /***** */
2103 iMatrix<iMatrix<double>> RationalizeDersBasisFuns2D(const iMatrix<double> &DersBasisX,const iMatrix<double> &DersBasisY,
2104 const iMatrix<double> &activeWeights, unsigned int pX, unsigned int pY) {
2105     // Matriz de salida 2x2 de matrices
2106     iMatrix<iMatrix<double>> Rationalized(2, 2);
2107
2108     // Inicializamos las submatrices (pY+1)x(pX+1)
2109     for (int i = 0; i < 2; ++i){
2110         for (int j = 0; j < 2; ++j){
2111             Rationalized(i, j).Resize(pY + 1, pX + 1);
2112         }
2113     }
2114
2115     // Calcular la superficie de pesos y sus derivadas
2116     double W = 0.0;
2117     double Wx = 0.0;
2118     double Wy = 0.0;

```



```

2118     for (unsigned int j = 0; j <= pY; ++j)
2119     {
2120         for (unsigned int i = 0; i <= pX; ++i)
2121         {
2122             double w = activeWeights(j, i);
2123             double Nx = DersBasisX(0, i);
2124             double Ny = DersBasisY(0, j);
2125             double Nx_x = DersBasisX(1, i);
2126             double Ny_y = DersBasisY(1, j);
2127
2128             W += Nx * Ny * w;
2129             Wx += Nx_x * Ny * w;
2130             Wy += Nx * Ny_y * w;
2131         }
2132     }
2133
2134     // Racionalización de funciones base y derivadas
2135     for (unsigned int j = 0; j <= pY; ++j)
2136     {
2137         for (unsigned int i = 0; i <= pX; ++i)
2138         {
2139             double w = activeWeights(j, i);
2140             double Nx = DersBasisX(0, i);
2141             double Ny = DersBasisY(0, j);
2142             double Nx_x = DersBasisX(1, i);
2143             double Ny_y = DersBasisY(1, j);
2144
2145             double N = Nx * Ny * w;
2146
2147             // Función base racionalizada
2148             Rationalized(0, 0)(j, i) = N / W;
2149
2150             // Derivada respecto a X
2151             Rationalized(1, 0)(j, i) = w * (Nx_x * Ny * W - Nx * Ny * Wx) / (W * W);
2152
2153             // Derivada respecto a Y
2154             Rationalized(0, 1)(j, i) = w * (Nx * Ny_y * W - Nx * Ny * Wy) / (W * W);
2155         }
2156     }
2157
2158     return Rationalized;
2159 }
2160
2161 /***** */
2162 /***** */
2163
2164 void ExportToTxt(const iMatrix<double>& SolEvals, const iMatrix<std::vector<double>>& Surf, const std::string& filename){
2165     std::ofstream file(filename);
2166     if (!file.is_open()) {
2167         std::cerr << "Error abriendo archivo para escritura: " << filename << std::endl;
2168         return;
2169     }
2170
2171     size_t nx = SolEvals.GetNumRows();
2172     size_t ny = SolEvals.GetNumCols();
2173     //std::cout << "1";
2174     for (size_t i = 0; i < nx; ++i) {
2175         //std::cout << "2";
2176         for (size_t j = 0; j < ny; ++j) {
2177
2178             const auto& xyz = Surf(i, j);
2179             //std::cout << xyz.size();
2180             double val = SolEvals(i, j);
2181             file<< std::setprecision(std::numeric_limits<double>::max_digits10) << xyz[0] << " " << xyz[1] << " " << xyz[2] << " " << val << "\n";
2182         }
2183     }
2184
2185     file.close();
2186     std::cout << "Datos exportados correctamente a " << filename << std::endl;
2187 }
2188
2189

```

### 6.3.4. Curvas de NURBS

#### Código del dibujo de las curvas de NURBS

```

1  RUTA_ARCHIVO_CURVA =
   ↪ 'C:/Users/carlo/OneDrive/Escritorio/Uni/TFG/DataFiles/Nurbs/Curvas/EjCurva3d.txt';
2  NOMBRE_ARCHIVO_SVG = 'CurvaNurbs_3D_Plot.svg';
3
4  PC = [
5      10, 0, 0;
6      15, 10, 5;
7      5, 15, 20;
8      0, 5, 10;
9      10, 0, 0
10 ];

```

```

11
12 try
13     D = load(RUTA_ARCHIVO_CURVA);
14 catch
15     error('ERROR: No se pudo cargar el archivo. Comprueba que la ruta sea correcta: %s',
16         ↪ RUTA_ARCHIVO_CURVA);
17 end
18 [M, N] = size(D);
19
20 C = D';
21
22 figure;
23 hold on;
24 grid on;
25
26
27 plot3(C(:,1), C(:,2), C(:,3), 'b-', 'LineWidth', 2);
28 plot3(PC(:,1), PC(:,2), PC(:,3), 'r--', 'LineWidth', 1, 'DisplayName', 'Polígono de Control');
29 plot3(PC(:,1), PC(:,2), PC(:,3), 'ro', 'MarkerSize', 4, 'MarkerFaceColor', 'r', 'DisplayName',
30     ↪ 'Puntos de Control');
31
32 min_coords = min([C; PC]);
33 max_coords = max([C; PC]);
34 margen = 2;
35 axis([min_coords(1)-margen max_coords(1)+margen ...
36     min_coords(2)-margen max_coords(2)+margen ...
37     min_coords(3)-margen max_coords(3)+margen]);
38
39 view(3);
40 set(gca, 'Box', 'on');
41 print(NOMBRE_ARCHIVO_SVG, '-dsvg');
42 disp(['Gráfico final guardado en: ', fullfile(pwd, NOMBRE_ARCHIVO_SVG)]);

```

### Código de las derivadas de las funciones base de NURBS

```

1 int p=3;
2 std::vector<double> Nurbs={0,0,0,0,0.5,1,1,1,1};
3 std::vector<double> weights={1,1,5,1,1};
4
5 double N_Steps = 1000;
6 std::vector<double> eval(Nurbs.size()-p);
7 iMatrix<double> ders(p+1,p+1);
8 iMatrix<double> sol(Nurbs.size()-p,N_Steps+1);
9 //std::vector<double> Nurb(NurbsVector.size());
10 double Step=1/(N_Steps);
11 auto start = std::chrono::high_resolution_clock::now();
12 for(unsigned int i = 0; i<N_Steps+1; i++){
13
14     double t=Step*i;
15     //std::cout << "Iteración iniciada " << i<< std::endl;
16     int span = FindSpan(Nurbs, t, p, Nurbs.size()-2-p);
17     ders = RatDersBasisFuns(span,t,p,1,Nurbs, weights);
18     ders.PrintMatrix();
19     std::cout<<"-----"<<std::endl;
20     for(int j = span-p; j<=span;j++){
21         if(j>=0) {
22             sol(j,i)=ders(1,j-span+p);
23         }
24         else{
25             //Caso error.
26         }
27     }

```

```

28     }
29 }
30
31 Write_BasisFunctions(sol, "path+name");
32 auto end = std::chrono::high_resolution_clock::now();
33 auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>(end - start);
34
35 std::cout << "Tiempo tomado: " << duration.count() << " nanosegundos\n";

```

### Código de las funciones base de NURBS

```

1  int p=3;
2  double N_Steps = 1000;
3  std::vector<double> KnotVector={0,0,0,0,0.5,1,1,1,1};
4  std::vector<double> WeightVector={1,1,5,1,1};
5  iMatrix<double> sol(KnotVector.size()-p-1,N_Steps+1);
6
7  double Step=1/(N_Steps);
8  for(unsigned int i = 0; i<N_Steps+1; i++){
9      double t=Step*i;
10     int span = FindSpan(KnotVector, t, p, KnotVector.size()-2-p);
11     std::vector<double> eval= RationalBasisFuns(span, t , p, KnotVector, WeightVector);
12     for(int j = span-p; j<=span;j++){
13         if(j>=0) {
14             sol(j,i)=eval[j-span+p];
15         }
16         else{
17             //Caso error.
18         }
19     }
20 }
21 Write_BasisFunctions(sol, "path+name");

```

## 6.3.5. Superficies de NURBS

### Código del dibujo de las superficies de NURBS

```

1  dataX = dlmread(
    ↪ 'C:\Users\carlo\OneDrive\Escritorio\Uni\TFG\DataFiles\Nurbs\Superficies\SuperficieRationalEj2X.txt'
    ↪ , ' ');
2  dataY = dlmread(
    ↪ 'C:\Users\carlo\OneDrive\Escritorio\Uni\TFG\DataFiles\Nurbs\Superficies\SuperficieRationalEj2Y.txt'
    ↪ , ' ');
3  dataZ = dlmread(
    ↪ 'C:\Users\carlo\OneDrive\Escritorio\Uni\TFG\DataFiles\Nurbs\Superficies\SuperficieRationalEj2Z.txt'
    ↪ , ' ');
4
5  hold on
6  surf(dataX, dataY, dataZ);
7
8  dataCtrlX =
    ↪ dlmread('C:\Users\carlo\OneDrive\Escritorio\Uni\TFG\DataFiles\Nurbs\CtrlPts\Ej2X.txt', ' ');
9  dataCtrlY =
    ↪ dlmread('C:\Users\carlo\OneDrive\Escritorio\Uni\TFG\DataFiles\Nurbs\CtrlPts\Ej2Y.txt', ' ');
10 dataCtrlZ =
    ↪ dlmread('C:\Users\carlo\OneDrive\Escritorio\Uni\TFG\DataFiles\Nurbs\CtrlPts\Ej2Z.txt', ' ');
11
12 [n1 n2] = size(dataCtrlX);
13

```

```

14 for i=1:n1
15     plot3(dataCtrlX(i,:),dataCtrlY(i,:),dataCtrlZ(i,:), color='r');
16 endfor
17
18 for j=1:n2
19     plot3(dataCtrlX(:,j),dataCtrlY(:,j),dataCtrlZ(:,j), color='r');
20 endfor
21
22 scatter3(dataCtrlX(:), dataCtrlY(:), dataCtrlZ(:), 30, 'k', 'filled', 'MarkerEdgeColor', 'w');
23 view(30, 45);
24 grid on;
25
26 print -dsvg 'Superficie_NURBS_Ej1.svg';
27 hold off;

```

### Código del cálculo de las superficies de NURBS

```

1  std::vector<iMatrix<double>> CtrlPts = ReadDataFile_CtrlPts("path+nameCtrlPts");
2  iMatrix<double> Weights =ReadDataFile_Matrix("path+nameWeights");
3  std::vector<iMatrix<double>> CtrlPtsW=WeightCtrlPtsSurf(CtrlPts,Weights);
4  std::vector<double> V1={0,0,0,0.5,0.5,1,1,1};
5  std::vector<double> V2={0,0,0,0,1,1,1,1};
6  int p1=2;
7  int p2=3;
8  double N_Steps_1 = 25;
9  double N_Steps_2 = 25;
10
11 double Step_1=1/N_Steps_1;
12 double Step_2=1/N_Steps_2;
13
14 std::vector<iMatrix<double>> eval(N_Steps_1+1);
15 for(unsigned int i=0; i<N_Steps_1+1;i++){
16
17     iMatrix<double> auxMatrix(3,N_Steps_2+1);
18     for(unsigned int j=0; j< N_Steps_2+1;j++){
19
20         std::vector<double> aux = SurfacePointRational(V1.size()-2-p1,p1,V1, V2.size()-2-p2,p2,V2, CtrlPtsW,i*Step_1);
21         for(unsigned int k=0; k<3;k++){
22             auxMatrix(k,j)=aux[k];
23         }
24     }
25     eval[i]=auxMatrix;
26 }
27
28 Write_BezierSurface(eval,"path+nameSurf" );
29 Write_CtrlPts(CtrlPts,"path+nameCtrlPts");

```